МОЕ ПОВТОРЕНИЕ ПРОЕКТА

Учебник для переписывания EH/Unwind Analyzer руками: архитектура, язык C, план файлов, проверки и упражнения.

Версия учебника: 2026-07-01. Проект: EH/Unwind Analyzer 1.0.0, report schema 1.1.0.

Договор с учеником

Эта книга написана для человека, который честно начинает почти с нуля. Твоя цель - не перепечатать код механически, а каждый раз понимать: какие bytes мы читаем, какой объект строим, кто владеет памятью, что проверяем и почему следующий модуль может доверять результату.

Главный маршрут проекта:

```
untrusted PE bytes
  -> util: безопасные bytes и строки
  -> pe: модель PE32+ AMD64 и перевод RVA в file offset
  -> unwind: .pdata / RUNTIME_FUNCTION / UNWIND_INFO
  -> heuristics: findings, score, confidence, verdict
  -> optional dynamic: Windows Debug API evidence
  -> report: console, CSV, JSON, HTML, Ghidra CSV
  -> main: CLI, владение ресурсами, exit codes
```

Как читать эту книгу

Не читай ее как роман. Читай как маршрут сборки. В каждом большом разделе есть три слоя: что понять, что написать руками, как проверить себя. Если ты пока не знаешь C, не перескакивай сразу к `unwind.c`: этот файл станет понятным только после `Buf`, integer widths, `struct`, pointer ownership и PE sections.

Содержание

1. Нулевая карта проекта и смысл задачи.
2. Мини-курс C ровно под этот проект.
3. Предметная область: PE, RVA, `.pdata`, `UNWIND_INFO`, отчеты.
4. Архитектура: файлы, модули, ownership, data flow.
5. Подробная архитектура блоков: ответственность, входы, выходы, взаимодействия.
6. Пошаговый план написания проекта руками.
7. Объясняющая карта файлов: зачем файл нужен, что в нем писать и как он работает.
8. Версионный маршрут: как собрать проект маленькими рабочими состояниями.
9. Подробный маршрут по каждому файлу.
10. Как проверять себя тестами и маленькими лабораторными.
11. Глоссарий и карта ошибок.

Правило одного маленького шага

Пиши маленькими кусками. В C очень легко получить ошибку, которая выглядит мистически, если ты изменил сразу 200 строк. Правильный учебный ритм: написать 10-30 строк, собрать, запустить маленький тест, только потом идти дальше. Проект большой, но он состоит из простых инженерных кирпичей.

Что значит 'разобраться в каждой строчке'

- Ты можешь назвать типы всех переменных в строке.
- Ты понимаешь, кто владеет памятью, на которую указывает pointer.
- Ты знаешь, какая проверка защищает от выхода за границы.
- Ты можешь объяснить, что случится при ошибочном input.
- Ты понимаешь, какая следующая функция использует результат этой строки.

1. Что это за проект

EH/Unwind Analyzer - защитный анализатор Windows PE32+ AMD64 exception/unwind metadata. Он не пытается запускать статический input и не делает дизассемблирование `.text` в static mode. Он читает bytes PE-файла, извлекает exception directory, строит таблицу функций, декодирует `UNWIND_INFO`, фиксирует структурные отклонения и пишет отчеты. Dynamic mode существует отдельно и на Windows действительно запускает target под debugger.

Главная идея

В Windows x64 stack unwinding и exception handling завязаны не только на инструкции CPU, но и на metadata. Эта metadata говорит системе, где начинается и заканчивается функция, как prolog менял stack pointer, какие регистры сохранялись, есть ли exception handler и есть ли chained unwind record. Если metadata повреждена или необычно размещена, reverse engineer получает сигнал: здесь нужно ручное внимание.

Дерево основных файлов

Файл	Строк	Первые функции/точки интереса
CMakeLists.txt	288	-
CMakePresets.json	113	-
include/util.h	73	le16, le32, le64
include/pe.h	68	-
include/unwind.h	115	-
include/heuristics.h	110	-
include/report.h	73	-
include/dynamic.h	123	-
include/app_info.h	25	-
src/util.c	349	buf_init_from_file, buf_free_owned, buf_read, buf_read_u8, buf_read_u16
src/pe.c	302	u32_max, is_power_of_two_u32, read_dos_lfanew, safe_add_u32, ranges_overlap_u64
src/unwind.c	682	unwind_flags_to_string, slots_for_code, parse_unwindinfo_header, read_u16_slot, read_u32_from_slots
src/heuristics.c	1045	sec_name_eq, sec_name_contains_ci, sec_name_allowed_for_unwind, sec_name_plausibly_unwind_named, sec_name_code_like
src/report.c	1364	append_regmask, anomalies_to_str, runtime_function_size, csv_cell, build_unwind_comment
src/dynamic_win.c	1254	mods_init, strlower_inplace, mods_add, mods_refresh_toolhelp, mods_refresh_accounted
src/app_info.c	69	app_utc_timestamp
src/main.c	528	print_version, usage, append_invocation_arg, build_invocation, parse_long_range
tests/unit/unit_unwind.c	515	make_fake_img_1sec, test_alloc_small, test_push_nonvol, test_codeoffset_order_and_operand_sanity, test_real_world_compatible_operands
tests/unit/unit_heuristics.c	82	setup_sections, main
tests/unit/unit_core_edges.c	284	finding_index, test_section_range_wrap, test_header_mapping_uses_declared_size, test_empty_exception_directory, test_malformed_exception_directory_status
tests/test_cli_contract.py	220	-

Архитектурный конвейер

```
main.c
  parse_args()
  buf_init_from_file()
  pe_parse()
  unwind_read_runtime_functions_ex()
  for each RuntimeFunction:
      unwind_parse()
  optional:
      heuristics_analyze()
      dynamic_collect_windows()
  optional outputs:
      report_write_csv()
      report_write_json_ex()
      report_write_html()
  cleanup and return exit code
```

Модули и ответственность

Модуль	За что отвечает	Что отдает следующему слою
util	Bytes, строки, file I/O, SHA-256	`Buf`, `StrBuf`, безопасные read helpers
pe	PE32+ AMD64 headers, sections, exception directory	`PeImage`, sections, RVA mapping
unwind	`.pdata`, `RUNTIME_FUNCTION`, `UNWIND_INFO`	`RuntimeFunction[]`, `UnwindSummary[]`
heuristics	Findings, score, confidence, verdict	`HeuristicReport`
dynamic	Runtime evidence через Windows Debug API	`DynamicReport`
report	CSV, JSON, HTML, console, Ghidra CSV	Файлы отчетов
app_info	Версии, compiler/platform/build mode, UTC	`ReportMetadata` поля
main	CLI и orchestration	Exit code и выбранные outputs

Самое важное архитектурное решение

Модули не должны читать мысли друг друга. `pe.c` не знает, как считать score. `heuristics.c` не читает файл напрямую. `report.c` не парсит PE. Такой дизайн позволяет учить проект блоками: каждый блок строит одну модель и передает ее дальше.

2. С с нуля, но без лишней воды

Этот раздел не заменяет полный учебник по С. Он дает ровно те понятия, которые будут встречаться в проекте. Если ты понимаешь этот раздел, исходники перестают быть стеной символов и превращаются в набор повторяющихся инженерных приемов.

Компилятор, linker и единица трансляции

С-файл компилируется отдельно. Header не является отдельной программой: он вставляется препроцессором в `.c`, который его `#include`-ит. Linker потом соединяет объектные файлы и проверяет, что объявленные функции действительно где-то определены. Поэтому в этом проекте `.h` описывает контракт, `.c` реализует контракт, а `CMakeLists.txt` говорит, какие `.c` участвуют в сборке.

Типы фиксированной ширины

`uint8_t`, `uint16_t`, `uint32_t`, `uint64_t` нужны, потому что PE и UNWIND_INFO задают поля в конкретных размерах. `int` может быть удобен для счетчиков, но для поля из файла лучше брать тип с точной шириной. Если спецификация говорит 'DWORD', обычно это 32 бита, значит в коде нужен `uint32_t`.

Указатель

Указатель хранит адрес. В проекте указатели часто означают 'я даю функции доступ к уже существующему объекту'. `PeImage* out` значит: функция заполнит объект, который создал вызывающий код. `const Buf* b` значит: функция читает буфер, но не должна менять сам `Buf` и его bytes.

```
bool pe_parse(PeImage* out, const Buf* b, StrBuf* err)
{
    // out: куда функция запишет результат
    // b: откуда функция читает bytes
    // err: куда функция добавит текст ошибки
}
```

Проверка `NULL`

В С нет автоматической защиты от обращения по нулевому указателю. Поэтому почти каждая публичная функция начинает с guard: `if (!out || !b || !err) return false;`. Это не формальность, а страховка границы API.

Массив и размер

В С массив не носит с собой длину. Если функция получает pointer на bytes, она обязана получать и `size`. Именно поэтому `Buf` содержит `data` и `size` вместе. Это маленькая структура, но она предотвращает целый класс ошибок.

Владение памятью

Вопрос 'кто делает `free`?' важнее синтаксиса. `buf_init_from_file` выделяет память и возвращает ее через `owned_mem`; освобождает ее вызывающий код через `buf_free_owned`. `pe_parse` выделяет `sections`, а `pe_free` освобождает. Такой договор надо видеть у каждого модуля.

Строка в C

C-строка заканчивается нулевым байтом. Если забыть место под `\0`, строковые функции уйдут читать чужую память. `StrBuf` всегда хранит `len`, `cap` и дописывает ноль после каждого append.

Переполнение

В парсере недоверенного файла нельзя писать `off + len <= size`, если `off + len` может переполниться. Проект часто пишет безопаснее: сначала `if (off > size)`, потом `if (len > size - off)`. Так сложение заменяется вычитанием уже проверенного диапазона.

```
// Опаснее:
if (off + len > b->size) return false;

// Надежнее:
if (off > b->size) return false;
if (len > b->size - off) return false;
```

Endian

PE хранит числа little-endian. Даже если твой процессор тоже little-endian, проект вручную собирает числа из байтов. Это делает код явным и независимым от неочевидного поведения платформы.

Флаги и битовые маски

Флаг - это один бит внутри числа. `flags & UNW_FLAG_EHANDLER` отвечает на вопрос: установлен ли этот бит. `anomalies |= UA_TRUNCATED` добавляет признак проблемы, не стирая остальные признаки.

```
if (flags & UNW_FLAG_CHAININFO) {
    out->has_chain = true;
}

out->anomalies |= UA_TRUNCATED;
out->anomalies |= UA_BAD_OPERAND;
```

Ранний выход

В C часто проще остановиться сразу после ошибки. Но ранний выход требует освобождать уже выделенные ресурсы. `main.c` поэтому выглядит длинным: он не только запускает этапы, но и аккуратно чистит память на каждом пути.

Тесты как учебник

Unit tests в этом проекте не просто проверяют код. Они показывают минимальные искусственные bytes, из которых рождается `UNWIND_INFO`. Когда непонятно, как работает формат, открой `tests/unit/unit_unwind.c` и разбери один тест как лабораторную работу.

Мини-лабораторные перед проектом

1. Напиши программу, которая принимает path и печатает размер файла.
2. Добавь функцию `read_u32_le(bytes, offset)` и проверь ее на массиве `{0x78,0x56,0x34,0x12}`.
3. Сделай структуру `StrBuf`, добавь `append`, напечатай результат.
4. Сделай структуру `Section` и функцию `rva_to_off` для одного искусственного section.

5. Собери эти упражнения через CMake или обычный gcc, не переходи к PE, пока они не работают.

3. Предметная область проекта

PE как карта для загрузчика

PE-файл - это не просто байты программы. Это договор с Windows loader: где заголовки, какие секции есть, какие диапазоны файла превращаются в какие диапазоны виртуальной памяти, где лежат таблицы импорта, экспорта, ресурсов и exception metadata. Analyzer читает только нужную часть этого договора, но обязан проверять базовую целостность всей карты.

```
File offset:  смещение от начала файла на диске.
RVA:          relative virtual address, адрес относительно ImageBase.
VA:           ImageBase + RVA, адрес в памяти процесса.

Пример:
  ImageBase         = 0x140000000
  section .xdata RVA = 0x3000
  поле указывает RVA = 0x3040
  virtual address    = 0x140003040
```

`.pdata` и `RUNTIME_FUNCTION`

На Windows x64 exception directory обычно указывает на `.pdata`. Эта область содержит массив `RUNTIME_FUNCTION`. Каждая запись занимает 12 байт: begin RVA, end RVA и RVA структуры `UNWIND_INFO`. Диапазон функции читается как half-open interval: `[BeginAddress, EndAddress)`. Это значит: begin включен, end уже не включен.

```
typedef struct RuntimeFunction {
    uint32_t begin_rva;
    uint32_t end_rva;
    uint32_t unwind_rva;
} RuntimeFunction;
```

`UNWIND_INFO`

`UNWIND_INFO` описывает, что prolog функции сделал со stack и регистрами. Это не машинный код, а compact metadata. Первый байт содержит version и flags. Затем идут prolog size, count of unwind codes, frame register/offset. После массива unwind slots может идти handler RVA или chained runtime function.

Поле	Смысл для ученика
Version	Проект поддерживает V1 и V2. V3 отклоняется как другой формат.
Flags	EHANDLER, UHANDLER, CHAININFO. Определяют хвост структуры.
Prolog size	Размер prolog в bytes, нужен для sanity checks code offsets.
CountOfCodes	Сколько 2-байтовых slots занимает массив операций.
Frame register/off	Опциональный frame pointer и смещение.
Unwind codes	Кодированные операции push/save/alloc/set frame/epilog.
Tail	Handler RVA или chained `RUNTIME_FUNCTION`, если флаги требуют.

Что такое эвристика

Эвристика - это инженерное правило для приоритизации внимания. Она не доказывает, что файл вредоносный, и не является обученной статистической моделью. В проекте каждая эвристика видна: есть finding code, family, severity, score weight и weakening rules.

Static и dynamic

- Static mode читает bytes и не выполняет target. Его можно безопаснее гонять в CI и на больших corpus.
- Dynamic mode запускает target под Windows Debug API. Это пересечение execution boundary, нужен изолированный VM/lab.
- Static отвечает на вопрос: что записано в PE-файле.
- Dynamic отвечает на вопрос: что произошло в конкретном запуске.
- Final assessment сильнее, когда static и dynamic evidence коррелируют.

4. Архитектура и ownership

Data flow

```
input path
  -> buf_init_from_file
      owns: input bytes through owned_mem
  -> pe_parse
      owns: img.sections
  -> unwind_read_runtime_functions_ex
      owns: RuntimeFunction* rf
  -> calloc UnwindSummary[] and bool us_ok[]
  -> unwind_parse for each rf[i]
  -> heuristics_analyze
      owns: hr.details StrBuf
  -> dynamic_collect_windows
      owns: dr.details StrBuf, process/debug resources during run
  -> report writers borrow all model objects
  -> cleanup in reverse-ish order
```

Карта владения памятью

Кто создает	Что создает	Кто освобождает	Функция освобождения
`buf_init_from_file`	input bytes	`main`	`buf_free_owned`
`pe_parse`	`PeImage.sections`	`main`	`pe_free`
`unwind_read_runtime_functions_ex`	`RuntimeFunction*`	`main`	`free`
`calloc` in `main`	`UnwindSummary*`, `bool*`	`main`	`free`
`heuristics_analyze`	`HeuristicReport.details`	`main`	`heuristics_free`
`dynamic_report_init`	`DynamicReport.details`	`main`	`dynamic_report_free`
`report_write_*`	temporary `StrBuf`	same function	`sb_free`

Почему проект не смешивает слои

Если `pe.c` начал бы сразу считать suspicious score, его стало бы сложно тестировать: PE parser должен быть проверяемым сам по себе. Если `report.c` начал бы перепарсивать файл, JSON мог бы расходиться с console. Если `dynamic_win.c` был бы обязательным для сборки, static mode перестал бы быть cross-platform. Разделение ответственности - не украшение, а способ удержать проект в голове.

Exit codes

Код	Смысл
0	Запрошенный анализ и outputs завершены.
1	Ошибка input, анализа, dynamic session или записи output.
2	Неверная командная строка.

5. Подробный разбор блоков и взаимодействий

Архитектуру этого проекта надо понимать не как список файлов, а как цепочку превращений. На вход приходит путь к PE-файлу. Дальше проект превращает путь в bytes, bytes в модель PE, модель PE в таблицу runtime functions, runtime functions в summaries unwind metadata, summaries в findings и score, а затем все это в отчеты для человека и инструментов. Каждый блок имеет узкую ответственность. Если блок начинает делать чужую работу, проект становится непонятным и его сложно повторить руками.

Главное правило архитектуры

У каждого блока есть вопрос, на который он отвечает. `util` отвечает: как безопасно читать bytes и строить строки? `pe` отвечает: как устроен PE и где лежат секции? `unwind` отвечает: что записано в `.pdata` и `UNWIND_INFO`? `heuristics` отвечает: насколько эти наблюдения подозрительны? `report` отвечает: как показать результат наружу? `main` отвечает: в каком порядке все это вызвать и что освободить.

Схема потока данных

```
CLI arguments
  -> main.c parses options
  -> util.c reads input file into Buf
  -> pe.c builds PeImage
  -> unwind.c reads RuntimeFunction[] from exception directory
  -> unwind.c parses each UNWIND_INFO into UnwindSummary[]
  -> heuristics.c builds HeuristicReport
  -> dynamic_win.c optionally adds runtime DynamicReport
  -> report.c serializes everything to console / JSON / CSV / HTML
  -> main.c frees every owned object and returns exit code
```

Эта схема важна потому, что данные идут в одну сторону. `report.c` не должен заново читать PE. `heuristics.c` не должен открывать файл. `pe.c` не должен решать, вредоносный файл или нет. Чем строже это разделение, тем легче тебе писать проект по частям: сначала bytes, потом PE, потом unwind, потом смысл.

Карта блоков

Блок	За что отвечает	Что принимает	Что отдает
`main.c`	Оркестрация: аргументы, порядок вызовов, обработка ошибок, cleanup, exit codes.	`argc/argv`, пути, flags пользователя.	Запущенный pipeline, файлы отчетов, код завершения 0/1/2.
`util.h/c`	Безопасные базовые операции: bytes, endian, строки, запись файлов, hash.	Путь к файлу, `Buf`, offset/length, куски текста.	`Buf`, числа `u16/u32/u64`, `StrBuf`, записанный файл, SHA-256.
`pe.h/c`	Модель PE32+ AMD64: headers, sections, exception directory, RVA mapping.	`Buf` с bytes файла.	`PeImage`: секции, размеры, directories, structural counters.
`unwind.h/c`	Чтение `.pdata` и расшифровка `UNWIND_INFO`.	`Buf`, `PeImage`, exception directory, unwind RVA.	`RuntimeFunction[]`, `UnwindSummary[]`, anomaly flags.
`heuristics.h/c`	Интерпретация наблюдений: findings, score, confidence, verdict.	`PeImage`, runtime functions, unwind summaries, optional dynamic report.	`HeuristicReport` с evidence families, score и объяснениями.
`report.h/c`	Внешний контракт: console, CSV, Ghidra CSV, JSON, HTML.	Готовые модели: metadata, PE, unwind, heuristics, dynamic.	Текстовые отчеты, которые можно читать и проверять.
`dynamic.h` / `dynamic_win.c`	Опциональное runtime evidence через Windows Debug API.	Путь к target, dynamic options, static context.	`DynamicReport`: события VEH/VCH/UEF, runtime function table, hits.
`tests/`	Доказательство контрактов каждого блока и всего CLI.	Искусственные bytes, sample PE, CLI commands.	Понимание, что поведение не сломалось после изменения.

Блок `main.c`: дирижер, а не рабочий цех

`main.c` не должен знать, как именно устроен PE header или как экранировать JSON. Его работа - принять команду пользователя и провести данные по маршруту. Сначала он разбирает аргументы: где input, какие outputs нужны, включен ли strict ABI, нужен ли dynamic mode. Потом он вызывает `buf_init_from_file`, `pe_parse`, `unwind_read_runtime_functions_ex`, `unwind_parse`, `heuristics_analyze`, writers отчетов и cleanup.

Почему это важно: если ты напишешь парсинг PE прямо в `main.c`, проект станет большой простыней. Ты не сможешь отдельно проверить PE parser, отдельно report writer, отдельно heuristics. `main.c` должен быть длинным только из-за обработки ошибок и освобождения ресурсов, а не из-за предметной логики.

- Что `main.c` может делать: читать CLI options, вызывать модули, хранить pointers на созданные объекты, решать exit code.
- Что `main.c` не должен делать: вручную читать PE headers, считать score, экранировать JSON, декодировать unwind opcodes.
- Главный учебный вопрос: если любой шаг упал, какие уже созданные ресурсы надо освободить?

Блок `util`: фундамент безопасности

`util` существует потому, что C сам не хранит размер массива рядом с pointer. Для парсера бинарного файла это опасно: один неправильный offset - и программа читает чужую память. `Buf` делает договор явным: есть pointer `data` и есть `size`, дальше читать можно только через функции, которые проверяют границы.

`StrBuf` решает другую базовую проблему C: обычная строка `char*` не знает свою емкость. Отчеты могут быть маленькими или огромными, поэтому writer-ам нужна строка, которая растет через `realloc`, всегда оставляет место под `\0` и запоминает ошибку выделения через `failed`.

Объект	Кто создает	Кто использует	Зачем нужен
`Buf`	`buf_init_from_file` или тест	`pe.c`, `unwind.c`, tests	Безопасно читать bytes по offset.
`StrBuf`	report writers, helper functions	`report.c`, иногда error builders	Собирать текст неизвестной длины.
`owned_mem`	`buf_init_from_file`	`main.c` освобождает	Хранить реальные bytes файла в heap.
SHA-256 hex	`sha256_hex`	metadata reports	Идентифицировать точный input file.

Блок `pe`: карта PE-файла

`pe.c` берет `Buf` и строит `PeImage`. Это не отчет и не подозрение. Это карта местности: какие секции есть, какие у них raw ranges, какие virtual ranges, где exception directory, какой SizeOfImage, какие structural проблемы уже видны. После этого другие блоки перестают гадать, где лежат bytes.

Самая важная функция взаимодействия - `pe_rva_to_fileoff`. Unwind metadata хранит адреса как RVA, а файл читается по file offset. Когда `unwind.c` хочет прочитать `UNWIND_INFO` по RVA, он не должен сам искать секцию. Он спрашивает `pe.c`: переведи мне этот RVA в offset, если это вообще возможно.

```
unwind.c has:  unwind_info_rva = 0x1234
unwind.c asks: pe_rva_to_fileoff(pe, 0x1234, &off)
pe.c checks:   which section owns this RVA?
pe.c checks:   is this RVA backed by bytes in the file?
pe.c returns:  off, or false if reading would be unsafe
```

Почему `pe.c` не считает score: поврежденная секция - это наблюдение, а не verdict. Решение о подозрительности принимает `heuristics.c`, потому что только он видит все семейства признаков вместе.

Блок `unwind`: предметная область проекта

`unwind.c` делает две разные работы. Первая - прочитать `.pdata` как массив `RUNTIME_FUNCTION`. Вторая - по каждому `UnwindInfoAddress` прочитать `UNWIND_INFO`. Это надо держать отдельно в голове: `.pdata` говорит, какие функции описаны, а `UNWIND_INFO` говорит, как раскрутить stack frame и где могут быть handlers.

Часть	Что читает	Что получается	Кому нужно дальше
`.pdata`	Записи по 12 bytes: begin, end, unwind RVA.	`RuntimeFunction[]`.	`heuristics.c`, `report.c`, цикл parse в `main.c`.

Часть	Что читает	Что получается	Кому нужно дальше
`UNWIND_INFO` header	Version, flags, prolog, count, frame byte.	Базовый `UnwindSummary`.	`heuristics.c`, HTML/JSON report.
Unwind codes	2-byte slots переменной длины.	Stack/register effects, anomaly flags.	`heuristics.c`, человек в отчете.
Tail	Handler RVA или chained runtime function.	Handler/chain context.	Placement checks, manual review.

Почему `unwind.c` не должен сам решать, подозрительно ли это: parser обязан быть максимально нейтральным. Он говорит 'запись обрезана', 'opcode неизвестен', 'handler RVA такой-то'. А уже `heuristics.c` решает, насколько это важно с учетом секций, количества записей, других признаков и weakening rules.

Блок `heuristics`: слой смысла

`heuristics.c` - это место, где технические наблюдения превращаются в объяснимые признаки. Он получает уже готовые модели от `pe.c` и `unwind.c`. Он не читает bytes и не пишет JSON. Его работа - собрать counters, создать findings, посчитать score, confidence, evidence level и verdict.

- Из `PeImage` он берет structural counters: section out-of-bounds, overlaps, exception directory problems.
- Из `RuntimeFunction[]` он берет invalid ranges, overlaps, duplicate begins, unsorted entries, unwind RVA problems.
- Из `UnwindSummary[]` он берет parse failures, anomaly flags, handlers, placement context.
- Из `DynamicReport` он может взять runtime signals, если dynamic mode был включен.
- На выходе он дает не приговор, а triage: насколько случай требует внимания и почему.

Здесь важно различать `score` и `confidence`. Score отвечает: сколько подозрительных признаков набралось по весам. Confidence отвечает: насколько надежна такая интерпретация. Один слабый признак может дать небольшой score и низкую confidence. Несколько независимых семейств evidence дают более сильную картину.

Блок `report`: перевод внутренней модели наружу

`report.c` не анализирует файл. Он берет готовые структуры и превращает их в форматы. Это похоже на переводчика: смысл уже создан, теперь его надо безопасно записать в console, CSV, JSON или HTML. Поэтому `report.c` обязан заниматься escaping, schema version, порядком полей, atomic write, но не обязан знать, почему конкретный finding получил такой score.

Формат	Для чего нужен	Что важно не сломать
Console	Быстро понять результат после запуска.	Короткий verdict, score, главные findings.
JSON	Автотесты, интеграция, защита schema.	Валидный JSON, стабильные имена полей, escaping.
CSV	Таблицы, фильтрация, простая обработка.	Кавычки, запятые, переносы строк.
Ghidra CSV	Перенос результатов в reverse engineering workflow.	RVA, comments/bookmarks, ожидаемые колонки importer-а.
HTML	Человекочитаемый отчет.	HTML escaping, CSP, понятная структура evidence.

Блок `dynamic`: отдельная граница опасности

Static pipeline просто читает bytes. Dynamic mode запускает target. Это принципиально другая граница риска. Поэтому dynamic mode отделен в `dynamic_win.c`, включается явно и является Windows-specific. Его результат не заменяет static analysis, а добавляет runtime evidence: были ли регистрации VEH/VCH/UEF, сработали ли handlers, вызывались ли runtime function table APIs.

Взаимодействие такое: `main.c` после static analysis смотрит, просил ли пользователь dynamic mode. Если да - запускает collector, получает `DynamicReport`, передает его в `heuristics` или report layer. Если dynamic mode не удался, static result не исчезает. В отчете должна быть причина, почему runtime evidence неполное.

Как блоки взаимодействуют на одном примере

1. Пользователь передает `sample.exe --json out.json`.
2. `main.c` разбирает аргументы и вызывает `buf_init_from_file`.
3. `util.c` возвращает `Buf`: bytes файла и размер.
4. `pe.c` проверяет `MZ`, `PE`, AMD64, PE32+, читает секции и exception directory.
5. `unwind.c` берет exception directory из `PeImage` и читает `RuntimeFunction[]`.
6. `unwind.c` для каждой записи переводит unwind RVA через `pe_rva_to_fileoff` и читает `UNWIND_INFO`.
7. `heuristics.c` смотрит на PE problems, runtime function problems, unwind anomalies и создает findings.
8. `report.c` сериализует metadata, PE summary, unwind summaries и heuristic report в JSON.
9. `main.c` вызывает cleanup: `heuristics_free`, `free`, `pe_free`, `buf_free_owned`.

Запрещенные смешивания ответственности

Если хочется сделать так	Почему это плохо	Как правильно
В `pe.c` сразу увеличивать suspicious score.	PE parser станет невозможно тестировать как нейтральный parser.	В `pe.c` только counters, score - в `heuristics.c`.
В `report.c` заново искать секции и RVA.	Разные отчеты могут начать расходиться в логике.	Report получает готовый `PeImage` и только выводит его.
В `unwind.c` открывать файл по path.	Появится второй способ чтения input и риск разных ошибок.	Все bytes идут через `Buf` из `util.c`.
В `main.c` руками формировать JSON.	Оркестратор станет смесью CLI, анализа и сериализации.	Вызвать `report_write_json`.
Всегда запускать dynamic mode.	Это опасно для недоверенных файлов и ломает cross-platform static mode.	Dynamic mode только по явному флагу и в изоляции.

Что надо уметь объяснить на защите

- Почему `Buf` стоит раньше PE parser и почему голый pointer недостаточен.
- Почему `PeImage` является картой адресов, а не результатом подозрительности.
- Почему `unwind.c` зависит от `pe_rva_to_fileoff`, а не переводит RVA сам.
- Почему `heuristics.c` отделен от parser-ов и от report writer-ов.

- Почему score - это triage, а не вероятность вредоносности.
- Почему dynamic mode optional и требует изоляции.
- Почему cleanup находится в `main.c`, хотя память создают разные модули.

6. Сборка как часть обучения

Сборка в C - это не кнопка 'магия'. Это отдельная модель проекта: какие `.c` входят в library, какой executable их использует, какие предупреждения включены, какие тесты запускаются. Начинать повторение проекта лучше с минимальной сборки, а не с полного `unwind.c`.

Минимальный порядок

1. Создай `include/` и `src/`.
2. Создай пустые `util.h`, `util.c`, `main.c`.
3. Сделай `CMakeLists.txt` с library `ehua_core` и executable `eh_unwind_analyzer`.
4. Включи C17.
5. Добавь warnings: `-Wall -Wextra -Wpedantic -Wconversion -Wshadow -Wformat=2`.
6. Собери пустую программу с `--help`.
7. Только потом добавляй PE parser.

```
cmake --preset debug
cmake --build --preset debug
ctest --preset debug

cmake --preset release
cmake --build --preset release
ctest --preset release
```

Как относиться к warnings

В этом проекте warnings-as-errors включены не для жесткости ради жесткости. В C warning часто означает 'здесь компилятор увидел место, где человек мог ошибиться с типом, размером или format string'. Когда ты учишься, warning - это бесплатный преподаватель.

7. Файл за файлом: что, зачем и как должно работать

Этот раздел добавлен специально как обучающая версия маршрута. Здесь каждый файл рассматривается не как пункт 'создай и напиши', а как деталь системы. Для каждого блока сначала объясняется, зачем файл вообще нужен, какую проблему он закрывает, какие сущности должны появиться в заголовке `.h`, какая логика должна жить в `.c`, как данные проходят через модуль и почему следующий модуль сможет на него опереться.

Как пользоваться этой главой

Не переписывай сразу полный файл из готового проекта. Сначала сделай минимальный скелет, собери проект, потом добавь одну функцию, опять собери, потом напиши маленькую проверку. Если ты можешь словами объяснить, какой input у функции, какой output, кто владеет памятью и что будет при ошибке, значит кусок действительно начал становиться твоим.

CMakeLists.txt и CMakePresets.json

Зачем этот файл существует

Эти файлы нужны не для анализа PE, а для управления всей стройкой проекта. В C один `.c` файл сам по себе не превращается в программу: его надо скомпилировать, связать с другими объектными файлами, дать include-пути, включить предупреждения и отдельно собрать тесты.

Какую проблему он решает

Главная проблема новичка в C - непонятно, почему код написан, но программа не собирается. CMake фиксирует контракт сборки: какие исходники входят в ядро, какой исполняемый файл является CLI, какие тесты должны линковаться с ядром, какие предупреждения считаются ошибками.

Что должно быть в заголовке или конфигурации

- `CMakeLists.txt` должен объявить проект, стандарт C17 и опции сборки.
- В нем должна быть библиотека ядра, например `ehua_core`, куда входят `util.c`, `pe.c`, `unwind.c`, `heuristics.c`, `report.c`, `app_info.c`.
- Отдельно должен быть executable `eh_unwind_analyzer`, который добавляет `src/main.c` и линкуется с `ehua_core`.
- `include/` должен быть публичным include-каталогом, потому что `.c` файлы и тесты подключают заголовки оттуда.
- `CMakePresets.json` нужен для повторяемых команд `debug`, `release`, `mingw` без ручного ввода длинных параметров.

Что должно быть в реализации

- Здесь нет `.c` реализации, но есть логика зависимостей: кто от кого зависит и что должно пересобираться при изменении файла.
- Добавь warnings: они будут твоим ранним преподавателем по типам, форматным строкам, неиспользуемым переменным и опасным преобразованиям.

- Добавь CTest, чтобы после каждого маленького шага можно было запускать не весь проект руками, а автоматические проверки.

Как это будет работать в проекте

1. Ты пишешь `src/util.c`, компилятор превращает его в объектный файл.
2. CMake собирает несколько объектных файлов в библиотеку `ehua_core`.
3. `main.c` вызывает функции из ядра, а linker соединяет вызовы с реализациями.
4. Unit tests тоже линкуются с `ehua_core`, поэтому можно проверять `pe.c` или `unwind.c` без запуска полного CLI.

Минимальный скелет, с которого можно начать

```
cmake_minimum_required(VERSION 3.20)
project(eh_unwind_analyzer C)

set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)

add_library(ehua_core
    src/util.c
    src/pe.c
    src/unwind.c
    src/heuristics.c
    src/report.c
    src/app_info.c
)
target_include_directories(ehua_core PUBLIC include)

add_executable(eh_unwind_analyzer src/main.c)
target_link_libraries(eh_unwind_analyzer PRIVATE ehua_core)

enable_testing()
```

Как понять, что блок готов

- `cmake --preset debug` создает build-каталог без ручной настройки.
- `cmake --build --preset debug` собирает хотя бы пустой CLI.
- `ctest --preset debug` запускается, даже если сначала тестов мало.
- Если добавил новый `.c`, он явно появился в списке источников, а не потерялся.

Что ты здесь учишь в C

- Что такое translation unit: каждый `.c` компилируется отдельно.
- Что такое header include path.
- Что такое linker и почему прототип функции в `.h` не равен реализации.
- Почему warnings важны для C, где язык часто разрешает опасные вещи.

include/util.h и src/util.c

Зачем этот файл существует

`util` - фундамент. Он не знает про PE, unwind и подозрительность. Его задача - дать безопасные кирпичи: прочитать файл в память, читать байты по смещению без выхода за границы, собирать растущую строку отчета, писать файл атомарно и считать hash входного файла.

Какую проблему он решает

Все остальные модули работают с недоверенным бинарным файлом. Если каждый модуль сам будет писать `fread`, `off + len`, `realloc` и форматирование строк, ошибки размножатся. `util` собирает опасные низкоуровневые операции в одном месте и делает их проверяемыми.

Что должно быть в заголовке или конфигурации

- `typedef struct Buf { const uint8_t* data; size_t size; } Buf;` - представление массива байтов с длиной.
- Прототипы `buf_init_from_file`, `buf_free_owned`, `buf_read`, `buf_read_u16`, `buf_read_u32`, `buf_read_u64`.
- `typedef struct StrBuf { char* s; size_t len; size_t cap; bool failed; } StrBuf;` - растущая строка для отчетов.
- Прототипы `sb_init`, `sb_appendn`, `sb_append`, `sb_appendf`, `sb_free`, `sb_ok`.
- Прототипы `file_write_atomic` и `sha256_hex`, потому что отчетам нужны надежная запись и идентичность входного файла.

Что должно быть в реализации

- `buf_read` должен сначала проверять аргументы, потом `off > size`, потом `len > size - off`, и только после этого делать `memcpy`.
- `buf_read_u32` должен читать 4 байта и вручную собирать little-endian число, а не делать опасное приведение указателя.
- `buf_init_from_file` открывает файл, узнает размер, выделяет память, читает bytes и возвращает владение вызывающему коду.
- `StrBuf` должен всегда оставлять строку с завершающим `\0` и не терять старую строку при неудачном `realloc`.
- `file_write_atomic` пишет во временный файл, а затем заменяет итоговый, чтобы не оставить битый отчет при сбое.

Как это будет работать в проекте

1. `main.c` просит `buf_init_from_file` загрузить target.exe.
2. `pe.c` получает `Buf` и читает из него заголовки через `buf_read_u32`, не касаясь `FILE*`.
3. `report.c` создает `StrBuf`, добавляет JSON/HTML/CSV текст, проверяет `sb_ok` и пишет результат через `file_write_atomic`.
4. В конце `main.c` вызывает `buf_free_owned`, потому что именно он владеет выделенной памятью входного файла.

Минимальный скелет, с которого можно начать

```
typedef struct Buf {
    const uint8_t* data;
    size_t size;
} Buf;

bool buf_read(const Buf* b, size_t off, void* out, size_t len) {
    if (!b || !out) return false;
    if (off > b->size) return false;
    if (len > b->size - off) return false;
    memcpy(out, b->data + off, len);
    return true;
}

bool buf_read_u32(const Buf* b, size_t off, uint32_t* out) {
    uint8_t x[4];
    if (!out || !buf_read(b, off, x, sizeof x)) return false;
    *out = ((uint32_t)x[0]) |
           ((uint32_t)x[1] << 8) |
           ((uint32_t)x[2] << 16) |
           ((uint32_t)x[3] << 24);
    return true;
}
```

Как понять, что блок готов

- Попытка прочитать 4 байта из 2-байтового буфера возвращает `false`, а не падает.
- Число bytes `01 00 00 00` читается как `1`.
- После нескольких `sb_append` строка имеет правильный `len` и завершающий ноль.
- Каждая функция ясно отвечает на вопрос: кто выделил память и кто ее освобождает.

Что ты здесь учишь в C

- Указатели и `NULL`.
- `size_t` против `uint32_t`.
- Проверка границ без integer overflow.
- `malloc`, `realloc`, `free` и владение памятью.
- Почему C-строка требует завершающий нулевой байт.

include/pe.h и src/pe.c

Зачем этот файл существует

`pe` превращает сырой файл из набора bytes в понятную модель PE32+ AMD64: заголовки, секции, exception directory, размеры образа и карту RVA -> file offset. Без этого `unwind.c` не знает, где в файле лежит `.pdata`, а `heuristics.c` не знает, какие диапазоны подозрительны.

Какую проблему он решает

PE использует разные координаты: file offset - место в файле, RVA - адрес относительно ImageBase в памяти, section raw range - bytes на диске, virtual range - область в памяти. Ошибка в этой карте ломает весь анализ. Поэтому `pe.c` не просто читает поля, а проверяет границы, пересечения и выравнивание.

Что должно быть в заголовке или конфигурации

- Константы: `MZ`, `PE\0\0`, `IMAGE_FILE_MACHINE_AMD64`, `PE32+`, индекс exception directory.
- `PeSection`: имя секции, RVA, virtual size, raw pointer, raw size, characteristics.
- `PeImage`: image base, size of image, alignments, массив секций, exception directory RVA/size, счетчики проблем.
- Прототипы `pe_parse`, `pe_free`, `pe_find_section_for_rva`, `pe_rva_to_fileoff`.

Что должно быть в реализации

- `pe_parse` должен занулять `out`, читать DOS header, брать `e_lfanew`, проверять PE signature.
- Дальше читается COFF header: machine, число секций, размер optional header.
- Optional header PE32+ дает ImageBase, SectionAlignment, FileAlignment, SizeOfImage, DataDirectory.
- Цикл по section headers создает массив `PeSection`.
- После чтения надо посчитать structural counters: raw out-of-bounds, virtual out-of-bounds, overlaps, misalignment.
- `pe_rva_to_fileoff` должен возвращать `false`, если RVA попал в virtual tail, которого нет на диске.

Как это будет работать в проекте

1. `main.c` передает `Buf` в `pe_parse`.
2. `pe_parse` заполняет `PeImage`, но не копирует весь файл: он только строит карту и метаданные.
3. `unwind.c` спрашивает у `PeImage`, где exception directory и как перевести RVA unwind info в file offset.
4. `heuristics.c` берет из `PeImage` счетчики поврежденных секций и использует их как признаки.
5. В конце `main.c` вызывает `pe_free`, потому что `PeImage` владеет массивом секций.

Минимальный скелет, с которого можно начать

```
typedef struct PeSection {
    char name[9];
    uint32_t virtual_address;
    uint32_t virtual_size;
    uint32_t raw_ptr;
    uint32_t raw_size;
    uint32_t characteristics;
} PeSection;

bool pe_rva_to_fileoff(const PeImage* pe, uint32_t rva, size_t* out) {
    const PeSection* s = pe_find_section_for_rva(pe, rva);
    if (!s || !out) return false;
    uint32_t delta = rva - s->virtual_address;
    if (delta >= s->raw_size) return false;
    *out = (size_t)s->raw_ptr + delta;
    return true;
}
```

Как понять, что блок готов

- Не-PE файл отвергается понятной ошибкой.

- PE32/x86 отвергается, потому что проект анализирует PE32+ AMD64 unwind.

- RVA внутри секции переводится в file offset.

- RVA в виртуальном хвосте секции не считается доступным байтом файла.

- Секция, чей raw range выходит за размер файла, увеличивает счетчик проблемы.

Что ты здесь учишь в C

- `struct` как модель предметной области.

- Чтение бинарного формата по фиксированным offset.

- Разница между `uint32_t` RVA и `size_t` file offset.

- Циклы по массиву структур.

- Ошибки формата как нормальный результат функции, а не авария.

include/unwind.h и src/unwind.c

Зачем этот файл существует

`unwind` читает именно ту часть Windows x64, ради которой создан проект: `.pdata` как массив `RUNTIME_FUNCTION` и `UNWIND_INFO` как описание пролога, stack frame, saved registers, handlers и chains.

Какую проблему он решает

`.pdata` не хранит красивые структуры C. Это компактные bytes с битовыми полями, переменной длиной записей и хвостами, которые появляются только при определенных flags. Если читать это без строгого алгоритма, легко перепутать opcode, выйти за границы или принять мусор за handler.

Что должно быть в заголовке или конфигурации

- `RuntimeFunction`: `begin_rva`, `end_rva`, `unwind_info_rva`.
- `UnwindSummary`: version, flags, prolog size, count of codes, frame register, stack allocation, saved registers, handler RVA, anomalies, parse error.
- `UnwindParseOpts`: режимы strict ABI, follow chain, ограничения глубины.
- Константы flags: EHANDLER, UHANDLER, CHAININFO.
- Константы anomalies: truncated, unsupported version, bad opcode, bad operand, alignment, handler out-of-bounds.

Что должно быть в реализации

- `unwind_read_runtime_functions_ex` берет exception directory из `PeImage`, переводит RVA в file offset и читает записи по 12 байт.
- `unwind_parse` читает первый byte как version+flags, затем prolog, count, frame byte.
- `slots_for_code` должен знать, сколько 2-байтовых slots занимает каждый UWOP.
- Главный цикл идет `for (i = 0; i < count;)`, потому что шаг зависит от opcode.
- После slots надо учитывать padding до четного количества slots, затем optional handler tail или chain tail.
- Follow-chain включается осторожно и с ограничением глубины, чтобы поврежденный файл не загнал анализатор в бесконечность.

Как это будет работать в проекте

1. `main.c` получает массив `RuntimeFunction`.
2. Для каждой записи `main.c` вызывает `unwind_parse` и кладет результат в массив `UnwindSummary`.
3. `heuristics.c` потом сравнивает begin/end/unwind RVA и смотрит anomalies из каждого summary.
4. `report.c` показывает человеку не сырые bytes, а расшифровку: flags, handler, ошибки, stack effect.

Минимальный скелет, с которого можно начать

```
bool unwind_parse(const Buf* b, const PeImage* pe,
                  uint32_t unwind_rva, UnwindSummary* out) {
    size_t off = 0;
    uint8_t header[4];
    if (!pe_rva_to_fileoff(pe, unwind_rva, &off)) return false;
    if (!buf_read(b, off, header, sizeof header)) return false;

    out->version = header[0] & 0x7u;
    out->flags = header[0] >> 3;
    out->prolog_size = header[1];
    out->code_count = header[2];
    out->frame_register = header[3] & 0x0fu;
    out->frame_offset = header[3] >> 4;

    for (uint32_t i = 0; i < out->code_count;) {
        /* read slot, decode op, validate operand slots, then i += slots */
    }
    return true;
}
```

Как понять, что блок готов

- Запись `.pdata` размером не кратным 12 отмечается как проблема.
- `UNWIND_INFO` с unsupported version не парсится как обычная версия.
- Opcode, которому нужны дополнительные slots, проверяет наличие этих slots.
- EHANDLER/UHANDLER читают handler RVA из хвоста только после правильного выравнивания.
- Ошибку парсинга видно в `UnwindSummary`, а не только в общем сообщении CLI.

Что ты здесь учишь в C

- Битовые поля: mask, shift, flags.
- `switch` по opcode.
- Цикл с переменным шагом.
- Байтовый формат переменной длины.
- Защитное программирование при недоверенном input.

include/heuristics.h и src/heuristics.c

Зачем этот файл существует

`heuristics` превращает технические наблюдения в человеческий вывод: findings, severity, score, suspicion, confidence, evidence level и verdict. Это слой интерпретации, а не слой чтения файла.

Какую проблему он решает

PE parser и unwind parser говорят 'видел overlap', 'handler лежит в writable section', 'unwind info не читается'. Для защиты и отчета этого мало: нужно сгруппировать признаки, объяснить серьезность, не завысить вывод по одному слабому сигналу и показать, где требуется ручная проверка.

Что должно быть в заголовке или конфигурации

- `HeuristicFinding`: code, family, severity, message, representative RVA, affected count.
- `HeuristicMitigation`: причина, почему confidence снижен.
- `HeuristicReport`: score 0..100, suspicion, confidence, evidence level, verdict, arrays findings/mitigations.
- Константы максимального числа findings и mitigation entries.
- Прототипы `heuristics_analyze`, `heuristics_free`, функции получения текстовых имен severity/evidence.

Что должно быть в реализации

- Сначала собрать counters из `PeImage`: section oob, overlaps, exception directory oob.
- Потом пройти массив `RuntimeFunction`: invalid ranges, begin/end outside image, unwind RVA outside file/image, unsorted, overlaps, duplicates.
- Потом посмотреть `UnwindSummary`: parse errors, anomalies, handler placement, writable/executable metadata.
- Reuse одинаковых unwind RVA удобно считать маленькой hash map с open addressing.
- После counters добавить findings с понятными codes и messages.
- Score считать по весам, confidence - отдельно по силе и разнообразию evidence, затем применить weakening rules.

Как это будет работать в проекте

1. `heuristics_analyze` получает уже готовые `PeImage`, `RuntimeFunction[]`, `UnwindSummary[]`.
2. Он не читает файл сам: его вход - результаты предыдущих слоев.
3. Он создает `HeuristicReport`, который потом используют console/JSON/HTML writers.
4. Если finding один и слабый, confidence должен снижаться: проект не должен притворяться статистической моделью.

Минимальный скелет, с которого можно начать

```
typedef struct HeuristicFinding {
    const char* code;
    const char* family;
    const char* severity;
    uint32_t representative_rva;
    size_t affected_count;
    char message[256];
} HeuristicFinding;

static void add_finding(HeuristicReport* r,
                        const char* code,
                        const char* family,
                        const char* severity,
                        uint32_t rva,
                        size_t count,
                        const char* message) {
    if (r->finding_count >= HEURISTICS_MAX_FINDINGS) return;
    /* fill next slot, then increment finding_count */
}
```

Как понять, что блок готов

- Benign sample с нормальной таблицей получает score 0 или низкий score без high verdict.
- Два сильных независимых признака дают strong evidence.
- Один placement-only признак не превращается автоматически в уверенное обвинение.
- Все findings имеют code, severity и объяснимый affected count.
- `heuristics_free` освобождает все, что было выделено внутри report.

Что ты здесь учишь в C

- Массивы структур фиксированного размера.
- `qsort` и компараторы.
- Простая hash map.
- Разделение score и confidence.
- Почему инженерная эвристика не равна вероятности.

include/report.h и src/report.c

Зачем этот файл существует

`report` отвечает за внешний контракт проекта. Пользователь и защита видят не внутренние структуры C, а console summary, CSV, Ghidra CSV, JSON и HTML. Этот слой должен быть скучным и надежным: правильно экранировать текст, не ломать формат и писать файлы атомарно.

Какую проблему он решает

Даже хороший анализ бесполезен, если JSON невалиден, CSV ломается на запятой, HTML допускает неэкранированный текст, а файл отчета остается наполовину записанным. `report.c` отделяет вычисление evidence от представления evidence.

Что должно быть в заголовке или конфигурации

- `ReportMetadata`: версия приложения, версия schema, путь входного файла, размер, sha256, время.
- `AssessmentSummary`: verdict, score, suspicion, confidence, requires manual review.
- Прототипы `report_write_console`, `report_write_json`, `report_write_csv`, `report_write_ghidra_csv`, `report_write_html`.
- Функции escaping: JSON string, CSV cell, HTML text.

Что должно быть в реализации

- Каждый writer должен сначала проверить аргументы.
- Текст собирается в `StrBuf`, а не печатается кусками прямо в файл.
- JSON writer обязан следить за запятыми между элементами массива.
- CSV writer обязан заключать поле в кавычки, если внутри есть запятая, кавычка или newline.
- HTML writer обязан экранировать пользовательские данные и добавлять CSP.
- Запись на диск идет через `file_write_atomic`.

Как это будет работать в проекте

1. `main.c` собирает metadata, `PeImage`, unwind summaries и heuristic report.
2. В зависимости от CLI options он вызывает один или несколько report writers.
3. Writer не пересчитывает score и не перечитывает PE: он только сериализует готовую модель.
4. Python tests читают JSON/CSV/HTML и проверяют, что внешний контракт не изменился случайно.

Минимальный скелет, с которого можно начать

```
bool report_write_json(const char* path,
                       const ReportMetadata* meta,
                       const PeImage* pe,
                       const HeuristicReport* hr) {
    StrBuf sb;
    sb_init(&sb);
    sb_append(&sb, "{\n");
    sb_appendf(&sb, "  \"schema_version\": \"%s\",\n", meta->schema_version);
    sb_appendf(&sb, "  \"score\": %u,\n", hr->score);
    sb_append(&sb, "  \"findings\": [\n");
    /* append escaped findings */
    sb_append(&sb, "  ]\n}\n");
    if (!sb_ok(&sb)) { sb_free(&sb); return false; }
    bool ok = file_write_atomic(path, sb.s, sb.len);
    sb_free(&sb);
    return ok;
}
```

Как понять, что блок готов

- JSON открывается парсером без ошибок.
- Строка с кавычкой в message корректно экранируется.
- CSV не ломает колонку, если в тексте есть запятая.
- HTML показывает кириллицу и специальные символы без поломанной верстки.
- При ошибке записи функция возвращает `false`, а `main.c` может вернуть exit code 1.

Что ты здесь учишь в C

- Сериализация данных.
- Форматные строки и `printf`-семейство.
- Экранирование специальных символов.
- Работа с динамической строкой.
- Контракт между программой и внешними тестами.

include/dynamic.h и src/dynamic_win.c

Зачем этот файл существует

`dynamic` - необязательный Windows-слой, который не просто смотрит на файл, а запускает target под debugger и собирает runtime evidence: VEH/VCH/UEF, RtlAddFunctionTable, срабатывания handler и другие события исключений.

Какую проблему он решает

Статический анализ показывает, что в файле есть подозрительная структура. Динамический режим может дать дополнительное основание: код действительно регистрирует runtime function table или обработчик исключений срабатывает во время выполнения. Но запускать чужой код опасно, поэтому этот режим должен быть изолированным и отделенным от обычного static pipeline.

Что должно быть в заголовке или конфигурации

- `DynamicOptions`: путь к target, аргументы, timeout, flags включения наблюдений.
- `DynamicEvent`: тип события, адрес/RVA, текстовое описание.
- `DynamicReport`: массив событий, counters, флаг completed, error message.
- Прототипы `dynamic_report_init`, `dynamic_report_free`, `dynamic_collect`.
- В заголовке должен быть portable контракт, а Windows API детали остаются в `dynamic_win.c`.

Что должно быть в реализации

- На не-Windows сборке должен быть stub, который честно сообщает, что dynamic mode недоступен.
- На Windows `CreateProcess` запускает процесс как debuggee.
- Debug loop читает события, ставит breakpoints, восстанавливает оригинальный byte, откатывает RIP и ставит trap flag.
- INT3 event добавляет `DynamicEvent`; single-step переустанавливает breakpoint.
- Timeout и cleanup обязательны: debuggee нельзя оставлять зависшим.

Как это будет работать в проекте

1. `main.c` запускает dynamic mode только если пользователь явно попросил соответствующую опцию.
2. Dynamic report добавляется к итоговому report рядом со static evidence.
3. Финальная confidence может увеличиться, если dynamic evidence коррелирует со static findings.
4. Если dynamic mode не завершился, static report все равно остается полезным, но в отчете будет причина неполноты.

Минимальный скелет, с которого можно начать

```
bool dynamic_collect(const DynamicOptions* opt, DynamicReport* out) {
#ifndef _WIN32
    dynamic_report_add_error(out, "dynamic mode is available only on Windows");
    return false;
#else
    /* create debuggee, set breakpoints, process debug events, cleanup */
    return true;
#endif
}
```

Как понять, что блок готов

- Обычный static анализ собирается и работает без Windows debugger.
- На неподдерживаемой платформе dynamic mode дает понятную ошибку, а не undefined behavior.
- При timeout процесс завершается или отсоединяется предсказуемо.
- События dynamic report сериализуются в JSON/HTML и не смешиваются с static counters.

Что ты здесь учишь в C

- Условная компиляция `#ifdef _WIN32`.
- Разделение portable API и platform-specific реализации.
- Работа с ресурсами ОС.
- Состояние debug loop.
- Почему небезопасный runtime режим должен быть отдельным от парсинга файла.

include/app_info.h и src/app_info.c

Зачем этот файл существует

`app_info` кажется маленьким, но он нужен для воспроизводимости. Отчет должен говорить, какой версией анализатора и какой версией схемы он создан. Иначе через месяц невозможно понять, почему старый JSON отличается от нового.

Какую проблему он решает

Версия приложения, schema version и build metadata часто размазываются по коду строковыми литералами. Тогда один writer пишет одно, CLI `--version` другое, а тесты третье. `app_info` делает единый источник правды.

Что должно быть в заголовке или конфигурации

- Константы или функции: `app_name`, `app_version`, `report_schema_version`, `build_compiler`.
- Прототип функции, которая заполняет `ReportMetadata` базовыми версиями.
- Никакой логики PE или эвристик в этом заголовке быть не должно.

Что должно быть в реализации

- Вернуть строковые константы через функции или экспортировать `const char*`.
- При наличии макросов сборки аккуратно подставить compiler/build info.
- `main.c` и `report.c` должны брать версии отсюда, а не хранить свои копии.

Как это будет работать в проекте

1. Пользователь вызывает `--version`, и `main.c` печатает данные из `app_info`.
2. JSON writer берет `schema_version` из той же точки.
3. Release tests проверяют, что версия и schema согласованы.

Минимальный скелет, с которого можно начать

```
const char* app_name(void) {
    return "EH/Unwind Analyzer";
}

const char* app_version(void) {
    return "1.0.0";
}

const char* report_schema_version(void) {
    return "1.1.0";
}
```

Как понять, что блок готов

- `--version` и JSON metadata показывают одну и ту же версию.
- При смене schema обновляется одно место и тесты видят новое значение.
- В модуле нет зависимостей на PE/unwind/heuristics.

Что ты здесь учишь в C

- Строковые литералы и время их жизни.

- Глобальные константы против функций доступа.
- Минимальный модуль без владения памятью.
- Зачем проекту единый источник метаданных.

src/main.c

Зачем этот файл существует

`main.c` - дирижер, а не место для всей логики. Он читает аргументы командной строки, запускает этапы в правильном порядке, обрабатывает ошибки, вызывает writers и освобождает все ресурсы.

Какую проблему он решает

Новичку часто хочется писать весь проект в `main`, потому что так сначала проще. Но тогда невозможно отдельно тестировать PE parser, отдельно unwind parser и отдельно reports. `main.c` должен соединять готовые модули, а не становиться вторым проектом внутри проекта.

Что должно быть в заголовке или конфигурации

- У `main.c` обычно нет своего `.h`, потому что он не предоставляет API другим модулям.
- В начале должны быть include нужных публичных заголовков: `util.h`, `pe.h`, `unwind.h`, `heuristics.h`, `report.h`, `dynamic.h`, `app_info.h`.
- Полезно иметь локальную структуру `CliOptions`, но она остается `static` внутри `main.c`.

Что должно быть в реализации

- Функция `parse_args` разбирает input path, output paths, flags `--json`, `--html`, `--strict-abi`, `--dynamic`, `--help`, `--version`.
- Главный pipeline: read file -> parse PE -> read runtime functions -> parse unwind for each -> heuristics -> optional dynamic -> reports.
- Каждый failure path должен выставлять сообщение, переходить к cleanup и возвращать правильный exit code.
- Cleanup должен освобождать `Buf`, `PeImage`, arrays runtime/unwind, heuristic report, dynamic report независимо от того, где случилась ошибка.

Как это будет работать в проекте

1. Пользователь запускает CLI с путем к PE.
2. `main.c` проверяет аргументы и загружает файл.
3. Дальше он по одному вызывает независимые модули и передает результаты вперед.
4. Если выбран JSON/HTML/CSV, он вызывает соответствующие writers.
5. Перед выходом он освобождает все ресурсы и возвращает 0, 1 или 2.

Минимальный скелет, с которого можно начать

```
int main(int argc, char** argv) {
    CliOptions opt;
    Buf input = {0};
    uint8_t* owned = NULL;
    PeImage pe = {0};
    RuntimeFunction* rf = NULL;
    UnwindSummary* summaries = NULL;
    HeuristicReport hr = {0};
    int exit_code = 1;

    if (!parse_args(argc, argv, &opt)) return 2;
    if (!buf_init_from_file(opt.input_path, &input, &owned)) goto cleanup;
    if (!pe_parse(&input, &pe, opt.err, sizeof opt.err)) goto cleanup;
    /* read runtime functions, parse unwind, analyze, write reports */
    exit_code = 0;

cleanup:
    heuristics_free(&hr);
    free(summaries);
    free(rf);
    pe_free(&pe);
    buf_free_owned(owned);
    return exit_code;
}
```

Как понять, что блок готов

- `--help` работает без input file.

- Неверные аргументы дают exit code 2.

- Ошибки анализа или записи дают exit code 1.

- Нормальный анализ и успешная запись reports дают exit code 0.

- Sanitizer или внимательная проверка показывает, что на failure path память не течет.

Что ты здесь учишь в C

- `argc/argv`.

- Локальные структуры состояния.

- `goto cleanup` как честный способ не дублировать освобождение ресурсов в C.

- Разница между ошибкой пользователя и ошибкой анализа.

- Почему orchestration отличается от бизнес-логики.

tests/unit и Python CLI tests

Зачем этот файл существует

Тесты в этом проекте - не только защита от регрессий, но и учебные лабораторные. Они позволяют проверить маленький кусок без ожидания, пока весь анализатор станет полным.

Какую проблему он решает

Когда ты учишь C, самая тяжелая ошибка - написать много кода и не понимать, где он сломался. Unit tests уменьшают зону поиска: `buf_read` проверяется на 8 bytes, `unwind_parse` - на искусственном массиве bytes, report JSON - на маленькой модели данных.

Что должно быть в заголовке или конфигурации

- `tests/unit/unit_core_edges.c`: границы `Buf`, PE edge cases, маленькие искусственные inputs.
- `tests/unit/unit_unwind.c`: байты `UNWIND_INFO`, opcodes, truncated records.
- `tests/unit/unit_heuristics.c`: counters -> findings -> score/confidence.
- `tests/unit/unit_report_json.c` и HTML/CSV tests: внешний контракт writers.
- Python tests проверяют CLI как черный ящик: exit codes, schema, файлы отчетов.

Что должно быть в реализации

- Для C unit tests удобно писать маленькие helper asserts и явно создавать byte arrays.
- Каждый test должен проверять одну идею, а не весь проект сразу.
- Python tests запускают собранный executable и читают output files стандартными парсерами.
- Regression samples нужны позже, когда базовые функции уже работают.

Как это будет работать в проекте

1. Ты написал `buf_read` - сразу создал тест на нормальное чтение и выход за границы.
2. Ты написал один unwind opcode - сразу создал bytes, где этот opcode встречается.
3. Ты добавил finding - сразу проверил, что JSON содержит его code и severity.
4. Перед переходом к следующему модулю запускаешь CTest.

Минимальный скелет, с которого можно начать

```
static void test_buf_read_bounds(void) {
    const uint8_t bytes[] = {1, 2, 3};
    Buf b = { bytes, sizeof bytes };
    uint32_t value = 0;

    assert(!buf_read_u32(&b, 0, &value));
    assert(buf_read(&b, 1, &value, 2));
}

int main(void) {
    test_buf_read_bounds();
    return 0;
}
```

Как понять, что блок готов

- Один test падает - ты понимаешь, какой контракт нарушен.
- После каждого учебного шага есть команда сборки и теста.

- Тесты не требуют настоящего malware, потому что многие форматы можно собрать из bytes руками.
- CLI tests проверяют то, что увидит пользователь, а unit tests - внутренние детали.

Что ты здесь учишь в C

- `assert` и минимальные test runners.
- Искусственные byte arrays.
- Разница между white-box и black-box тестом.
- Почему тест - это письменное объяснение ожидаемого поведения.

8. Как писать проект версиями, чтобы он всегда оставался живым

Большой проект нельзя учить как один гигантский файл. Его лучше собирать версиями. Каждая версия должна компилироваться, запускаться и чему-то учить. Так ты не тонешь в синтаксисе C и одновременно видишь, как из маленьких функций рождается полный анализатор.

Версия	Цель	Что писать	Что должно работать	Что ты учишь
0	Пустой CLI	`CMakeLists.txt`, `main.c`, `app_info`	Программа отвечает на `--help` и `--version`.	Ты учишь, что такое сборка, executable, exit code и аргументы командной строки.
1	Безопасные bytes	`util.h/c`: `Buf`, `buf_read`, little-endian helpers	Можно читать искусственный byte array и файл без выхода за границы.	Ты учишь указатели, размеры, `memcpy`, integer overflow.
2	Растущие строки	`StrBuf`, `sb_append`, `sb_appendf`	Можно собрать строку отчета в памяти.	Ты учишь heap, capacity, `realloc`, C-строку с `\0`.
3	Минимальный PE parser	`pe_parse`: MZ, PE signature, AMD64, PE32+	Программа отличает нужный PE32+ AMD64 от неподходящего файла.	Ты учишь binary format, offsets, magic constants.
4	Секции и RVA	`PeSection`, section loop, `pe_rva_to_fileoff`	Можно объяснить, где RVA находится в файле.	Ты учишь массив структур и разницу virtual/raw координат.
5	.pdata	`RuntimeFunction[]` из exception directory	Можно вывести begin/end/unwind RVA каждой функции.	Ты учишь чтение таблиц фиксированной записи по 12 bytes.
6	UNWIND_INFO header и slots	`unwind_parse`, flags, CountOfCodes, opcodes	Можно расшифровать один unwind record и увидеть anomalies.	Ты учишь bit masks, switch, переменный размер записи.
7	Эвристики	`heuristics_analyze`, findings, score, confidence	Технические counters превращаются в объяснимый verdict.	Ты учишь агрегацию, сортировку, hash map, rule-based decision.
8	Отчеты	`report.c`: console, JSON, CSV, HTML	Результат анализа можно сохранить и проверить внешними инструментами.	Ты учишь serialization, escaping, внешний контракт.
9	Полный main pipeline	`main.c`: args, orchestration, cleanup	CLI соединяет все модули и корректно освобождает ресурсы.	Ты учишь lifecycle ресурсов и failure paths.
10	Dynamic mode	`dynamic_win.c` и интеграция report	Runtime evidence добавляется как отдельный слой, не ломая static analysis.	Ты учишь conditional compilation, Windows API и почему запуск чужого кода отделен.

Почему такой порядок лучше для новичка

- Сначала ты учишь безопасно обращаться с bytes, потому что PE и unwind - это просто bytes с правилами.

- PE parser идет раньше unwind parser, потому что unwind RVA невозможно читать без карты секций.

- Heuristics идут после парсеров, потому что они не должны сами лазить в файл.
- Reports идут после модели результата, потому что сериализация должна выводить готовые данные, а не вычислять их.
- `main.c` удобнее доводить в конце: когда модули уже умеют работать отдельно, orchestration становится понятной.
- Dynamic mode последний, потому что он сложнее, Windows-specific и потенциально опаснее обычного чтения файла.

Формула одного учебного шага

1. Сформулируй один маленький контракт: например, `buf_read` возвращает `false`, если чтение выходит за конец.
2. Запиши прототип в `.h`, чтобы стало ясно, какие input и output у функции.
3. Напиши минимальную реализацию в `.c`.
4. Собери проект. Исправь warnings до перехода дальше.
5. Напиши один тест или ручную команду, которая доказывает контракт.
6. Объясни себе вслух: кто владеет памятью, какие границы проверены, какой следующий модуль использует результат.

Главная мысль

Ты не обязан понимать весь EH/Unwind Analyzer в первый день. Но каждая версия должна быть маленькой, рабочей и объяснимой. Если версия 4 понятна, то версия 5 - это не новый монстр, а просто таблица `.pdata`, которая пользуется уже понятной картой RVA.

9. Подробный план написания каждого файла

Ниже - маршрут переписывания проекта руками. Не начинай с полного файла. Каждый модуль разбит на смысл, новые C-понятия, структуры, функции, порядок написания и проверки. Когда написан один шаг, собирай проект. Если сборка сломалась, не иди дальше: вернись к последнему маленькому изменению.

util.h / util.c

Фундамент: чтение файла в память, безопасное чтение байтов, little-endian числа, растущая строка, атомарная запись файла, SHA-256.

Что выучить в C на этом модуле

- указатели и `NULL`
- `size_t`, `uint8_t`, `uint32_t`, `uint64_t`
- `malloc`, `calloc`, `realloc`, `free`
- проверка выхода за границы буфера
- маленький endian
- вариадические функции через `va_list`
- переполнение целых чисел
- владение памятью

Главные структуры и функции

Структуры/константы	Функции
Buf, StrBuf, Sha256Ctx	buf_init_from_file, buf_read, buf_read_u16/u32/u64, sb_init, sb_appendn, sb_appendf, file_write_atomic, sha256_hex

Порядок написания

Ниже каждый пункт раскрыт как учебный шаг. Сначала смотри не на синтаксис, а на смысл: зачем этот объект появляется, какую опасность он убирает, как данные пойдут дальше и чем доказать, что шаг действительно работает.

Шаг 1. Создай `Buf`: два поля, `const uint8_t* data` и `size_t size`. Пока не пиши PE.

Зачем: `Buf` нужен как самый первый договор проекта: любые bytes всегда идут вместе с длиной. Один `uint8_t*` в C не знает, сколько памяти за ним лежит.

Какую проблему решает: Он решает проблему слепого указателя. Без `size` любая функция может случайно читать дальше конца файла, а для парсера недоверенного PE это критическая ошибка.

Как работает: `data` указывает на первый байт, `size` говорит, сколько байтов разрешено читать. Сам `Buf` памятью не владеет: он только описывает область. Владение остается у того, кто выделил массив.

Что писать руками: В `util.h` подключи `<stdint.h>` и `<stddef.h>`, объяви `typedef struct Buf { const uint8_t* data; size_t size; } Buf;`. Пока никаких PE-полей, только универсальная обертка над bytes.

Как проверить: Создай маленький массив `{1,2,3}` и `Buf b = {bytes, 3}`. Ты должен уметь словами сказать: pointer указывает на byte 1, читать можно ровно 3 bytes.

Шаг 2. Напиши `buf_read`: сначала проверки аргументов, потом проверка `off > size`, потом `len > size - off`, затем `memcpy`.

Зачем: `buf_read` - сторож всех последующих парсеров. Через него `pe.c` и `unwind.c` будут брать bytes из файла, не рискуя выйти за границу.

Какую проблему решает: Он решает две проблемы: неверные аргументы (`NULL`) и опасные диапазоны (`off + len` может выйти за конец или переполниться). Поэтому проверка пишется через `len > size - off`.

Как работает: Функция сначала доказывает, что чтение безопасно, и только потом делает `memcpy`. Если доказать безопасность нельзя, она возвращает `false`, а вызывающий код обязан остановиться или записать ошибку.

Что писать руками: В `util.c` напиши `bool buf_read(const Buf* b, size_t off, void* out, size_t len)`. Внутри: `if (!b || !out)`, затем `if (off > b->size)`, затем `if (len > b->size - off)`, затем `memcpy(out, b->data + off, len)`.

Как проверить: Проверь три случая: чтение внутри буфера возвращает `true`; чтение с `off == size` и `len == 0` допустимо; чтение `len == 1` при `off == size` возвращает `false`.

Шаг 3. Напиши `le16`, `le32`, `le64`. Научись видеть, что байты файла не обязаны совпадать с порядком байтов CPU.

Зачем: `le16`, `le32`, `le64` нужны, потому что файл хранит числа как последовательность bytes, а не как готовые переменные C.

Какую проблему решает: Это решает проблему неявного доверия CPU и выравниванию. Если просто привести `uint8_t*` к `uint32_t*`, можно получить undefined behavior или зависимость от платформы.

Как работает: Little-endian означает: младший байт числа лежит первым. Число `0x12345678` в файле будет bytes `78 56 34 12`. Функция собирает число сдвигами и `|`.

Что писать руками: Напиши `static uint16_t le16(const uint8_t x[2])`, `static uint32_t le32(const uint8_t x[4])`, `static uint64_t le64(const uint8_t x[8])`. Делай cast перед shift: `((uint32_t)x[3] << 24)`.

Как проверить: Проверь на bytes `{0x78,0x56,0x34,0x12}`: `le32` должен вернуть `0x12345678`. Если ты можешь нарисовать это на бумаге, шаг понят.

Шаг 4. Добавь `buf_read_u16/u32/u64`: читаешь временный массив байтов и переводишь в число.

Зачем: `buf_read_u16/u32/u64` соединяют безопасное чтение диапазона и перевод little-endian bytes в число.

Какую проблему решает: Это решает проблему повторения. PE parser будет десятки раз читать `uint16_t` и `uint32_t`; если каждый раз писать `buf_read` и `le32` вручную, легко ошибиться.

Как работает: Функция создает временный массив bytes на stack, вызывает `buf_read`, а потом передает bytes в `le16/le32/le64`. Наружу выходит уже нормальное число C.

Что писать руками: Напиши три функции с одинаковой формой: проверить `out`, объявить `uint8_t tmp[N]`, вызвать `buf_read(b, off, tmp, sizeof tmp)`, если не вышло - `false`, иначе `*out = leN(tmp)`.

Как проверить: Проверь, что `buf_read_u32` не меняет `*out`, если чтение не удалось, либо хотя бы возвращает `false` и вызывающий код не использует старое значение.

Шаг 5. Напиши чтение файла: `fopen`, `fseek`, `ftell`, выделение памяти, `fread`, закрытие файла.

Зачем: Чтение файла нужно, чтобы превратить путь от пользователя в `Buf`, с которым уже могут работать PE parser и остальные модули.

Какую проблему решает: Это решает проблему границы между операционной системой и анализатором. Дальше весь проект хочет работать с bytes в памяти, а не таскать везде `FILE*`.

Как работает: `fopen` открывает файл, `fseek/ftell` узнают размер, `malloc` выделяет массив, `fread` заполняет его, затем файл закрывается. Важно закрывать файл и при успехе, и при ошибке.

Что писать руками: Напиши `bool buf_init_from_file(const char* path, Buf* out, uint8_t** owned_mem)`. `out->data` должен указывать на выделенную память, `out->size` на размер, а `*owned_mem` вернуть владельцу для будущего `free`.

Как проверить: Проверь пустой файл: размер 0, функция не падает, `data` можно сделать не `NULL` через выделение хотя бы 1 byte или аккуратно обработать нулевой размер по единому договору проекта.

Шаг 6. Сделай `StrBuf`: `s`, `len`, `cap`, `failed`. Не добавляй строку без места под завершающий ноль.

Зачем: `StrBuf` нужен, потому что отчеты неизвестной длины: JSON, HTML и CSV растут по мере добавления секций, функций и findings.

Какую проблему решает: Он решает проблему обычных C-строк: у `char*` нет длины и емкости. Без `len/cap` легко перезаписать память или забыть место под завершающий `\0`.

Как работает: `s` указывает на выделенную строку, `len` - сколько полезных символов уже записано, `cap` - сколько всего выделено, `failed` - флаг, что дальнейшие append больше нельзя считать надежными.

Что писать руками: В `util.h` объяви `typedef struct StrBuf { char* s; size_t len; size_t cap; bool failed; } StrBuf;` и функции `sb_init`, `sb_free`, `sb_appendn`, `sb_append`, `sb_appendf`, `sb_ok`.

Как проверить: После `sb_init` строка должна быть валидной C-строкой: `s[0] == '\0'`, `len == 0`, `cap >= 1`, `failed == false`.

Шаг 7. Реализуй `sb_reserve` через `realloc`. Проверь, что рост емкости не переполняет `SIZE_MAX`.

Зачем: `sb_reserve` - механизм роста памяти для `StrBuf`. Все append зависят от него.

Какую проблему решает: Он решает проблему нехватки места и переполнения размера. Если нужно `len + n + 1`, надо заранее проверить, что сложение не перешло через `SIZE_MAX`.

Как работает: Функция сравнивает нужный размер с `cap`. Если места хватает - ничего не делает. Если не хватает - увеличивает емкость, обычно удваивая, но каждый шаг проверяет overflow.

Что писать руками: Напиши `static bool sb_reserve(StrBuf* sb, size_t need)`. Используй временный pointer: `char* p = realloc(sb->s, new_cap)`. Старый `sb->s` меняй только если `p != NULL`.

Как проверить: Искусственно попроси большой размер или смоделируй ошибку: после неудачи `failed` должен стать `true`, а старый buffer не должен быть потерян.

Шаг 8. Реализуй `sb_appendn`, потом `sb_append`, потом `sb_appendf`.

Зачем: `sb_appendn`, `sb_append`, `sb_appendf` - это три уровня добавления текста: bytes с длиной, обычная C-строка и форматированный текст.

Какую проблему решает: Они решают проблему безопасной сборки отчетов. Writer не должен думать о `realloc` и `\0`; он должен сказать: добавь такой текст.

Как работает: `sb_appendn` резервирует `len + n + 1`, копирует bytes в `s + len`, увеличивает `len`, ставит `s[len] = '\0'`. `sb_append` вызывает `strlen`. `sb_appendf` сначала узнает нужную длину через `vsnprintf`.

Что писать руками: Пиши именно в таком порядке: сначала `appendn`, потому что он базовый; потом `append`, потому что это thin wrapper; потом `appendf`, потому что там появятся `va_list`, `va_start`, `va_end`.

Как проверить: Проверь строку после трех добавлений: содержимое правильное, `len == strlen(s)`, последний byte после текста равен `\0`.

Шаг 9. Сделай `file_write_atomic`: сначала `.tmp.<pid>`, потом замена итогового файла.

Зачем: `file_write_atomic` нужен для отчетов: лучше не создать файл вообще, чем оставить битый JSON или HTML, если запись оборвалась посередине.

Какую проблему решает: Он решает проблему частично записанного результата. Пользователь может открыть отчет после сбоя и подумать, что анализатор выдал валидный вывод, хотя файл оборвался.

Как работает: Функция пишет во временный путь рядом с итоговым файлом, закрывает его, проверяет ошибки, затем заменяет итоговый файл через rename/MoveFileEx. Если что-то сломалось, временный файл удаляется.

Что писать руками: Сначала собери имя `target + .tmp.<pid>`, открой временный файл, запиши все bytes, закрой, затем сделай замену. На Windows учти, что заменить существующий файл может потребовать отдельной функции.

Как проверить: Проверь, что после успешной записи итоговый файл содержит ровно переданные bytes, а временный файл не остался рядом.

Шаг 10. SHA-256 можно написать последним: это самостоятельный алгоритм, он нужен для идентичности входного файла в отчетах.

Зачем: SHA-256 нужен не для поиска признаков, а для идентичности входного файла в отчете.

Какую проблему решает: Он решает проблему доказуемости: если на защите или в тестах обсуждается отчет, hash показывает, к какому именно бинарнику он относится. Имя файла может совпадать, содержимое - нет.

Как работает: Алгоритм читает bytes блоками, обновляет внутреннее состояние и в конце выдает 32 bytes digest, который печатается как 64 hex-символа.

Что писать руками: Пиши SHA-256 последним, потому что это отдельный алгоритм. Сначала можно временно вернуть placeholder, но в финале нужны `sha256_init`, `sha256_update`, `sha256_final`, `sha256_hex`.

Как проверить: Проверь на известном тест-векторе: SHA-256 пустой строки должен быть `e3b0c44298fc1c149afbf4c8996fb924...`.

Проверки перед переходом дальше

- Пустой файл читается без падения, размер равен 0, но память все равно не `NULL`.
- `buf_read` возвращает `false`, если просишь байты за концом.
- `sb_appendf` после нескольких добавлений оставляет строку с `\0` в конце.
- После ошибки выделения `StrBuf.failed` остается `true`, а вызывающий код может остановиться.

Как объяснять себе каждую строку

Для каждой строки задавай четыре вопроса: какой объект она читает, какой объект меняет, какой failure path возможен, какой следующий слой будет использовать результат. Если строка только украшает код и не проходит эти вопросы, скорее всего ты пока ее не понял.

```
bool buf_read(const Buf* b, size_t off, void* out, size_t len) {
    if (!b || !out) return false;
    if (off > b->size) return false;
    if (len > b->size - off) return false;
    memcpy(out, b->data + off, len);
    return true;
}
```

pe.h / pe.c

Строит минимальную модель PE32+ AMD64: заголовки, секции, exception directory, проверки диапазонов и перевод RVA в file offset.

Что выучить в C на этом модуле

- бинарный формат как контракт
- структура как модель предметной области
- магические числа и именованные константы
- границы массива секций
- виртуальный адрес против смещения в файле
- валидация до доверия

Главные структуры и функции

Структуры/константы	Функции
PeSection, PeImage	pe_parse, pe_free, pe_find_section_for_rva, pe_rva_to_fileoff

Порядок написания

Ниже каждый пункт раскрыт как учебный шаг. Сначала смотри не на синтаксис, а на смысл: зачем этот объект появляется, какую опасность он убирает, как данные пойдут дальше и чем доказать, что шаг действительно работает.

Шаг 1. Начни с констант `MZ`, `PE\0\0`, `AMD64`, `PE32+`, индекса exception directory.

Зачем: Константы `MZ`, `PE\0\0`, AMD64 и PE32+ нужны, чтобы код говорил языком формата, а не магическими числами.

Какую проблему решает: Они решают проблему непонятного кода: `0x5A4D` без имени ничего не объясняет, а `PE_MAGIC_MZ` сразу говорит, какую сигнатуру проверяем.

Как работает: PE parser будет читать числа из файла и сравнивать с этими константами. Если сигнатура не совпала, это не наш формат или файл поврежден.

Что писать руками: В `pe.c` или `pe.h` объяви именованные `#define`/`enum` для сигнатур, machine type, magic PE32+ и индекса exception directory.

Как проверить: Проверь, что файл без `MZ` отвергается до чтения NT headers.

Шаг 2. Опиши `PeSection`: имя, virtual address, virtual size, raw pointer, raw size, characteristics.

Зачем: `PeSection` нужен как модель одной секции PE: где она лежит в памяти, где ее bytes лежат в файле и какие у нее права.

Какую проблему решает: Он решает проблему смешивания координат. Без отдельной структуры ты будешь путать RVA, raw pointer, virtual size и raw size.

Как работает: Каждый section header из PE превращается в один `PeSection`; дальше RVA mapping и heuristics будут работать уже с этим массивом.

Что писать руками: Опиши поля `name[9]`, `virtual_address`, `virtual_size`, `raw_ptr`, `raw_size`, `characteristics`.

Как проверить: Проверь на одном section header: имя завершается нулем, а числовые поля совпадают с bytes файла.

Шаг 3. Опиши `PeImage`: глобальные поля образа, массив секций, счетчики проблем.

Зачем: `PeImage` нужен как общий результат PE parsing: не просто секции, а весь контекст образа.

Какую проблему решает: Он решает проблему передачи десятка отдельных переменных между функциями. Следующим модулям нужен один объект, который описывает разобранный PE.

Как работает: `PeImage` владеет массивом секций, хранит image size, alignments, exception directory и счетчики structural problems.

Что писать руками: В `pe.h` объяви `PeImage` и сразу рядом прототип `pe_free`, чтобы договор владения памятью был виден рядом со структурой.

Как проверить: После `pe_parse` можно вывести `section_count` и exception directory RVA/size без повторного чтения файла.

Шаг 4. В `pe_parse` первым делом зануляй `out`, чтобы при ошибке не осталось мусора.

Зачем: Зануление `out` в начале `pe_parse` нужно, чтобы ошибка на середине не оставила мусорные указатели.

Какую проблему решает: Это решает проблему cleanup: если `pe_free` вызовут после частичной ошибки, он должен видеть либо валидную память, либо нули.

Как работает: Функция ставит `*out` в известное состояние, затем постепенно заполняет поля. Любой failure path остается безопасным.

Что писать руками: Начни `pe_parse` с проверки аргументов и `memset(out, 0, sizeof *out)`.

Как проверить: Смоделируй ошибку после начала парсинга и убедись, что `pe_free` не падает.

Шаг 5. Прочитай `e_lfanew` из DOS header: это смещение до NT headers.

Зачем: `e_lfanew` - мост от DOS header к настоящим NT headers PE.

Какую проблему решает: Он решает проблему расположения PE header: он не обязан начинаться сразу после первых bytes файла.

Как работает: Parser читает 32-битное смещение из DOS header, проверяет, что оно внутри файла, и от него читает `PE\0\0`.

Что писать руками: Через `buf_read_u32(b, 0x3c, &lfanew)` получи offset, затем проверяй диапазон перед чтением signature.

Как проверить: Проверь файл, где `e_lfanew` указывает за конец: parser должен вернуть ошибку, а не читать мусор.

Шаг 6. Проверь сигнатуру PE и COFF Machine. Проект сознательно принимает только AMD64.

Зачем: Проверка PE signature и Machine нужна, чтобы не применять x64 unwind rules к чужому формату.

Какую проблему решает: Она решает проблему ложного анализа. x86 PE или не-PE файл могут иметь похожие bytes, но правила `.pdata` и `UNWIND_INFO` будут другими.

Как работает: После `PE\0\0` идет COFF header. Из него читается Machine и NumberOfSections; Machine должен быть AMD64.

Что писать руками: Прочитай signature, затем `Machine`, затем `NumberOfSections`. Если Machine не AMD64 - верни понятную ошибку.

Как проверить: Проверь, что x86 PE отвергается сообщением про unsupported machine.

Шаг 7. Прочитай optional header PE32+: image base, alignment, image/header size, число data directories.

Зачем: Optional header PE32+ хранит размеры и таблицу data directories, без которых нельзя найти exception directory.

Какую проблему решает: Он решает проблему контекста: section headers сами по себе не говорят ImageBase, SizeOfImage и directories.

Как работает: Parser читает magic PE32+, image base, alignment, size of image, number of directories и проверяет, что optional header достаточно большой.

Что писать руками: Добавь чтение полей только после проверки `SizeOfOptionalHeader`; не читай directory, если его индекс отсутствует.

Как проверить: Проверь PE с малым optional header: parser должен отказать аккуратно.

Шаг 8. Найди exception directory по индексу 3. Сохрани RVA и размер.

Зачем: Exception directory - точка входа в `.pdata`, то есть в главную предметную область проекта.

Какую проблему решает: Она решает вопрос: где находится таблица `RUNTIME_FUNCTION` и сколько bytes она занимает.

Как работает: В PE data directories entry 3 хранит RVA и size. Эти значения сохраняются в `PeImage`, но сами записи читает уже `unwind.c`.

Что писать руками: Прочитай directory RVA/size по индексу 3 и сохрани в `pe->exception_rva`, `pe->exception_size`.

Как проверить: Проверь PE без exception directory: проект не падает, а дальше heuristics понимает, что runtime functions нет.

Шаг 9. Прочитай секции циклом по 40 байт на заголовок.

Зачем: Цикл по секциям нужен, чтобы построить карту файла.

Какую проблему решает: Он решает проблему адресации: почти все дальнейшие RVA придется искать внутри section table.

Как работает: Каждая секция занимает 40 bytes header. Parser читает имя, virtual size/address, raw size/pointer и characteristics.

Что писать руками: Выдели массив `PeSection* sections = calloc(section_count, sizeof *sections)` и заполни его в цикле.

Как проверить: Проверь, что `section_count == 0` или слишком большой count обрабатывается без переполнения выделения.

Шаг 10. Посчитай структурные проблемы: выход raw за файл, virtual за SizeOfImage, overlaps, alignment.

Зачем: Structural counters нужны, чтобы parser не только строил карту, но и фиксировал странности самой PE-структуры.

Какую проблему решает: Они решают проблему потери evidence. Если секция выходит за файл или пересекается с другой, это важно для heuristics и отчета.

Как работает: После чтения секций отдельным проходом проверь raw bounds, virtual bounds, overlaps и alignment. Не смешивай это с чтением полей.

Что писать руками: Добавь счетчики в `PeImage` и увеличивай их при каждом нарушении.

Как проверить: Создай искусственную секцию с raw range за концом файла и проверь, что счетчик стал больше нуля.

Шаг 11. Реализуй `pe_find_section_for_rva` и `pe_rva_to_fileoff`: это карта, без которой unwind parser слепой.

Зачем: `pe_find_section_for_rva` и `pe_rva_to_fileoff` нужны, чтобы все последующие модули могли перейти от адреса в памяти к bytes на диске.

Какую проблему решает: Они решают главную координатную проблему PE: RVA не равен file offset.

Как работает: Сначала ищется секция, чей virtual range содержит RVA. Затем `delta = rva - section.virtual_address`; если `delta < raw_size`, file offset равен `raw_ptr + delta`.

Что писать руками: Напиши функцию поиска секции отдельно от функции перевода RVA, чтобы их можно было тестировать независимо.

Как проверить: Проверь RVA в начале секции, внутри секции и в virtual tail за `raw_size`: последний случай должен вернуть `false`.

Проверки перед переходом дальше

- Файл не с `MZ` отвергается с понятной ошибкой.
- PE32/x86 отвергается, потому что проект про PE32+ AMD64.
- RVA внутри заголовков мапится напрямую.
- RVA в виртуальном хвосте секции не мапится в файл.

Как объяснять себе каждую строку

Для каждой строки задавай четыре вопроса: какой объект она читает, какой объект меняет, какой failure path возможен, какой следующий слой будет использовать результат. Если строка только украшает код и не проходит эти вопросы, скорее всего ты пока ее не понял.

RVA -> file offset:
1. Найти section, где virtual_address <= RVA < virtual_address + span.
2. delta = RVA - section.virtual_address.
3. Если delta >= raw_size, bytes нет в файле: это virtual tail.
4. file_offset = raw_ptr + delta.

unwind.h / unwind.c

Читает `.pdata` как массив `RUNTIME_FUNCTION` и декодирует `UNWIND_INFO` V1/V2, операции пролога, handler и chain.

Что выучить в C на этом модуле

- битовые поля
- маски флагов
- переменный размер записи
- `switch` по opcode
- цикл с шагом, зависящим от данных
- строгие и совместимые проверки ABI
- bounded recursion через chain

Главные структуры и функции

Структуры/константы	Функции
RuntimeFunction, UnwindSummary, UnwindParseOpts	unwind_read_runtime_functions_ex, unwind_parse, slots_for_code, unwind_parse_fail, unwind_reg_name, unwind_flags_to_string

Порядок написания

Ниже каждый пункт раскрыт как учебный шаг. Сначала смотри не на синтаксис, а на смысл: зачем этот объект появляется, какую опасность он убирает, как данные пойдут дальше и чем доказать, что шаг действительно работает.

Шаг 1. Сначала реализуй только чтение `RUNTIME_FUNCTION`: каждая запись 12 байт, три `uint32_t` RVA.

Зачем: Чтение `RUNTIME_FUNCTION` сначала нужно потому, что `.pdata` - это список входов в `UNWIND_INFO`.

Какую проблему решает: Оно решает проблему навигации: без begin/end/unwind RVA непонятно, какие функции вообще описаны в таблице.

Как работает: Каждая запись ровно 12 bytes: BeginAddress, EndAddress, UnwindInfoAddress, все как little-endian `uint32_t` RVA.

Что писать руками: Напиши цикл по exception directory size / 12, через `buf_read_u32` считывай три поля в `RuntimeFunction`.

Как проверить: Проверь directory size не кратный 12: лишние bytes должны стать anomaly или ошибкой, а не отдельной записью.

Шаг 2. В `UnwindSummary` добавь только базовые поля: version, flags, prolog, count, frame.

Зачем: `UnwindSummary` сначала должен содержать только базовые поля, чтобы ты не пытался понять весь ABI за один раз.

Какую проблему решает: Он решает проблему сложности: parser может начать с header, а stack effects, handlers и chains добавить позже.

Как работает: Summary - это сжатая расшифровка `UNWIND_INFO`, которую потом читает heuristics и report.

Что писать руками: Добавь поля version, flags, prolog size, count of codes, frame register/offset, parse_error, anomalies.

Как проверить: Проверь один минимальный `UNWIND_INFO`: header читается, даже если codes пока не интерпретируются полностью.

Шаг 3. Реализуй чтение header `UNWIND_INFO`: первый байт делится на version и flags.

Зачем: Header `UNWIND_INFO` нужен как оглавление записи: он говорит версию, flags и сколько slots читать дальше.

Какую проблему решает: Он решает проблему переменного размера: без CountOfCodes ты не знаешь, где заканчивается массив unwind codes.

Как работает: Первый byte делится на version и flags, четвертый byte - на frame register и frame offset.

Что писать руками: Прочитай 4 bytes header, выдели bits через `&`, `>>`, сохрани в summary.

Как проверить: Проверь version 3: parser должен отметить unsupported version, если проект поддерживает только V1/V2.

Шаг 4. Сделай `slots_for_code`: для каждого unwind opcode верни, сколько 2-байтовых slots он занимает.

Зачем: `slots_for_code` нужен, потому что разные UWOP занимают разное количество 2-байтовых slots.

Какую проблему решает: Он решает проблему правильного шага цикла. Если всегда делать `i++`, parser быстро потеряет границы записи.

Как работает: Функция смотрит opcode и opinfo и возвращает 1, 2 или 3 slots. Для неизвестного opcode возвращает ошибку.

Что писать руками: Напиши `switch (op)` и явно перечисли поддерживаемые UWOP.

Как проверить: Проверь opcode, которому нужны 3 slots, при `CountOfCodes == 1`: parser должен увидеть truncation.

Шаг 5. Напиши цикл `for (i = 0; i < count;)`: читаешь slot, выделяешь CodeOffset, Op, OpInfo, увеличиваешь `i += slots`.

Зачем: Цикл `for (i = 0; i < count;)` нужен, потому что шаг зависит от содержимого записи.

Какую проблему решает: Он решает проблему чтения массива переменной структуры внутри формально одинаковых slots.

Как работает: На каждой итерации parser читает slot, выделяет CodeOffset, UnwindOp, OpInfo, вычисляет slots и проверяет `i + slots <= count`.

Что писать руками: Не увеличивай `i` в заголовке цикла; увеличивай его после успешной интерпретации opcode.

Как проверить: Проверь два opcodes подряд: второй должен читаться с правильного индекса.

Шаг 6. Для каждого opcode сначала добавь распознавание и текст, потом вычисление stack/register effects.

Зачем: Интерпретация opcode нужна, чтобы отчет показывал не просто bytes, а смысл: push register, allocate stack, save register.

Какую проблему решает: Она решает проблему человеческого чтения. Реверс-инженер должен видеть, что описывает unwind metadata.

Как работает: Для каждого UWOP summary обновляет счетчики stack allocation, saved registers, frame usage или textual details.

Что писать руками: Сначала добавь распознавание и текст, потом математические эффекты stack/register, чтобы не смешивать две сложности.

Как проверить: Проверь `UWOP_ALLOC_SMALL`: по opinfo должен получиться ожидаемый размер stack allocation.

Шаг 7. Добавь anomaly flags: unsupported version, truncated, bad opcode, bad operand, code offset, alignment.

Зачем: Anomaly flags нужны, чтобы parser не терял причину странности.

Какую проблему решает: Они решают проблему одного общего `false`: отчету важно различать truncation, bad opcode, bad operand и alignment.

Как работает: Флаги накапливаются через `anomalies |= FLAG`, поэтому одна запись может иметь несколько проблем.

Что писать руками: Определи bit masks и ставь их в конкретных местах проверки.

Как проверить: Проверь запись с bad opcode и truncation: обе причины должны сохраниться, если обе обнаружены.

Шаг 8. Добавь handler tail: если флаги EHANDLER/UHANDLER, после slots лежит handler RVA.

Зачем: Handler tail нужен, потому что EHANDLER/UHANDLER flags означают, что после unwind codes лежит RVA обработчика.

Какую проблему решает: Он решает проблему связи metadata с exception handling: именно handler часто важен для подозрительных кейсов.

Как работает: После массива codes есть выравнивание до четного количества slots, затем 4-byte handler RVA.

Что писать руками: Вычисли offset tail аккуратно: `code_slots_rounded = (count + 1) & ~1`, затем читай handler RVA.

Как проверить: Проверь EHANDLER запись: handler RVA должен попасть в summary и потом в отчет.

Шаг 9. Добавь chain tail: если CHAININFO, после slots лежит еще один `RUNTIME_FUNCTION`.

Зачем: Chain tail нужен для `UNW_FLAG_CHAININFO`: запись может ссылаться на другой `RUNTIME_FUNCTION`.

Какую проблему решает: Он решает проблему составных unwind records. Без chain parser будет считать хвост handler-ом или мусором.

Как работает: При CHAININFO после codes лежат три `uint32_t`, то есть embedded `RUNTIME_FUNCTION`.

Что писать руками: Добавь отдельную ветку: если CHAININFO, не читай handler RVA, а читай chained begin/end/unwind.

Как проверить: Проверь запись с CHAININFO: parser не должен одновременно считать ее EHANDLER.

Шаг 10. Включи follow-chain только после того, как одиночный record работает. Ограничение depth защищает от циклов.

Зачем: Follow-chain надо включать только после одиночной записи, потому что это рекурсивная логика.

Какую проблему решает: Он решает проблему полноты анализа, но добавляет риск циклов и глубоких цепочек.

Как работает: Parser идет по chained unwind RVA с ограничением depth и набором sanity checks.

Что писать руками: Добавь `max_chain_depth` в options и уменьши его при каждом переходе.

Как проверить: Проверь искусственную self-chain: parser должен остановиться по лимиту, а не зависнуть.

Проверки перед переходом дальше

- V3 не парсится как V1/V2: это отдельный формат.
- Если opcode требует 3 slots, а `CountOfCodes` заканчивается раньше, парсер возвращает ошибку.
- Невыровненный `UNWIND_INFO` отмечается anomaly, но не обязательно делает запись фатальной.
- Parse error сохраняется в `UnwindSummary.parse_error`, чтобы отчет знал точную причину.

Как объяснять себе каждую строку

Для каждой строки задавай четыре вопроса: какой объект она читает, какой объект меняет, какой failure path возможен, какой следующий слой будет использовать результат. Если строка только украшает код и не проходит эти вопросы, скорее всего ты пока ее не понял.

```
slot:
  low byte  = CodeOffset
  high byte = (OpInfo << 4) | UnwindOp

while i < CountOfCodes:
  read slot i
  determine slots_for_code(op, opinfo)
  validate i + slots <= CountOfCodes
  interpret operands
  i += slots
```

heuristics.h / heuristics.c

Превращает наблюдения в findings, score, confidence, evidence level, verdict и weakening rules.

Что выучить в C на этом модуле

- инженерная эвристика против статистической вероятности
- массивы структур фиксированного размера
- `qsort` и компаратор
- простая hash map с open addressing
- отношения и проценты
- семейства доказательств
- снижение уверенности при слабом контексте

Главные структуры и функции

Структуры/константы	Функции
HeuristicFinding, HeuristicMitigation, HeuristicReport, HMap	heuristics_analyze, add_finding, add_mitigation, finalize_verdict, hmap_inc, heuristics_free

Порядок написания

Ниже каждый пункт раскрыт как учебный шаг. Сначала смотри не на синтаксис, а на смысл: зачем этот объект появляется, какую опасность он убирает, как данные пойдут дальше и чем доказать, что шаг действительно работает.

Шаг 1. Сначала перенеси счетчики из `PeImage`: section problems и exception directory problems.

Зачем: Счетчики из `PeImage` надо перенести первыми, потому что структурные проблемы секций - базовый слой evidence.

Какую проблему решает: Это решает проблему потери контекста: unwind может быть странным именно потому, что секции уже повреждены.

Как работает: Heuristics читает готовые counters и превращает их в внутренние признаки, не перечитывая PE.

Что писать руками: В начале `heuristics_analyze` скопируй или учти raw_oob, virtual_oob, overlaps, exception directory problems.

Как проверить: Проверь PE с одной raw-oob секцией: должен появиться соответствующий finding или счетчик.

Шаг 2. Пройди все `RuntimeFunction`: invalid ranges, out-of-bounds begin/end, out-of-bounds unwind RVA, unsorted.

Зачем: Проход по `RuntimeFunction` нужен, чтобы проверить саму `.pdata`: begin/end ranges, unwind RVA, сортировку.

Какую проблему решает: Он решает проблему таблицы, которая формально есть, но указывает за пределы образа или противоречит себе.

Как работает: Для каждой записи проверяются `begin < end`, оба адреса внутри image, unwind RVA доступен, порядок не убывает.

Что писать руками: Напиши цикл по `rf_count` и отдельные counters для каждого класса нарушения.

Как проверить: Проверь запись `begin >= end`: она должна повышать invalid range counter.

Шаг 3. Параллельно считай reuse `UnwindInfoRVA` через маленькую hash map.

Зачем: Reuse `UnwindInfoRVA` нужно считать, потому что много функций, указывающих на один unwind info, может быть нормой или признаком упаковки.

Какую проблему решает: Это решает проблему частотного признака: одиночное совпадение неинтересно, массовое совпадение уже требует внимания.

Как работает: Маленькая hash map считает, сколько раз встречается каждый unwind RVA, и потом ищется максимум/доля.

Что писать руками: Реализуй `HMap` с open addressing: key - RVA, value - count.

Как проверить: Проверь три одинаковых RVA: max reuse должен стать 3.

Шаг 4. Если unwind parse failed или `anomalies` содержит не только ABI warnings, увеличивай счетчики integrity.

Зачем: Parse failures и anomalies из `UnwindSummary` надо учитывать отдельно от structural PE problems.

Какую проблему решает: Это решает проблему смешивания семей evidence: поврежденная секция и плохой unwind opcode - разные причины.

Как работает: Heuristics смотрит `summary.parse_error` и `summary.anomalies`, но strict ABI warnings может учитывать мягче, чем hard truncation.

Что писать руками: Добавь counters: parse_fail_count, hard_anomaly_count, abi_warning_count.

Как проверить: Проверь, что один strict ABI warning сам по себе не делает high score.

Шаг 5. Проверь placement: где лежит unwind info и где лежит handler target.

Зачем: Placement checks нужны, чтобы понять, где лежит unwind info и handler: в read-only metadata, executable, writable section.

Какую проблему решает: Это решает проблему контекста. Один и тот же handler RVA выглядит по-разному, если он указывает в `.text` или writable data.

Как работает: Через `pe_find_section_for_rva` определяется секция для unwind info и handler target, затем смотрятся characteristics.

Что писать руками: Добавь counters для writable metadata, executable metadata, writable handler, non-executable handler.

Как проверить: Проверь handler в writable секции: finding должен объяснять именно placement.

Шаг 6. Отсортируй копию begin/end, чтобы найти overlaps и duplicate begins.

Зачем: Сортировка копии begin/end нужна, чтобы найти overlaps и duplicate begins независимо от исходного порядка.

Какую проблему решает: Она решает проблему анализа отношений между записями `.pdata`.

Как работает: Копия массива сортируется по begin, потом соседние записи сравниваются на overlap и duplicate begin.

Что писать руками: Сделай отдельную структуру range и `qsort` comparator.

Как проверить: Проверь две пересекающиеся функции: overlap counter должен увеличиться.

Шаг 7. Добавь findings: code, family, severity, message, representative RVA, affected count.

Зачем: Findings нужны, чтобы заменить набор counters человеческими сообщениями.

Какую проблему решает: Они решают проблему отчета: пользователю нужен code, severity, family, affected count и representative RVA, а не только `score=40`.

Как работает: Каждый finding добавляется через одну функцию, которая контролирует лимит массива и формат message.

Что писать руками: Напиши `add_finding(report, code, family, severity, rva, count, message)`.

Как проверить: Проверь переполнение лимита findings: программа не должна писать за пределы массива.

Шаг 8. Собери score по матрице весов. Важно: score это triage, не доказательство.

Зачем: Score нужен как triage-оценка, но не как доказательство вредоносности.

Какую проблему решает: Он решает проблему сортировки случаев: что смотреть первым, что слабее, где нужно ручное внимание.

Как работает: Weights добавляются за разные findings, затем итог ограничивается 0..100.

Что писать руками: Опиши веса константами и собирай score после того, как counters и findings уже известны.

Как проверить: Проверь, что score никогда не становится больше 100.

Шаг 9. Посчитай confidence: базовое значение, бонусы за семейства/сильные сигналы, затем weakening rules.

Зачем: Confidence нужно считать отдельно, потому что высокий score по одному шаткому признаку не равен надежному выводу.

Какую проблему решает: Это решает проблему честности интерпретации. Confidence отвечает не 'насколько плохо', а 'насколько уверенно мы читаем evidence'.

Как работает: Бонусы идут за разные семейства и сильные признаки, штрафы - за single signal, small sample, placement-only.

Что писать руками: После score добавь расчет confidence и список mitigations, которые объясняют снижение.

Как проверить: Проверь один слабый finding: confidence должен быть ниже, чем при нескольких независимых семействах.

Шаг 10. Сформируй `verdict`, `evidence_level` и `requires_manual_review`.

Зачем: Verdict и manual review нужны как финальная формулировка для человека.

Какую проблему решает: Они решают проблему последнего шага: отчет должен сказать, что делать с результатом, а не только показать числа.

Как работает: Verdict берется по thresholds score, evidence level - по количеству сильных/средних признаков, manual review включается для неоднозначных или сильных случаев.

Что писать руками: Сделай отдельную `finalize_verdict`, чтобы thresholds были в одном месте.

Как проверить: Проверь границы 15, 40, 70: verdict должен меняться предсказуемо.

Проверки перед переходом дальше

- Одиночный необычный section name не должен превращать файл в `high_suspicion`.
- Writable metadata и handler writable вместе усиливают case.
- Strict ABI warnings не повышают score сами по себе.
- Все findings влезают в `HEURISTICS_MAX_FINDINGS`, а переполнение не ломает details.

Как объяснять себе каждую строку

Для каждой строки задавай четыре вопроса: какой объект она читает, какой объект меняет, какой failure path возможен, какой следующий слой будет использовать результат. Если строка только украшает код и не проходит эти вопросы, скорее всего ты пока ее не понял.

```
observation counters -> finding -> score -> evidence level -> confidence -> verdict

score answers: "насколько это подозрительно по весам?"
confidence answers: "насколько надежна эта интерпретация с учетом контекста?"
```

report.h / report.c

Сериализует модель в console, CSV, Ghidra CSV, JSON и HTML. Отчеты являются внешним контрактом проекта.

Что выучить в C на этом модуле

- serialization
- CSV escaping
- JSON escaping и UTF-8
- HTML escaping и Content Security Policy
- модель отчета и schema
- атомарная запись результата
- ручной план валидации

Главные структуры и функции

Структуры/константы	Функции
ReportMetadata, AssessmentSummary, ManualValidationSteps	report_print_brief, report_write_csv, report_write_json_ex, report_write_html, report_print_heuristics

Порядок написания

Ниже каждый пункт раскрыт как учебный шаг. Сначала смотри не на синтаксис, а на смысл: зачем этот объект появляется, какую опасность он убирает, как данные пойдут дальше и чем доказать, что шаг действительно работает.

Шаг 1. Начни с console: вывести path, Machine, ImageBase, SizeOfImage, Exception Directory, count.

Зачем: Сначала нужна модель отчета, потому что writers должны получать один и тот же набор данных.

Какую проблему решает: Это решает проблему расхождения output formats: console, JSON и HTML не должны считать разные verdict.

Как работает: Metadata и summary собираются один раз, а writer только превращает их в текст.

Что писать руками: В `report.h` опиши структуры metadata/summary или прототипы writer functions с одинаковыми входами.

Как проверить: Проверь, что JSON и console показывают один score.

Шаг 2. Добавь CSV без Ghidra mode: одна строка на function, явные колонки.

Зачем: Console output нужен для быстрого запуска без файлов.

Какую проблему решает: Он решает проблему первого контакта пользователя с программой: команда должна сразу показать verdict и главные findings.

Как работает: Writer печатает короткий summary, затем список findings в порядке важности.

Что писать руками: Напиши функцию, которая принимает готовый report и `FILE*` или путь, не перечитывая PE.

Как проверить: Проверь запуск без `--json`: пользователь все равно видит результат.

Шаг 3. Добавь Ghidra CSV: begin, end, generated name, comment.

Зачем: CSV нужен для таблиц, фильтрации и импорта в другие инструменты.

Какую проблему решает: Он решает проблему плоского представления findings: одна строка - один finding или одна runtime function.

Как работает: CSV cell должен экранировать кавычки, запятые и переносы строк.

Что писать руками: Напиши helper `csv_escape_cell` и используй его в каждом поле.

Как проверить: Проверь message с запятой и кавычкой: CSV должен открываться корректно.

Шаг 4. Сделай escape функций для CSV, JSON и HTML. Не доверяй строкам из входа.

Зачем: Ghidra CSV нужен отдельно, потому что это не просто таблица, а формат для работы в reverse engineering инструменте.

Какую проблему решает: Он решает проблему переноса findings в Ghidra как bookmarks/comments/function hints.

Как работает: Writer выбирает поля, которые нужны importer-у: RVA, тип события, текст комментария.

Что писать руками: Не смешивай обычный CSV и Ghidra CSV в одной функции; у них разный внешний контракт.

Как проверить: Проверь, что строка содержит RVA в ожидаемом формате.

Шаг 5. Опиши `ReportMetadata`: версия, schema, timestamp, sha256, размер, command line, build.

Зачем: JSON нужен как главный машинно-читаемый отчет.

Какую проблему решает: Он решает проблему автоматической проверки, интеграции и защиты: можно показать schema, fields, evidence.

Как работает: JSON writer строит объект с metadata, PE summary, unwind entries, heuristic report и optional dynamic evidence.

Что писать руками: Следи за запятыми между элементами массива и экранируй строки.

Как проверить: Проверь отчет через JSON parser, а не глазами.

Шаг 6. Добавь model validation: report writer должен отказаться от невозможных указателей/границ.

Зачем: HTML нужен как человекочитаемая страница отчета.

Какую проблему решает: Он решает проблему просмотра без специальных инструментов: открыл файл и видишь verdict, sections, findings.

Как работает: HTML writer должен экранировать все данные из файла и отчета, чтобы текст не стал HTML-кодом.

Что писать руками: Добавь `html_escape` и Content Security Policy, не вставляй внешние скрипты.

Как проверить: Проверь message с `<` и `&`: на странице должны быть символы, а не сломанная разметка.

Шаг 7. Собери JSON по schema, затем добавь HTML dashboard.

Зачем: Atomic write в report layer нужен, чтобы любой формат либо записался полностью, либо не считался готовым.

Какую проблему решает: Это решает проблему битых отчетов после ошибки диска или прерывания.

Как работает: Writer сначала собирает весь текст в `StrBuf`, проверяет `sb_ok`, потом вызывает `file_write_atomic`.

Что писать руками: Не пиши отчет прямо по частям в итоговый файл.

Как проверить: Проверь, что при ошибке writer возвращает `false`, а `main.c` дает exit code 1.

Шаг 8. В отчете отдели static evidence, dynamic evidence, final assessment и manual validation steps.

Зачем: Manual validation steps нужны, чтобы проект честно говорил: это основание для подозрения, а не автоматический приговор.

Какую проблему решает: Это решает проблему защиты: можно объяснить, что именно должен проверить реверс-инженер после finding.

Как работает: Report добавляет к findings список ручных действий: проверить секцию, handler target, unwind bytes, dynamic correlation.

Что писать руками: Сформируй эти steps рядом с heuristic report, чтобы JSON/HTML показывали одинаковую логику.

Как проверить: Проверь high suspicion case: в отчете есть не только score, но и что делать дальше.

Проверки перед переходом дальше

- Кавычки, переносы строк и запятые в CSV не ломают файл.
- Невалидный UTF-8 не проходит в JSON как сырые байты.
- HTML не грузит внешние скрипты, стили, шрифты или картинки.
- Временные `.tmp.<pid>` файлы не остаются после успешной записи.

Как объяснять себе каждую строку

Для каждой строки задавай четыре вопроса: какой объект она читает, какой объект меняет, какой failure path возможен, какой следующий слой будет использовать результат. Если строка только украшает код и не проходит эти вопросы, скорее всего ты пока ее не понял.

Любой output writer:

1. Проверить аргументы.
2. Собрать текст в StrBuf.
3. Экранировать специальные символы формата.
4. Проверить sb_ok().
5. Записать через file_write_atomic().

dynamic.h / dynamic_win.c

Windows-only сбор runtime evidence через Debug API: exceptions, VEH/VCH/UEF, unwind handler breakpoints, dynamic function tables.

Что выучить в C на этом модуле

- условная компиляция `#ifdef _WIN32`
- Windows HANDLE и cleanup
- CreateProcess под debugger
- debug events
- INT3 breakpoint и re-arm
- GetThreadContext / SetThreadContext
- модульные базы и offsets
- граница безопасности execution

Главные структуры и функции

Структуры/константы	Функции
DynamicEvent, DynamicReport, ModList, Breakpoint, PendingRearmList	dynamic_report_init, dynamic_collect_windows, mods_refresh_toolhelp, bp_add_and_arm_selfcheck, add_api_bps_if_ready, dynamic_event_add

Порядок написания

Ниже каждый пункт раскрыт как учебный шаг. Сначала смотри не на синтаксис, а на смысл: зачем этот объект появляется, какую опасность он убирает, как данные пойдут дальше и чем доказать, что шаг действительно работает.

Шаг 1. Сначала реализуй non-Windows stub: dynamic mode сообщает, что поддержан только Windows.

Зачем: Сначала нужен portable header, чтобы остальной проект мог знать о dynamic mode без подключения Windows API.

Какую проблему решает: Это решает проблему зависимости: static analyzer должен собираться и без Windows debugger.

Как работает: `dynamic.h` описывает options/report/events, но не тянет тяжелую platform-specific реализацию в другие модули.

Что писать руками: Опиши `DynamicOptions`, `DynamicEvent`, `DynamicReport` и функции init/free/collect.

Как проверить: Проверь, что `main.c` компилируется с include `dynamic.h` на non-Windows.

Шаг 2. Опиши `DynamicReport`: счетчики, events, trace, details.

Зачем: Non-Windows stub нужен, потому что dynamic mode Windows-only, но проект целиком не должен из-за этого ломаться.

Какую проблему решает: Он решает проблему переносимости static mode.

Как работает: Stub возвращает `false` и понятное сообщение, если пользователь запросил dynamic mode на неподдерживаемой платформе.

Что писать руками: Добавь `#ifndef _WIN32` ветку с честной ошибкой.

Как проверить: Проверь сборку без Windows headers.

Шаг 3. На Windows собери command line с правильным quoting.

Зачем: Запуск debuggee нужен, чтобы наблюдать runtime exception behavior, а не только bytes на диске.

Какую проблему решает: Он решает проблему статической неопределенности: некоторые признаки подтверждаются только во время выполнения.

Как работает: Windows Debug API запускает процесс под отладчиком, после чего analyzer получает debug events.

Что писать руками: Используй `CreateProcess` с debug flags и сохрани handles для cleanup.

Как проверить: Проверь, что несуществующий target дает понятную ошибку.

Шаг 4. Запусти target через `CreateProcess` с debug flags.

Зачем: Debug loop нужен как центральный цикл сбора dynamic evidence.

Какую проблему решает: Он решает проблему асинхронных событий: exceptions, load dll, breakpoints приходят не линейно.

Как работает: Цикл ждет событие, определяет тип, обновляет report, продолжает выполнение debuggee.

Что писать руками: Напиши loop вокруг `WaitForDebugEvent` и `ContinueDebugEvent`.

Как проверить: Проверь timeout: цикл должен завершиться, а не ждать бесконечно.

Шаг 5. Обновляй список модулей через Toolhelp, чтобы переводить адреса в module+offset.

Зачем: Breakpoints на API/handlers нужны, чтобы поймать регистрации и срабатывания exception mechanisms.

Какую проблему решает: Они решают проблему невидимых runtime действий: статически вызов может быть косвенным или спрятанным.

Как работает: Analyzer заменяет первый byte инструкции на `0xCC`, сохраняет оригинальный byte и адрес.

Что писать руками: Сделай таблицу breakpoints: address, original byte, enabled, event kind.

Как проверить: Проверь, что breakpoint не ставится дважды на один адрес.

Шаг 6. Поставь API breakpoints на VEH/VCH/UEF и RtlAddFunctionTable/RtlInstallFunctionTableCallback.

Зачем: INT3 handling нужен, потому что breakpoint меняет поток исполнения.

Какую проблему решает: Он решает проблему корректного продолжения: после `0xCC` надо вернуть original byte и откатить RIP.

Как работает: При breakpoint event analyzer записывает evidence, восстанавливает byte, ставит RIP на адрес инструкции и включает trap flag.

Что писать руками: Добавь функцию обработки breakpoint hit отдельно от общего debug loop.

Как проверить: Проверь, что после hit процесс продолжает выполнение, а не застревает на том же INT3.

Шаг 7. Поставь breakpoints на handlers, найденные static stage.

Зачем: Single-step handling нужен, чтобы снова поставить breakpoint после выполнения оригинальной инструкции.

Какую проблему решает: Он решает проблему одноразового breakpoint: если его не вернуть, следующий вызов API не будет замечен.

Как работает: После single-step analyzer снова записывает `0xCC` на адрес и снимает режим single-step.

Что писать руками: Храни pending breakpoint, который надо переустановить.

Как проверить: Проверь два вызова одной API: должно быть два события.

Шаг 8. В debug loop различай CREATE_PROCESS, LOAD_DLL, EXCEPTION, EXIT_PROCESS.

Зачем: VEH/VCH/UEF события нужны, потому что проект исследует exception handling, а не просто факт запуска процесса.

Какую проблему решает: Они решают проблему связи runtime поведения с темой EH/unwind.

Как работает: Dynamic collector отмечает регистрации обработчиков и фактические exception hits.

Что писать руками: Добавь event kinds и counters для VEH, VCH, UEF.

Как проверить: Проверь тестовый sample с VEH: отчет должен содержать registration или hit.

Шаг 9. При INT3 сними breakpoint, откати RIP, включи trap flag, затем re-arm на single-step.

Зачем: Runtime function table events нужны, чтобы увидеть `RtlAddFunctionTable` и похожие механизмы.

Какую проблему решает: Они решают проблему динамически создаваемой unwind metadata, которой может не быть в статической `.pdata`.

Как работает: Breakpoint на API фиксирует адрес таблицы, размер и caller context насколько это доступно.

Что писать руками: Добавь отдельный event kind для RtlAddFunctionTable/RtlInstallFunctionTableCallback.

Как проверить: Проверь sample с runtime table: dynamic signal count должен увеличиться.

Шаг 10. Всегда чисти breakpoints и handles, даже если timeout или ошибка.

Зачем: Cleanup нужен, потому что dynamic mode владеет handles, breakpoints и потенциально живым процессом.

Какую проблему решает: Он решает проблему оставленных процессов и поврежденного debuggee state.

Как работает: На любом выходе нужно восстановить breakpoints, закрыть handles, остановить или отсоединить debuggee по правилам режима.

Что писать руками: Сделай единый cleanup path, как в `main.c`, и вызывай его при ошибке и timeout.

Как проверить: Проверь ошибку в середине: handles закрыты, процесс не остался висеть.

Проверки перед переходом дальше

- На Linux/macOS сборка статического режима проходит без Windows headers.
- Timeout завершает dynamic session и не оставляет зависший процесс.
- Instrumentation completeness показывает, где сбор evidence был неполным.
- Dynamic mode не заменяет sandbox: файл реально выполняется.

Как объяснять себе каждую строку

Для каждой строки задавай четыре вопроса: какой объект она читает, какой объект меняет, какой failure path возможен, какой следующий слой будет использовать результат. Если строка только украшает код и не проходит эти вопросы, скорее всего ты пока ее не понял.

```
Dynamic mode:
  create process as debuggee
  receive debug events
  set API/handler breakpoints
  on INT3:
      restore original byte
      rewind RIP
      set trap flag
      record event
  on single-step:
      re-arm breakpoint
```

app_info.h / app_info.c

Единая точка версий, build metadata и UTC timestamp для отчетов.

Что выучить в C на этом модуле

- макросы версии
- условная компиляция по compiler/platform
- время UTC
- маленький модуль как стабильный источник правды

Главные структуры и функции

Структуры/константы	Функции
EHUA_VERSION, EHUA_REPORT_SCHEMA_VERSION	app_version, app_report_schema_version, app_build_compiler, app_build_platform, app_build_mode, app_utc_timestamp

Порядок написания

Ниже каждый пункт раскрыт как учебный шаг. Сначала смотри не на синтаксис, а на смысл: зачем этот объект появляется, какую опасность он убирает, как данные пойдут дальше и чем доказать, что шаг действительно работает.

Шаг 1. Вынеси версии в header, чтобы main/report/tests видели одно значение.

Зачем: Единые строковые константы нужны, чтобы версия приложения не была размазана по разным файлам.

Какую проблему решает: Это решает проблему расхождения: CLI может печатать одно, JSON другое, документация третье.

Как работает: `app_info.c` становится единственным местом, откуда берутся app name/version/schema version.

Что писать руками: В `app_info.h` объяви функции `app_name`, `app_version`, `report_schema_version`.

Как проверить: Проверь, что `--version` и JSON metadata используют одну и ту же функцию.

Шаг 2. Определи compiler через `_MSC_VER`, `__clang__`, `__GNUC__`.

Зачем: `app_info.c` должен быть маленьким и независимым.

Какую проблему решает: Это решает проблему лишних зависимостей: версия не должна зависеть от PE parser или report writer.

Как работает: Функции возвращают строковые литералы со статическим временем жизни.

Что писать руками: Реализуй функции как `return "1.0.0";`, без выделения памяти.

Как проверить: Проверь, что вызывающий код не пытается `free` эти строки.

Шаг 3. Определи platform через `_WIN64`, `__linux__`, `__APPLE__`.

Зачем: Build metadata полезна для воспроизводимости.

Какую проблему решает: Она решает проблему вопроса 'каким компилятором и какой сборкой это получено'.

Как работает: Если доступны макросы сборки, функция возвращает строку compiler/build mode; если нет - stable fallback.

Что писать руками: Добавь compile-time macros аккуратно, чтобы отсутствие макроса не ломало сборку.

Как проверить: Проверь debug и release: output остается валидным.

Шаг 4. Сделай `app_build_mode`: `NDEBUG` означает release.

Зачем: Schema version нужна отдельно от app version.

Какую проблему решает: Она решает проблему совместимости отчетов: приложение может обновиться без изменения JSON schema, и наоборот.

Как работает: Report writer записывает schema version, а tests валидируют ожидаемый контракт.

Что писать руками: Не используй app version как schema version.

Как проверить: Проверь тест release metadata: schema не пустая и совпадает с ожидаемой.

Шаг 5. Сделай timestamp строго RFC 3339: `YYYY-MM-DDTHH:MM:SSZ`.

Зачем: Интеграция с `main.c` и `report.c` завершает модуль.

Какую проблему решает: Она решает проблему мертвого кода: если версии объявлены, но никто их не использует, контракт не работает.

Как работает: `main.c` печатает version, report metadata берет schema/name из `app_info`.

Что писать руками: Замени строковые литералы в других местах вызовами `app_info`.

Как проверить: Проверь поиском по проекту, что версия не продублирована в нескольких `.c` без причины.

Проверки перед переходом дальше

- `--version` печатает tool version и report schema.
- JSON metadata содержит ту же schema version, что и `app_info.h`.

Как объяснять себе каждую строку

Для каждой строки задавай четыре вопроса: какой объект она читает, какой объект меняет, какой failure path возможен, какой следующий слой будет использовать результат. Если строка только украшает код и не проходит эти вопросы, скорее всего ты пока ее не понял.

main.c

CLI, жизненный цикл программы, оркестрация модулей, владение памятью и exit codes.

Что выучить в C на этом модуле

- структура Args
- разбор аргументов
- коды выхода
- последовательность pipeline
- ранний выход и cleanup
- передача указателей между модулями
- redaction target arguments

Главные структуры и функции

Структуры/константы	Функции
Args, ParseResult	parse_args, build_invocation, print_one, main

Порядок написания

Ниже каждый пункт раскрыт как учебный шаг. Сначала смотри не на синтаксис, а на смысл: зачем этот объект появляется, какую опасность он убирает, как данные пойдут дальше и чем доказать, что шаг действительно работает.

Шаг 1. Начни с `usage` и `--version`: программу можно проверить еще до парсера PE.

Зачем: Структура CLI options нужна, чтобы не таскать десятки локальных переменных по `main`.

Какую проблему решает: Она решает проблему хаоса аргументов: input path, output paths, flags и режимы должны жить в одном объекте.

Как работает: `parse_args` заполняет `CliOptions`, дальше pipeline читает уже готовые поля.

Что писать руками: Создай `typedef struct CliOptions` внутри `main.c`, если она не нужна другим файлам.

Как проверить: Проверь `--help`: он не требует input file.

Шаг 2. Опиши `Args` с дефолтами: follow_chain true, max depth 8, timeout 15000, trace 64.

Зачем: Разбор `--help` и `--version` нужен до анализа файла.

Какую проблему решает: Он решает проблему лишних требований: пользователь должен увидеть справку даже без PE.

Как работает: Если найден help/version, программа печатает текст и выходит с code 0.

Что писать руками: Сделай эти флаги ранними ветками в `parse_args` или сразу после него.

Как проверить: Проверь запуск без аргументов и с `--help`: exit code должен быть предсказуемым.

Шаг 3. Напиши `parse_args`: каждую опцию проверяй отдельно, диапазоны чисел валидируй.

Зачем: Валидация аргументов нужна, чтобы отличать ошибку командной строки от ошибки анализа.

Какую проблему решает: Она решает проблему exit codes: неверная команда - это code 2, а не failed analysis.

Как работает: Parser проверяет обязательный input, конфликтующие опции и корректность путей output.

Что писать руками: После parse_args верни отдельный статус: ok/help/error.

Как проверить: Проверь неизвестную опцию: программа возвращает 2.

Шаг 4. В `main` сначала прочитай файл, затем `pe_parse`, затем `unwind_read_runtime_functions_ex`.

Зачем: Загрузка файла через `Buf` запускает pipeline.

Какую проблему решает: Она решает проблему перехода от CLI path к универсальным bytes.

Как работает: `main.c` вызывает `buf_init_from_file`, хранит `owned_mem`, и передает `Buf` дальше как read-only input.

Что писать руками: Не открывай файл повторно в `pe.c` или `unwind.c`.

Как проверить: Проверь несуществующий файл: ошибка понятная, память не освобождается дважды.

Шаг 5. Выдели массивы `UnwindSummary` и `bool us_ok` размером `rf_count ? rf_count : 1`.

Зачем: PE parse должен идти раньше unwind parse.

Какую проблему решает: Он решает проблему карты адресов: `.pdata` и `UNWIND_INFO` задаются RVA, а читать надо file offsets.

Как работает: `pe_parse` создает `PeImage`, который дальше используют все слои.

Что писать руками: После `pe_parse` при ошибке переходи в cleanup.

Как проверить: Проверь non-PE input: до unwind parser дело не доходит.

Шаг 6. Для каждой `RuntimeFunction` вызови `unwind_parse` и сохрани результат.

Зачем: Чтение runtime functions - первый unwind этап.

Какую проблему решает: Оно решает проблему списка функций: дальше нужно знать, сколько записей парсить и какие unwind RVA посещать.

Как работает: `unwind_read_runtime_functions_ex` возвращает массив и count, которыми владеет `main.c`.

Что писать руками: Храни pointer и count рядом; освобождай pointer в cleanup.

Как проверить: Проверь PE без `.pdata`: pipeline завершает анализ с пустым списком, а не падает.

Шаг 7. Если попросили `--dump`, выведи краткую строку по каждой функции.

Зачем: Парсинг каждого `UNWIND_INFO` нужен до heuristics.

Какую проблему решает: Он решает проблему evidence: heuristics должны видеть summaries и anomalies, а не сырые RVA.

Как работает: `main.c` выделяет массив summaries по `rf_count` и заполняет его в цикле.

Что писать руками: Если одна запись не парсится, сохрани parse error в summary и продолжай, если политика проекта это допускает.

Как проверить: Проверь один плохой unwind record: отчет должен показать ошибку этой записи.

Шаг 8. Если попросили `--heuristics`, вызови `heuristics_analyze` и запомни `hr_ptr`.

Зачем: Heuristics идут после всех static данных.

Какую проблему решает: Они решают проблему финальной интерпретации: только теперь есть PE context, runtime functions и unwind summaries.

Как работает: `heuristics_analyze` создает report, который потом передается writers.

Что писать руками: Не считай score в `main.c`; вызови отдельный модуль.

Как проверить: Проверь, что один и тот же input дает одинаковый score в console и JSON.

Шаг 9. Если попросили `--dynamic`, вызови collector и запомни `dr_ptr`.

Зачем: Dynamic mode должен быть optional.

Какую проблему решает: Он решает проблему безопасности и переносимости: static analysis не обязан запускать чужой код.

Как работает: `main.c` вызывает dynamic collector только при явном флаге и добавляет результат в общий отчет.

Что писать руками: Если dynamic mode failed, не уничтожай static result; запиши причину.

Как проверить: Проверь запуск без dynamic flag: Windows API не требуется.

Шаг 10. Отчеты пиши после анализа, когда есть metadata, SHA-256 и command line.

Зачем: Запись reports должна быть последним этапом после анализа.

Какую проблему решает: Она решает проблему целостности output: writers должны видеть полностью готовые данные.

Как работает: `main.c` смотрит CLI options и вызывает нужные writer functions.

Что писать руками: Если writer вернул `false`, выставь exit code 1.

Как проверить: Проверь `--json out.json`: файл появился и парсится.

Шаг 11. В конце освободи все: reports, buffers, arrays, PE sections, input bytes.

Зачем: Единый cleanup path нужен, потому что в C нет автоматического освобождения heap-памяти.

Какую проблему решает: Он решает проблему утечек и двойных освобождений при ранних ошибках.

Как работает: Все ресурсы инициализируются нулями, каждый failure path идет в один блок cleanup, где вызываются free-функции.

Что писать руками: Используй `goto cleanup` осознанно: это не грязный трюк, а нормальный C-паттерн для функции с несколькими ресурсами.

Как проверить: Проверь ошибку на каждом этапе: программа выходит без падения и освобождает уже созданные объекты.

Проверки перед переходом дальше

- Без аргументов exit code 2 и usage.
- Отсутствующий файл дает exit code 1.
- Неверный `--max-chain-depth 17` дает exit code 2.
- После `--` аргументы target не попадают в JSON metadata.

Как объяснять себе каждую строку

Для каждой строки задавай четыре вопроса: какой объект она читает, какой объект меняет, какой failure path возможен, какой следующий слой будет использовать результат. Если строка только украшает код и не проходит эти вопросы, скорее всего ты пока ее не понял.

10. Большой маршрут переписывания руками

Это не календарь по датам, а учебная последовательность. Один 'день' может занять вечер или неделю. Главное - завершать день работающим маленьким состоянием проекта.

Шаг	Цель	Что писать	Что учить
1	Сборка-скелет	CMake, `main.c`, `--help`, `--version` stub	единица трансляции, linker
2	Первые типы	`util.h`: `Buf`, `StrBuf`, прототипы	header guards / `#pragma once`, typedef struct
3	Чтение bytes	`buf_read`, `buf_read_u16/u32/u64`	указатели, границы, endian
4	Чтение файла	`buf_init_from_file`, `buf_free_owned`	FILE*, malloc, ownership
5	Растущая строка	`sb_init`, `sb_reserve`, `sb_appendn`	capacity, realloc
6	Format append	`sb_appendf`	variadic functions, vsnprintf
7	Atomic write	`file_write_atomic`	temporary files, cleanup
8	SHA-256	`sha256_hex` или временный stub с тестом	алгоритм и state
9	PE структуры	`pe.h`: constants, `PeSection`, `PeImage`	domain structs
10	DOS/PE header	`read_dos_lfanew`, signature checks	magic numbers, offsets
11	COFF/optional	Machine, PE32+ magic, ImageBase, alignments	binary layout
12	Data directory	exception RVA/size	таблица directories
13	Section headers	чтение секций	цикл, массив структур
14	PE validation	oob, overlaps, misalignment	инварианты
15	RVA mapping	`pe_find_section_for_rva`, `pe_rva_to_fileoff`	virtual vs raw
16	Runtime functions	`unwind_read_runtime_functions_ex`	массив 12-байтовых записей
17	Unwind header	version, flags, prolog, count, frame	bit fields
18	Slots table	`slots_for_code`	switch, variable record length
19	Unwind loop	parse slots, stack effects	loop with manual increment
20	Anomalies	bad opcode, truncated, handler oob, alignment	bit masks
21	Handler tail	EHANDLER/UHANDLER tail	условный layout
22	Chain tail	CHAININFO and follow-chain	bounded recursion
23	Heuristic counters	rf ranges, parse fail, placement	aggregation
24	Hash map reuse	`HMap`, max counts	open addressing
25	Findings	`add_finding`, representative RVA	structured evidence
26	Score/confidence	weights, mitigations, verdict	policy code
27	CSV reports	normal CSV and Ghidra CSV	escaping
28	JSON report	metadata, sections, functions, heuristics	serialization contract
29	HTML report	offline dashboard	HTML escaping, CSP
30	Dynamic stub	non-Windows path	conditional compilation
31	Dynamic Windows	debug loop and breakpoints	Windows API
32	Main orchestration	full pipeline and cleanup	resource lifecycle

Шаг	Цель	Что писать	Что учить
33	Unit tests	`unit_unwind`, `unit_core_edges`	fake input bytes
34	CLI contract	`test_cli_contract.py`	black-box tests
35	Regression/fuzz	sample corpus and mutation fuzz	robustness
36	Release hygiene	version, schema, packaging	product discipline

Контрольные ворота

- После `util`: можешь безопасно прочитать arbitrary bytes и собрать строку отчета.
- После `pe`: можешь открыть PE32+ AMD64 и объяснить каждую секцию.
- После `unwind`: можешь взять одну `.pdata` entry и найти ее `UNWIND_INFO`.
- После `heuristics`: можешь объяснить любой finding без слов 'магия' и 'мне кажется'.
- После `report`: можешь сохранить результат и проверить JSON schema/CSV/HTML.
- После `main`: можешь провести полный анализ sample и освободить все ресурсы.

11. Практикум: как руками пройти каждый модуль

Этот раздел нужен не для чтения, а для работы рядом с редактором. Открываешь модуль, выполняешь лабораторную, пишешь короткий ответ в тетрадь или комментарий рядом с кодом, собираешь проект. Если лабораторная не получается, значит еще рано переписывать следующий файл.

Практикум: util.h / util.c

Цель практикума: Фундамент: чтение файла в память, безопасное чтение байтов, little-endian числа, растущая строка, атомарная запись файла, SHA-256.

Лабораторная A: словарь модуля

1. Выпиши названия всех структур и функций модуля.
2. Напротив каждой структуры напиши, кто ее создает и кто освобождает.
3. Напротив каждой функции напиши одно предложение: какой input она принимает и какой output производит.
4. Отдельно отметь функции, которые могут вернуть `false` или код ошибки.
5. Собери проект после того, как добавишь только объявления в `.h`.

Лабораторная B: минимальный работающий кусок

Сделай самую маленькую версию модуля, которая компилируется и выполняет один честный сценарий. Не пытайся сразу покрыть все edge cases. Сначала нужен живой скелет, потом защита границ.

1. Выбери первую функцию из списка: `buf_init_from_file`.
2. Напиши happy path без сложных веток.
3. Добавь guard checks на `NULL` и неверные размеры.
4. Добавь один негативный тест.
5. Проверь, что ошибка не портит уже созданные объекты.

Лабораторная C: плохой input

Большая часть ценности проекта - в спокойном отношении к сломанному input. Поэтому для каждого модуля обязательно придумай три плохих входа: пустой, обрезанный, противоречивый. Программа должна вернуть ошибку или finding, но не падать.

- Пустой input: размер 0, нулевой count, отсутствующая секция или отсутствие events.
- Обрезанный input: header есть, но tail не помещается.
- Противоречивый input: begin больше end, RVA вне image, флаги несовместимы.
- Проверка: после плохого input вызывается cleanup и процесс завершается предсказуемым кодом.

Вопросы для устного объяснения

- Почему этот модуль не должен делать работу соседнего модуля?

- Какая структура здесь является главным результатом?

- Какая строка в модуле самая опасная с точки зрения выхода за границы?

- Какая ошибка новичка здесь наиболее вероятна?

- Какой тест докажет, что модуль можно передать следующему слою?

Микро-итерации

Итерация 1. Создай `Buf`: два поля, `const uint8_t* data` и `size_t size`. Пока не пиши PE. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 2. Напиши `buf_read`: сначала проверки аргументов, потом проверка `off > size`, потом `len > size - off`, затем `memcpy`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 3. Напиши `le16`, `le32`, `le64`. Научись видеть, что байты файла не обязаны совпадать с порядком байтов CPU. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 4. Добавь `buf_read_u16/u32/u64`: читаешь временный массив байтов и переводишь в число. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 5. Напиши чтение файла: `fopen`, `fseek`, `ftell`, выделение памяти, `fread`, закрытие файла. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 6. Сделай `StrBuf`: `s`, `len`, `cap`, `failed`. Не добавляй строку без места под завершающий ноль. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 7. Реализуй `sb_reserve` через `realloc`. Проверь, что рост емкости не переполняет `SIZE_MAX`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 8. Реализуй `sb_appendn`, потом `sb_append`, потом `sb_appendf`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 9. Сделай `file_write_atomic`: сначала `.tmp.<pid>`, потом замена итогового файла. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 10. SHA-256 можно написать последним: это самостоятельный алгоритм, он нужен для идентичности входного файла в отчетах. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Практикум: pe.h / pe.c

Цель практикума: Строит минимальную модель PE32+ AMD64: заголовки, секции, exception directory, проверки диапазонов и перевод RVA в file offset.

Лабораторная A: словарь модуля

1. Выпиши названия всех структур и функций модуля.
2. Напротив каждой структуры напиши, кто ее создает и кто освобождает.
3. Напротив каждой функции напиши одно предложение: какой input она принимает и какой output производит.
4. Отдельно отметь функции, которые могут вернуть `false` или код ошибки.
5. Собери проект после того, как добавишь только объявления в `.h`.

Лабораторная B: минимальный работающий кусок

Сделай самую маленькую версию модуля, которая компилируется и выполняет один честный сценарий. Не пытайся сразу покрыть все edge cases. Сначала нужен живой скелет, потом защита границ.

1. Выбери первую функцию из списка: `pe_parse`.
2. Напиши happy path без сложных веток.
3. Добавь guard checks на `NULL` и неверные размеры.
4. Добавь один негативный тест.
5. Проверь, что ошибка не портит уже созданные объекты.

Лабораторная C: плохой input

Большая часть ценности проекта - в спокойном отношении к сломанному input. Поэтому для каждого модуля обязательно придумай три плохих входа: пустой, обрезанный, противоречивый. Программа должна вернуть ошибку или finding, но не падать.

- Пустой input: размер 0, нулевой count, отсутствующая секция или отсутствие events.
- Обрезанный input: header есть, но tail не помещается.
- Противоречивый input: begin больше end, RVA вне image, флаги несовместимы.
- Проверка: после плохого input вызывается cleanup и процесс завершается предсказуемым кодом.

Вопросы для устного объяснения

- Почему этот модуль не должен делать работу соседнего модуля?
- Какая структура здесь является главным результатом?
- Какая строка в модуле самая опасная с точки зрения выхода за границы?
- Какая ошибка новичка здесь наиболее вероятна?
- Какой тест докажет, что модуль можно передать следующему слою?

Микро-итерации

Итерация 1. Начни с констант `MZ`, `PE\0\0`, `AMD64`, `PE32+`, индекса exception directory. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 2. Опиши `PeSection`: имя, virtual address, virtual size, raw pointer, raw size, characteristics. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 3. Опиши `PeImage`: глобальные поля образа, массив секций, счетчики проблем. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 4. В `pe_parse` первым делом зануляй `out`, чтобы при ошибке не осталось мусора. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 5. Прочитай `e_lfanew` из DOS header: это смещение до NT headers. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 6. Проверь сигнатуру PE и COFF Machine. Проект сознательно принимает только AMD64. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 7. Прочитай optional header PE32+: image base, alignment, image/header size, число data directories. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 8. Найди exception directory по индексу 3. Сохрани RVA и размер. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 9. Прочитай секции циклом по 40 байт на заголовок. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 10. Посчитай структурные проблемы: выход raw за файл, virtual за SizeOfImage, overlaps, alignment. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 11. Реализуй `pe_find_section_for_rva` и `pe_rva_to_fileoff`: это карта, без которой unwind parser слепой. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Практикум: unwind.h / unwind.c

Цель практикума: Читает `.pdata` как массив `RUNTIME_FUNCTION` и декодирует `UNWIND_INFO` V1/V2, операции пролога, handler и chain.

Лабораторная A: словарь модуля

1. Выпиши названия всех структур и функций модуля.
2. Напротив каждой структуры напиши, кто ее создает и кто освобождает.
3. Напротив каждой функции напиши одно предложение: какой input она принимает и какой output производит.
4. Отдельно отметь функции, которые могут вернуть `false` или код ошибки.
5. Собери проект после того, как добавишь только объявления в `.h`.

Лабораторная B: минимальный работающий кусок

Сделай самую маленькую версию модуля, которая компилируется и выполняет один честный сценарий. Не пытайся сразу покрыть все edge cases. Сначала нужен живой скелет, потом защита границ.

1. Выбери первую функцию из списка: `unwind_read_runtime_functions_ex`.
2. Напиши happy path без сложных веток.
3. Добавь guard checks на `NULL` и неверные размеры.
4. Добавь один негативный тест.
5. Проверь, что ошибка не портит уже созданные объекты.

Лабораторная C: плохой input

Большая часть ценности проекта - в спокойном отношении к сломанному input. Поэтому для каждого модуля обязательно придумай три плохих входа: пустой, обрезанный, противоречивый. Программа должна вернуть ошибку или finding, но не падать.

- Пустой input: размер 0, нулевой count, отсутствующая секция или отсутствие events.
- Обрезанный input: header есть, но tail не помещается.
- Противоречивый input: begin больше end, RVA вне image, флаги несовместимы.
- Проверка: после плохого input вызывается cleanup и процесс завершается предсказуемым кодом.

Вопросы для устного объяснения

- Почему этот модуль не должен делать работу соседнего модуля?
- Какая структура здесь является главным результатом?
- Какая строка в модуле самая опасная с точки зрения выхода за границы?
- Какая ошибка новичка здесь наиболее вероятна?
- Какой тест докажет, что модуль можно передать следующему слою?

Микро-итерации

Итерация 1. Сначала реализуй только чтение `RUNTIME_FUNCTION`: каждая запись 12 байт, три `uint32_t` RVA. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 2. В `UnwindSummary` добавь только базовые поля: version, flags, prolog, count, frame. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 3. Реализуй чтение header `UNWIND_INFO`: первый байт делится на version и flags. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 4. Сделай `slots_for_code`: для каждого unwind opcode верни, сколько 2-байтовых slots он занимает. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 5. Напиши цикл `for (i = 0; i < count;)`: читаешь slot, выделяешь CodeOffset, Op, OpInfo, увеличиваешь `i += slots`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 6. Для каждого opcode сначала добавь распознавание и текст, потом вычисление stack/register effects. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 7. Добавь anomaly flags: unsupported version, truncated, bad opcode, bad operand, code offset, alignment. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 8. Добавь handler tail: если флаги EHANDLER/UHANDLER, после slots лежит handler RVA. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 9. Добавь chain tail: если CHAININFO, после slots лежит еще один `RUNTIME_FUNCTION`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 10. Включи follow-chain только после того, как одиночный record работает. Ограничение depth защищает от циклов. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Практикум: heuristics.h / heuristics.c

Цель практикума: Превращает наблюдения в findings, score, confidence, evidence level, verdict и weakening rules.

Лабораторная A: словарь модуля

1. Выпиши названия всех структур и функций модуля.
2. Напротив каждой структуры напиши, кто ее создает и кто освобождает.
3. Напротив каждой функции напиши одно предложение: какой input она принимает и какой output производит.
4. Отдельно отметь функции, которые могут вернуть `false` или код ошибки.
5. Собери проект после того, как добавишь только объявления в `.h`.

Лабораторная B: минимальный работающий кусок

Сделай самую маленькую версию модуля, которая компилируется и выполняет один честный сценарий. Не пытайся сразу покрыть все edge cases. Сначала нужен живой скелет, потом защита границ.

1. Выбери первую функцию из списка: `heuristics_analyze`.
2. Напиши happy path без сложных веток.
3. Добавь guard checks на `NULL` и неверные размеры.
4. Добавь один негативный тест.
5. Проверь, что ошибка не портит уже созданные объекты.

Лабораторная C: плохой input

Большая часть ценности проекта - в спокойном отношении к сломанному input. Поэтому для каждого модуля обязательно придумай три плохих входа: пустой, обрезанный, противоречивый. Программа должна вернуть ошибку или finding, но не падать.

- Пустой input: размер 0, нулевой count, отсутствующая секция или отсутствие events.
- Обрезанный input: header есть, но tail не помещается.
- Противоречивый input: begin больше end, RVA вне image, флаги несовместимы.
- Проверка: после плохого input вызывается cleanup и процесс завершается предсказуемым кодом.

Вопросы для устного объяснения

- Почему этот модуль не должен делать работу соседнего модуля?
- Какая структура здесь является главным результатом?
- Какая строка в модуле самая опасная с точки зрения выхода за границы?
- Какая ошибка новичка здесь наиболее вероятна?
- Какой тест докажет, что модуль можно передать следующему слою?

Микро-итерации

Итерация 1. Сначала перенеси счетчики из `PeImage`: section problems и exception directory problems. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 2. Пройди все `RuntimeFunction`: invalid ranges, out-of-bounds begin/end, out-of-bounds unwind RVA, unsorted. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 3. Параллельно считай reuse `UnwindInfoRVA` через маленькую hash map. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 4. Если unwind parse failed или `anomalies` содержит не только ABI warnings, увеличивай счетчики integrity. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 5. Проверь placement: где лежит unwind info и где лежит handler target. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 6. Отсортируй копию begin/end, чтобы найти overlaps и duplicate begins. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 7. Добавь findings: code, family, severity, message, representative RVA, affected count. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 8. Собери score по матрице весов. Важно: score это triage, не доказательство. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 9. Посчитай confidence: базовое значение, бонусы за семейства/сильные сигналы, затем weakening rules. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 10. Сформируй `verdict`, `evidence_level` и `requires_manual_review`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Практикум: report.h / report.c

Цель практикума: Сериализует модель в console, CSV, Ghidra CSV, JSON и HTML. Отчеты являются внешним контрактом проекта.

Лабораторная A: словарь модуля

1. Выпиши названия всех структур и функций модуля.
2. Напротив каждой структуры напиши, кто ее создает и кто освобождает.
3. Напротив каждой функции напиши одно предложение: какой input она принимает и какой output производит.
4. Отдельно отметь функции, которые могут вернуть `false` или код ошибки.
5. Собери проект после того, как добавишь только объявления в `.h`.

Лабораторная B: минимальный работающий кусок

Сделай самую маленькую версию модуля, которая компилируется и выполняет один честный сценарий. Не пытайся сразу покрыть все edge cases. Сначала нужен живой скелет, потом защита границ.

1. Выбери первую функцию из списка: `report_print_brief`.
2. Напиши happy path без сложных веток.
3. Добавь guard checks на `NULL` и неверные размеры.
4. Добавь один негативный тест.
5. Проверь, что ошибка не портит уже созданные объекты.

Лабораторная C: плохой input

Большая часть ценности проекта - в спокойном отношении к сломанному input. Поэтому для каждого модуля обязательно придумай три плохих входа: пустой, обрезанный, противоречивый. Программа должна вернуть ошибку или finding, но не падать.

- Пустой input: размер 0, нулевой count, отсутствующая секция или отсутствие events.
- Обрезанный input: header есть, но tail не помещается.
- Противоречивый input: begin больше end, RVA вне image, флаги несовместимы.
- Проверка: после плохого input вызывается cleanup и процесс завершается предсказуемым кодом.

Вопросы для устного объяснения

- Почему этот модуль не должен делать работу соседнего модуля?
- Какая структура здесь является главным результатом?
- Какая строка в модуле самая опасная с точки зрения выхода за границы?
- Какая ошибка новичка здесь наиболее вероятна?
- Какой тест докажет, что модуль можно передать следующему слою?

Микро-итерации

Итерация 1. Начни с console: вывести path, Machine, ImageBase, SizeOfImage, Exception Directory, count. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 2. Добавь CSV без Ghidra mode: одна строка на function, явные колонки. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 3. Добавь Ghidra CSV: begin, end, generated name, comment. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 4. Сделай escape функций для CSV, JSON и HTML. Не доверяй строкам из входа. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 5. Опиши `ReportMetadata`: версия, schema, timestamp, sha256, размер, command line, build. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 6. Добавь model validation: report writer должен отказаться от невозможных указателей/границ. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 7. Собери JSON по schema, затем добавь HTML dashboard. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 8. В отчете отдели static evidence, dynamic evidence, final assessment и manual validation steps. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Практикум: dynamic.h / dynamic_win.c

Цель практикума: Windows-only сбор runtime evidence через Debug API: exceptions, VEH/VCH/UEF, unwind handler breakpoints, dynamic function tables.

Лабораторная A: словарь модуля

1. Выпиши названия всех структур и функций модуля.
2. Напротив каждой структуры напиши, кто ее создает и кто освобождает.
3. Напротив каждой функции напиши одно предложение: какой input она принимает и какой output производит.
4. Отдельно отметь функции, которые могут вернуть `false` или код ошибки.
5. Собери проект после того, как добавишь только объявления в `.h`.

Лабораторная B: минимальный работающий кусок

Сделай самую маленькую версию модуля, которая компилируется и выполняет один честный сценарий. Не пытайся сразу покрыть все edge cases. Сначала нужен живой скелет, потом защита границ.

1. Выбери первую функцию из списка: `dynamic_report_init`.
2. Напиши happy path без сложных веток.
3. Добавь guard checks на `NULL` и неверные размеры.
4. Добавь один негативный тест.
5. Проверь, что ошибка не портит уже созданные объекты.

Лабораторная C: плохой input

Большая часть ценности проекта - в спокойном отношении к сломанному input. Поэтому для каждого модуля обязательно придумай три плохих входа: пустой, обрезанный, противоречивый. Программа должна вернуть ошибку или finding, но не падать.

- Пустой input: размер 0, нулевой count, отсутствующая секция или отсутствие events.
- Обрезанный input: header есть, но tail не помещается.
- Противоречивый input: begin больше end, RVA вне image, флаги несовместимы.
- Проверка: после плохого input вызывается cleanup и процесс завершается предсказуемым кодом.

Вопросы для устного объяснения

- Почему этот модуль не должен делать работу соседнего модуля?
- Какая структура здесь является главным результатом?
- Какая строка в модуле самая опасная с точки зрения выхода за границы?
- Какая ошибка новичка здесь наиболее вероятна?
- Какой тест докажет, что модуль можно передать следующему слою?

Микро-итерации

Итерация 1. Сначала реализуй non-Windows stub: dynamic mode сообщает, что поддержан только Windows. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 2. Опиши `DynamicReport`: счетчики, events, trace, details. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 3. На Windows собери command line с правильным quoting. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 4. Запусти target через `CreateProcess` с debug flags. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 5. Обновляй список модулей через Toolhelp, чтобы переводить адреса в module+offset. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 6. Поставь API breakpoints на VEH/VCH/UEF и RtlAddFunctionTable/RtlInstallFunctionTableCallback. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 7. Поставь breakpoints на handlers, найденные static stage. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 8. В debug loop различай CREATE_PROCESS, LOAD_DLL, EXCEPTION, EXIT_PROCESS. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 9. При INT3 сними breakpoint, откати RIP, включи trap flag, затем re-arm на single-step. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 10. Всегда чисти breakpoints и handles, даже если timeout или ошибка. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Практикум: app_info.h / app_info.c

Цель практикума: Единая точка версий, build metadata и UTC timestamp для отчетов.

Лабораторная A: словарь модуля

1. Выпиши названия всех структур и функций модуля.
2. Напротив каждой структуры напиши, кто ее создает и кто освобождает.
3. Напротив каждой функции напиши одно предложение: какой input она принимает и какой output производит.
4. Отдельно отметь функции, которые могут вернуть `false` или код ошибки.
5. Собери проект после того, как добавишь только объявления в `.h`.

Лабораторная B: минимальный работающий кусок

Сделай самую маленькую версию модуля, которая компилируется и выполняет один честный сценарий. Не пытайся сразу покрыть все edge cases. Сначала нужен живой скелет, потом защита границ.

1. Выбери первую функцию из списка: `app_version`.
2. Напиши happy path без сложных веток.
3. Добавь guard checks на `NULL` и неверные размеры.
4. Добавь один негативный тест.
5. Проверь, что ошибка не портит уже созданные объекты.

Лабораторная C: плохой input

Большая часть ценности проекта - в спокойном отношении к сломанному input. Поэтому для каждого модуля обязательно придумай три плохих входа: пустой, обрезанный, противоречивый. Программа должна вернуть ошибку или finding, но не падать.

- Пустой input: размер 0, нулевой count, отсутствующая секция или отсутствие events.
- Обрезанный input: header есть, но tail не помещается.
- Противоречивый input: begin больше end, RVA вне image, флаги несовместимы.
- Проверка: после плохого input вызывается cleanup и процесс завершается предсказуемым кодом.

Вопросы для устного объяснения

- Почему этот модуль не должен делать работу соседнего модуля?
- Какая структура здесь является главным результатом?
- Какая строка в модуле самая опасная с точки зрения выхода за границы?
- Какая ошибка новичка здесь наиболее вероятна?
- Какой тест докажет, что модуль можно передать следующему слою?

Микро-итерации

Итерация 1. Вынеси версии в header, чтобы main/report/tests видели одно значение. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 2. Определи compiler через `_MSC_VER`, `__clang__`, `__GNUC__`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 3. Определи platform через `_WIN64`, `__linux__`, `__APPLE__`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 4. Сделай `app_build_mode`: `NDEBUG` означает release. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 5. Сделай timestamp строго RFC 3339: `YYYY-MM-DDTHH:MM:SSZ`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Практикум: main.c

Цель практикума: CLI, жизненный цикл программы, оркестрация модулей, владение памятью и exit codes.

Лабораторная A: словарь модуля

1. Выпиши названия всех структур и функций модуля.
2. Напротив каждой структуры напиши, кто ее создает и кто освобождает.
3. Напротив каждой функции напиши одно предложение: какой input она принимает и какой output производит.
4. Отдельно отметь функции, которые могут вернуть `false` или код ошибки.
5. Собери проект после того, как добавишь только объявления в `.h`.

Лабораторная B: минимальный работающий кусок

Сделай самую маленькую версию модуля, которая компилируется и выполняет один честный сценарий. Не пытайся сразу покрыть все edge cases. Сначала нужен живой скелет, потом защита границ.

1. Выбери первую функцию из списка: `parse_args`.
2. Напиши happy path без сложных веток.
3. Добавь guard checks на `NULL` и неверные размеры.
4. Добавь один негативный тест.
5. Проверь, что ошибка не портит уже созданные объекты.

Лабораторная C: плохой input

Большая часть ценности проекта - в спокойном отношении к сломанному input. Поэтому для каждого модуля обязательно придумай три плохих входа: пустой, обрезанный, противоречивый. Программа должна вернуть ошибку или finding, но не падать.

- Пустой input: размер 0, нулевой count, отсутствующая секция или отсутствие events.
- Обрезанный input: header есть, но tail не помещается.
- Противоречивый input: begin больше end, RVA вне image, флаги несовместимы.
- Проверка: после плохого input вызывается cleanup и процесс завершается предсказуемым кодом.

Вопросы для устного объяснения

- Почему этот модуль не должен делать работу соседнего модуля?
- Какая структура здесь является главным результатом?
- Какая строка в модуле самая опасная с точки зрения выхода за границы?
- Какая ошибка новичка здесь наиболее вероятна?
- Какой тест докажет, что модуль можно передать следующему слою?

Микро-итерации

Итерация 1. Начни с `usage` и `--version`: программу можно проверить еще до парсера PE. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 2. Опиши `Args` с дефолтами: follow_chain true, max depth 8, timeout 15000, trace 64. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 3. Напиши `parse_args`: каждую опцию проверяй отдельно, диапазоны чисел валидируй. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 4. В `main` сначала прочитай файл, затем `pe_parse`, затем `unwind_read_runtime_functions_ex`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 5. Выдели массивы `UnwindSummary` и `bool us_ok` размером `rf_count ? rf_count : 1`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 6. Для каждой `RuntimeFunction` вызови `unwind_parse` и сохрани результат. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 7. Если попросили `--dump`, выведи краткую строку по каждой функции. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 8. Если попросили `--heuristics`, вызови `heuristics_analyze` и запомни `hr_ptr`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 9. Если попросили `--dynamic`, вызови collector и запомни `dr_ptr`. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 10. Отчеты пиши после анализа, когда есть metadata, SHA-256 и command line. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

Итерация 11. В конце освободи все: reports, buffers, arrays, PE sections, input bytes. После этой итерации собери проект. Если это header-only шаг, проверь, что все `.c` видят объявления. Если это реализация, добавь хотя бы один минимальный вызов из теста или временного `main`.

12. Детальный дневник обучения

Ниже тот же маршрут, но в формате дневника. У каждого дня есть цель, результат, тема C и вопрос, на который ты должен ответить без подсказки. Не обязательно проходить по одному пункту в календарный день. Можно растянуть. Важно, чтобы каждый пункт заканчивался проверяемым состоянием.

День 1. Скелет проекта

Ожидаемый результат: Программа собирается и печатает help.

Тема C: translation unit, `main`, CMake target.

Вопрос дня: Почему `.h` сам по себе не компилируется как программа?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 2. Контракты util

Ожидаемый результат: Есть `util.h` с `Buf` и `StrBuf`.

Тема C: struct, prototypes, include guards.

Вопрос дня: Чем объявление функции отличается от определения?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 3. Безопасное чтение

Ожидаемый результат: `buf_read` не читает за пределами bytes.

Тема C: pointer arithmetic, `size_t`.

Вопрос дня: Почему проверка `len > size - off` безопаснее сложения?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 4. Endian helpers

Ожидаемый результат: Работают `le16/le32/le64`.

Тема C: битовые сдвиги.

Вопрос дня: Почему `0x12 0x34` не всегда число `0x1234`?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 5. Файл в память

Ожидаемый результат: `buf_init_from_file` читает path.

Тема C: `FILE*`, heap.

Вопрос дня: Кто освобождает память файла?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 6. Растущая строка

Ожидаемый результат: `StrBuf` добавляет текст.

Тема C: `realloc`, capacity.

Вопрос дня: Почему нужен отдельный `failed` flag?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 7. Форматирование

Ожидаемый результат: `sb_appendf` работает.

Тема C: `va_list`, `vsnprintf`.

Вопрос дня: Почему format string надо проверять warnings?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 8. Атомарная запись

Ожидаемый результат: Временный файл заменяет итоговый.

Тема C: filesystem failure paths.

Вопрос дня: Что будет, если запись прервется посередине?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 9. PE constants

Ожидаемый результат: Создан `pe.h`.

Тема C: macros, fixed-width types.

Вопрос дня: Почему `IMAGE_FILE_MACHINE_AMD64` лучше числа без имени?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 10. DOS header

Ожидаемый результат: `e_lfanew` читается из 0x3c.

Тема C: offsets.

Вопрос дня: Что означает `MZ`?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 11. PE signature

Ожидаемый результат: Проверяется `PE\0\0`.

Тема C: magic values.

Вопрос дня: Почему нельзя продолжать после неверной сигнатуры?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 12. COFF header

Ожидаемый результат: Читаются machine, sections, optional size.

Тема C: binary structure layout.

Вопрос дня: Почему проект отвергает x86?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 13. Optional header

Ожидаемый результат: Читаются ImageBase, alignments, sizes.

Тема C: PE32+ offsets.

Вопрос дня: Почему optional header здесь не optional?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 14. Data directory

Ожидаемый результат: Exception directory найден.

Тема C: таблицы.

Вопрос дня: Почему exception directory index равен 3?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 15. Sections

Ожидаемый результат: Секции читаются в массив.

Тема C: heap array.

Вопрос дня: Почему имя секции хранится как 8 bytes плюс `\0`?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь

короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 16. PE validation

Ожидаемый результат: Счетчики проблем заполняются.

Тема C: invariants.

Вопрос дня: Чем raw overlap отличается от virtual overlap?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 17. RVA mapping

Ожидаемый результат: RVA переводится в file offset.

Тема C: range mapping.

Вопрос дня: Почему virtual tail не имеет bytes в файле?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 18. Runtime table

Ожидаемый результат: `.pdata` превращается в `RuntimeFunction[]`.

Тема C: array of records.

Вопрос дня: Почему запись 12 bytes?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 19. Unwind header

Ожидаемый результат: Читаются version/flags/count/frame.

Тема C: bit fields.

Вопрос дня: Как из одного byte получить version и flags?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 20. Slots

Ожидаемый результат: Определяется длина opcode.

Тема C: `switch`.

Вопрос дня: Почему цикл идет `i += slots`, а не `i++`?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 21. Stack effects

Ожидаемый результат: Считается fixed stack.

Тема C: overflow-safe addition.

Вопрос дня: Почему `PUSH_MACHFRAME` добавляет 40 или 48 bytes?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 22. Anomalies

Ожидаемый результат: Флаги проблем выставляются.

Тема C: bitmask.

Вопрос дня: Почему можно хранить несколько anomalies в одном числе?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 23. Handler tail

Ожидаемый результат: Читается handler RVA.

Тема C: conditional layout.

Вопрос дня: Почему tail зависит от flags?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 24. Chain tail

Ожидаемый результат: Читается chained runtime function.

Тема C: bounded following.

Вопрос дня: Как проект защищается от chain loop?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 25. Heuristic counters

Ожидаемый результат: Считаются invalid/oob/parse fail.

Тема C: aggregation.

Вопрос дня: Почему сначала observation, потом finding?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 26. Reuse maps

Ожидаемый результат: Считается reuse unwind/handler.

Тема C: hash map.

Вопрос дня: Почему ratio без count опасен?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 27. Findings

Ожидаемый результат: Появляются structured findings.

Тема C: arrays of structs.

Вопрос дня: Что должен содержать хороший finding?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 28. Confidence

Ожидаемый результат: Score отделен от confidence.

Тема C: policy code.

Вопрос дня: Почему score не probability?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 29. CSV

Ожидаемый результат: Пишется обычный CSV.

Тема C: escaping.

Вопрос дня: Почему кавычки в CSV удваиваются?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 30. Ghidra CSV

Ожидаемый результат: Пишется импортируемый CSV.

Тема C: tool integration.

Вопрос дня: Почему имя функции `sub_<RVA>` полезно?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 31. JSON

Ожидаемый результат: Пишется schema-friendly report.

Тема C: JSON escaping.

Вопрос дня: Почему report metadata важна?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 32. HTML

Ожидаемый результат: Пишется offline dashboard.

Тема C: HTML escaping/CSP.

Вопрос дня: Почему нельзя вставлять сырой input в HTML?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 33. Dynamic stub

Ожидаемый результат: Non-Windows path собирается.

Тема C: `#ifdef`.

Вопрос дня: Почему Windows headers нельзя требовать в static mode?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 34. Windows dynamic

Ожидаемый результат: Понятен debug event loop.

Тема C: handles, breakpoints.

Вопрос дня: Почему dynamic mode нужен только в VM/lab?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 35. Main cleanup

Ожидаемый результат: Полный pipeline освобождает ресурсы.

Тема C: lifetime.

Вопрос дня: В каком порядке освобождаются объекты?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

День 36. Тесты

Ожидаемый результат: Запускаются unit/CLI tests.

Тема C: assertions.

Вопрос дня: Как тест превращает bytes в понятный пример?

Рабочий ритуал: сначала перечитай соответствующий header, затем напиши минимальную реализацию, затем добавь проверку плохого input, затем собери проект. В конце дня оставь короткую заметку: что понял, где ошибся, какой invariant теперь держит код.

13. Атлас типичных ошибок

Когда учишь C на реальном парсере, ошибки неизбежны. Важно не пугаться, а уметь распознать класс проблемы. Этот атлас помогает понять, куда смотреть.

Симптом	Вероятная причина	Что проверить
Segmentation fault	Обращение по плохому pointer	Проверить `NULL`, lifetime, выход за массив.
Use after free	Объект освобожден, но pointer еще используется	После `free` не трогать объект; занулять owner при необходимости.
Double free	Один heap pointer освобожден дважды	Назначить одного owner; не освобождать borrowed pointer.
Heap overflow	Запись за выделенную память	Проверить capacity, место под `\0`, индексы loops.
Integer overflow	Сумма/умножение перешло границу типа	Использовать проверки вида `a > max - b`.
Wrong endian	Bytes собраны в неправильном порядке	Проверить `le16/le32/le64` на ручном примере.
Off-by-one	На один byte/element больше или меньше	Проверить half-open ranges `[begin, end)`.
Uninitialized field	Структура не занулена	Добавить `memset` или явную инициализацию.
Leaked allocation	Нет `free` на error path	Пройти все `return` в функции и отметить ownership.
Dangling pointer	Pointer указывает на stack object, который уже исчез	Не возвращать адрес локальной переменной.
Bad format specifier	`printf` тип не совпал с `%`	Включить `-Wformat=2`, приводить типы явно.
CSV broken	Не экранированы запятые/кавычки/переносы	Использовать `csv_cell` для каждого cell.
JSON broken	Сырая строка содержит кавычку или control char	Всегда идти через `json_escape`.
HTML injection	Сырая строка попала в HTML	Всегда идти через `html_escape`.
RVA not mapped	RVA попал в virtual tail или вне sections	Проверить section span и raw_size.
Wrong section span	Использован только virtual_size или raw_size	В проекте часто нужен `max(virtual_size, raw_size)`.
Malformed `.pdata`	Размер exception directory не кратен 12	Вернуть malformed result, не читать partial entry.
Unsupported unwind version	Version не 1 и не 2	Не парсить V3 старым layout.
Opcode spans beyond count	Slots opcode выходят за CountOfCodes	Проверить `i + slots > count`.
Handler tail truncated	Флаг требует tail, но bytes нет	Вернуть parse fail и сохранить parse_error.
Chain loop	Chained records ссылаются по кругу	Хранить seen array и max depth.
False high suspicion	Одна слабая эвристика дала слишком сильный вывод	Проверить weakening rules и context.
Dynamic timeout	Target не вышел до deadline	Ставить timeout, чистить процесс и report state.

Симптом	Вероятная причина	Что проверить
Breakpoint not rearmed	INT3 снят, но не поставлен обратно	Проверить single-step path и pending rearm per thread.
Command line leak	Target args попали в report metadata	После `--` писать redaction marker.

14. Карточки вопросов по функциям

Эти карточки нужны для самопроверки. Берешь функцию из списка, открываешь исходник и отвечаешь на пять вопросов. Если не можешь ответить, не переписывай ее пока механически.

include/util.h

le16

- Какой главный input у `le16` и считается ли он trusted?
- Какой объект `le16` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `le16`?
- Какой тест или assert докажет, что `le16` работает хотя бы на одном плохом input?

le32

- Какой главный input у `le32` и считается ли он trusted?
- Какой объект `le32` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `le32`?
- Какой тест или assert докажет, что `le32` работает хотя бы на одном плохом input?

le64

- Какой главный input у `le64` и считается ли он trusted?
- Какой объект `le64` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `le64`?
- Какой тест или assert докажет, что `le64` работает хотя бы на одном плохом input?

src/util.c

buf_init_from_file

- Какой главный input у `buf_init_from_file` и считается ли он trusted?
- Какой объект `buf_init_from_file` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `buf_init_from_file`?
- Какой тест или assert докажет, что `buf_init_from_file` работает хотя бы на одном плохом input?

buf_free_owned

- Какой главный input у `buf_free_owned` и считается ли он trusted?
- Какой объект `buf_free_owned` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?

- Какой ресурс может потребовать cleanup внутри `buf_free_owned`?
- Какой тест или assert докажет, что `buf_free_owned` работает хотя бы на одном плохом input?

buf_read

- Какой главный input у `buf_read` и считается ли он trusted?
- Какой объект `buf_read` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `buf_read`?
- Какой тест или assert докажет, что `buf_read` работает хотя бы на одном плохом input?

buf_read_u8

- Какой главный input у `buf_read_u8` и считается ли он trusted?
- Какой объект `buf_read_u8` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `buf_read_u8`?
- Какой тест или assert докажет, что `buf_read_u8` работает хотя бы на одном плохом input?

buf_read_u16

- Какой главный input у `buf_read_u16` и считается ли он trusted?
- Какой объект `buf_read_u16` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `buf_read_u16`?
- Какой тест или assert докажет, что `buf_read_u16` работает хотя бы на одном плохом input?

buf_read_u32

- Какой главный input у `buf_read_u32` и считается ли он trusted?
- Какой объект `buf_read_u32` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `buf_read_u32`?
- Какой тест или assert докажет, что `buf_read_u32` работает хотя бы на одном плохом input?

buf_read_u64

- Какой главный input у `buf_read_u64` и считается ли он trusted?
- Какой объект `buf_read_u64` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `buf_read_u64`?
- Какой тест или assert докажет, что `buf_read_u64` работает хотя бы на одном плохом input?

sb_reserve

- Какой главный input у `sb_reserve` и считается ли он trusted?

- Какой объект `sb_reserve` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sb_reserve`?
- Какой тест или assert докажет, что `sb_reserve` работает хотя бы на одном плохом input?

sb_init

- Какой главный input у `sb_init` и считается ли он trusted?
- Какой объект `sb_init` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sb_init`?
- Какой тест или assert докажет, что `sb_init` работает хотя бы на одном плохом input?

sb_free

- Какой главный input у `sb_free` и считается ли он trusted?
- Какой объект `sb_free` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sb_free`?
- Какой тест или assert докажет, что `sb_free` работает хотя бы на одном плохом input?

sb_appendn

- Какой главный input у `sb_appendn` и считается ли он trusted?
- Какой объект `sb_appendn` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sb_appendn`?
- Какой тест или assert докажет, что `sb_appendn` работает хотя бы на одном плохом input?

sb_append

- Какой главный input у `sb_append` и считается ли он trusted?
- Какой объект `sb_append` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sb_append`?
- Какой тест или assert докажет, что `sb_append` работает хотя бы на одном плохом input?

sb_appendf

- Какой главный input у `sb_appendf` и считается ли он trusted?
- Какой объект `sb_appendf` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sb_appendf`?
- Какой тест или assert докажет, что `sb_appendf` работает хотя бы на одном плохом input?

sb_ok

- Какой главный input у `sb_ok` и считается ли он trusted?
- Какой объект `sb_ok` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sb_ok`?
- Какой тест или assert докажет, что `sb_ok` работает хотя бы на одном плохом input?

file_write_atomic

- Какой главный input у `file_write_atomic` и считается ли он trusted?
- Какой объект `file_write_atomic` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `file_write_atomic`?
- Какой тест или assert докажет, что `file_write_atomic` работает хотя бы на одном плохом input?

sha256_rotr

- Какой главный input у `sha256_rotr` и считается ли он trusted?
- Какой объект `sha256_rotr` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sha256_rotr`?
- Какой тест или assert докажет, что `sha256_rotr` работает хотя бы на одном плохом input?

sha256_transform

- Какой главный input у `sha256_transform` и считается ли он trusted?
- Какой объект `sha256_transform` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sha256_transform`?
- Какой тест или assert докажет, что `sha256_transform` работает хотя бы на одном плохом input?

sha256_init

- Какой главный input у `sha256_init` и считается ли он trusted?
- Какой объект `sha256_init` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sha256_init`?
- Какой тест или assert докажет, что `sha256_init` работает хотя бы на одном плохом input?

src/pe.c

u32_max

- Какой главный input у `u32_max` и считается ли он trusted?
- Какой объект `u32_max` изменяет или возвращает?

- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `u32_max`?
- Какой тест или assert докажет, что `u32_max` работает хотя бы на одном плохом input?

is_power_of_two_u32

- Какой главный input у `is_power_of_two_u32` и считается ли он trusted?
- Какой объект `is_power_of_two_u32` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `is_power_of_two_u32`?
- Какой тест или assert докажет, что `is_power_of_two_u32` работает хотя бы на одном плохом input?

read_dos_lfanew

- Какой главный input у `read_dos_lfanew` и считается ли он trusted?
- Какой объект `read_dos_lfanew` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `read_dos_lfanew`?
- Какой тест или assert докажет, что `read_dos_lfanew` работает хотя бы на одном плохом input?

safe_add_u32

- Какой главный input у `safe_add_u32` и считается ли он trusted?
- Какой объект `safe_add_u32` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `safe_add_u32`?
- Какой тест или assert докажет, что `safe_add_u32` работает хотя бы на одном плохом input?

ranges_overlap_u64

- Какой главный input у `ranges_overlap_u64` и считается ли он trusted?
- Какой объект `ranges_overlap_u64` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `ranges_overlap_u64`?
- Какой тест или assert докажет, что `ranges_overlap_u64` работает хотя бы на одном плохом input?

pe_parse

- Какой главный input у `pe_parse` и считается ли он trusted?
- Какой объект `pe_parse` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `pe_parse`?
- Какой тест или assert докажет, что `pe_parse` работает хотя бы на одном плохом input?

pe_free

- Какой главный input у `pe_free` и считается ли он trusted?
- Какой объект `pe_free` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `pe_free`?
- Какой тест или assert докажет, что `pe_free` работает хотя бы на одном плохом input?

pe_rva_to_fileoff

- Какой главный input у `pe_rva_to_fileoff` и считается ли он trusted?
- Какой объект `pe_rva_to_fileoff` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `pe_rva_to_fileoff`?
- Какой тест или assert докажет, что `pe_rva_to_fileoff` работает хотя бы на одном плохом input?

src/unwind.c

unwind_flags_to_string

- Какой главный input у `unwind_flags_to_string` и считается ли он trusted?
- Какой объект `unwind_flags_to_string` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `unwind_flags_to_string`?
- Какой тест или assert докажет, что `unwind_flags_to_string` работает хотя бы на одном плохом input?

slots_for_code

- Какой главный input у `slots_for_code` и считается ли он trusted?
- Какой объект `slots_for_code` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `slots_for_code`?
- Какой тест или assert докажет, что `slots_for_code` работает хотя бы на одном плохом input?

parse_unwindinfo_header

- Какой главный input у `parse_unwindinfo_header` и считается ли он trusted?
- Какой объект `parse_unwindinfo_header` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `parse_unwindinfo_header`?
- Какой тест или assert докажет, что `parse_unwindinfo_header` работает хотя бы на одном плохом input?

read_u16_slot

- Какой главный input у `read_u16_slot` и считается ли он trusted?

- Какой объект `read_u16_slot` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `read_u16_slot`?
- Какой тест или assert докажет, что `read_u16_slot` работает хотя бы на одном плохом input?

read_u32_from_slots

- Какой главный input у `read_u32_from_slots` и считается ли он trusted?
- Какой объект `read_u32_from_slots` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `read_u32_from_slots`?
- Какой тест или assert докажет, что `read_u32_from_slots` работает хотя бы на одном плохом input?

is_unwindable_gpr

- Какой главный input у `is_unwindable_gpr` и считается ли он trusted?
- Какой объект `is_unwindable_gpr` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `is_unwindable_gpr`?
- Какой тест или assert докажет, что `is_unwindable_gpr` работает хотя бы на одном плохом input?

is_unwindable_xmm

- Какой главный input у `is_unwindable_xmm` и считается ли он trusted?
- Какой объект `is_unwindable_xmm` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `is_unwindable_xmm`?
- Какой тест или assert докажет, что `is_unwindable_xmm` работает хотя бы на одном плохом input?

is_abi_nonvolatile_gpr

- Какой главный input у `is_abi_nonvolatile_gpr` и считается ли он trusted?
- Какой объект `is_abi_nonvolatile_gpr` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `is_abi_nonvolatile_gpr`?
- Какой тест или assert докажет, что `is_abi_nonvolatile_gpr` работает хотя бы на одном плохом input?

is_abi_nonvolatile_xmm

- Какой главный input у `is_abi_nonvolatile_xmm` и считается ли он trusted?
- Какой объект `is_abi_nonvolatile_xmm` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `is_abi_nonvolatile_xmm`?

- Какой тест или assert докажет, что `is_abi_nonvolatile_xmm` работает хотя бы на одном плохом input?

add_fixed_stack

- Какой главный input у `add_fixed_stack` и считается ли он trusted?
- Какой объект `add_fixed_stack` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `add_fixed_stack`?
- Какой тест или assert докажет, что `add_fixed_stack` работает хотя бы на одном плохом input?

alloc_large_mode0_suspicious

- Какой главный input у `alloc_large_mode0_suspicious` и считается ли он trusted?
- Какой объект `alloc_large_mode0_suspicious` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `alloc_large_mode0_suspicious`?
- Какой тест или assert докажет, что `alloc_large_mode0_suspicious` работает хотя бы на одном плохом input?

alloc_large_mode1_suspicious

- Какой главный input у `alloc_large_mode1_suspicious` и считается ли он trusted?
- Какой объект `alloc_large_mode1_suspicious` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `alloc_large_mode1_suspicious`?
- Какой тест или assert докажет, что `alloc_large_mode1_suspicious` работает хотя бы на одном плохом input?

save_nonvol_far_suspicious

- Какой главный input у `save_nonvol_far_suspicious` и считается ли он trusted?
- Какой объект `save_nonvol_far_suspicious` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `save_nonvol_far_suspicious`?
- Какой тест или assert докажет, что `save_nonvol_far_suspicious` работает хотя бы на одном плохом input?

save_xmm_far_suspicious

- Какой главный input у `save_xmm_far_suspicious` и считается ли он trusted?
- Какой объект `save_xmm_far_suspicious` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `save_xmm_far_suspicious`?
- Какой тест или assert докажет, что `save_xmm_far_suspicious` работает хотя бы на одном плохом input?

parse_chain_rf

- Какой главный input у `parse_chain_rf` и считается ли он trusted?
- Какой объект `parse_chain_rf` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `parse_chain_rf`?
- Какой тест или assert докажет, что `parse_chain_rf` работает хотя бы на одном плохом input?

unwind_parse_fail

- Какой главный input у `unwind_parse_fail` и считается ли он trusted?
- Какой объект `unwind_parse_fail` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `unwind_parse_fail`?
- Какой тест или assert докажет, что `unwind_parse_fail` работает хотя бы на одном плохом input?

unwind_parse

- Какой главный input у `unwind_parse` и считается ли он trusted?
- Какой объект `unwind_parse` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `unwind_parse`?
- Какой тест или assert докажет, что `unwind_parse` работает хотя бы на одном плохом input?

unwind_read_runtime_functions_ex

- Какой главный input у `unwind_read_runtime_functions_ex` и считается ли он trusted?
- Какой объект `unwind_read_runtime_functions_ex` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `unwind_read_runtime_functions_ex`?
- Какой тест или assert докажет, что `unwind_read_runtime_functions_ex` работает хотя бы на одном плохом input?

src/heuristics.c

sec_name_eq

- Какой главный input у `sec_name_eq` и считается ли он trusted?
- Какой объект `sec_name_eq` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sec_name_eq`?
- Какой тест или assert докажет, что `sec_name_eq` работает хотя бы на одном плохом input?

sec_name_contains_ci

- Какой главный input у `sec_name_contains_ci` и считается ли он trusted?

- Какой объект `sec_name_contains_ci` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sec_name_contains_ci`?
- Какой тест или assert докажет, что `sec_name_contains_ci` работает хотя бы на одном плохом input?

sec_name_allowed_for_unwind

- Какой главный input у `sec_name_allowed_for_unwind` и считается ли он trusted?
- Какой объект `sec_name_allowed_for_unwind` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sec_name_allowed_for_unwind`?
- Какой тест или assert докажет, что `sec_name_allowed_for_unwind` работает хотя бы на одном плохом input?

sec_name_plausibly_unwind_named

- Какой главный input у `sec_name_plausibly_unwind_named` и считается ли он trusted?
- Какой объект `sec_name_plausibly_unwind_named` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sec_name_plausibly_unwind_named`?
- Какой тест или assert докажет, что `sec_name_plausibly_unwind_named` работает хотя бы на одном плохом input?

sec_name_code_like

- Какой главный input у `sec_name_code_like` и считается ли он trusted?
- Какой объект `sec_name_code_like` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `sec_name_code_like`?
- Какой тест или assert докажет, что `sec_name_code_like` работает хотя бы на одном плохом input?

image_has_executable_section

- Какой главный input у `image_has_executable_section` и считается ли он trusted?
- Какой объект `image_has_executable_section` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `image_has_executable_section`?
- Какой тест или assert докажет, что `image_has_executable_section` работает хотя бы на одном плохом input?

h32

- Какой главный input у `h32` и считается ли он trusted?
- Какой объект `h32` изменяет или возвращает?

- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `h32`?
- Какой тест или assert докажет, что `h32` работает хотя бы на одном плохом input?

hmap_init

- Какой главный input у `hmap_init` и считается ли он trusted?
- Какой объект `hmap_init` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `hmap_init`?
- Какой тест или assert докажет, что `hmap_init` работает хотя бы на одном плохом input?

hmap_free

- Какой главный input у `hmap_free` и считается ли он trusted?
- Какой объект `hmap_free` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `hmap_free`?
- Какой тест или assert докажет, что `hmap_free` работает хотя бы на одном плохом input?

hmap_inc

- Какой главный input у `hmap_inc` и считается ли он trusted?
- Какой объект `hmap_inc` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `hmap_inc`?
- Какой тест или assert докажет, что `hmap_inc` работает хотя бы на одном плохом input?

hmap_grow

- Какой главный input у `hmap_grow` и считается ли он trusted?
- Какой объект `hmap_grow` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `hmap_grow`?
- Какой тест или assert докажет, что `hmap_grow` работает хотя бы на одном плохом input?

hmap_max_val

- Какой главный input у `hmap_max_val` и считается ли он trusted?
- Какой объект `hmap_max_val` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `hmap_max_val`?
- Какой тест или assert докажет, что `hmap_max_val` работает хотя бы на одном плохом input?

hmap_max_entry

- Какой главный input у `hmap_max_entry` и считается ли он trusted?
- Какой объект `hmap_max_entry` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `hmap_max_entry`?
- Какой тест или assert докажет, что `hmap_max_entry` работает хотя бы на одном плохом input?

hmap_unique

- Какой главный input у `hmap_unique` и считается ли он trusted?
- Какой объект `hmap_unique` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `hmap_unique`?
- Какой тест или assert докажет, что `hmap_unique` работает хотя бы на одном плохом input?

remember_location

- Какой главный input у `remember_location` и считается ли он trusted?
- Какой объект `remember_location` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `remember_location`?
- Какой тест или assert докажет, что `remember_location` работает хотя бы на одном плохом input?

first_pe_section_issue

- Какой главный input у `first_pe_section_issue` и считается ли он trusted?
- Какой объект `first_pe_section_issue` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `first_pe_section_issue`?
- Какой тест или assert докажет, что `first_pe_section_issue` работает хотя бы на одном плохом input?

cmp_rfidx

- Какой главный input у `cmp_rfidx` и считается ли он trusted?
- Какой объект `cmp_rfidx` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `cmp_rfidx`?
- Какой тест или assert докажет, что `cmp_rfidx` работает хотя бы на одном плохом input?

clamp_int

- Какой главный input у `clamp_int` и считается ли он trusted?
- Какой объект `clamp_int` изменяет или возвращает?

- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `clamp_int`?
- Какой тест или assert докажет, что `clamp_int` работает хотя бы на одном плохом input?

src/report.c

append_regmask

- Какой главный input у `append_regmask` и считается ли он trusted?
- Какой объект `append_regmask` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `append_regmask`?
- Какой тест или assert докажет, что `append_regmask` работает хотя бы на одном плохом input?

anomalies_to_str

- Какой главный input у `anomalies_to_str` и считается ли он trusted?
- Какой объект `anomalies_to_str` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `anomalies_to_str`?
- Какой тест или assert докажет, что `anomalies_to_str` работает хотя бы на одном плохом input?

runtime_function_size

- Какой главный input у `runtime_function_size` и считается ли он trusted?
- Какой объект `runtime_function_size` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `runtime_function_size`?
- Какой тест или assert докажет, что `runtime_function_size` работает хотя бы на одном плохом input?

csv_cell

- Какой главный input у `csv_cell` и считается ли он trusted?
- Какой объект `csv_cell` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `csv_cell`?
- Какой тест или assert докажет, что `csv_cell` работает хотя бы на одном плохом input?

build_unwind_comment

- Какой главный input у `build_unwind_comment` и считается ли он trusted?
- Какой объект `build_unwind_comment` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `build_unwind_comment`?

- Какой тест или assert докажет, что `build_unwind_comment` работает хотя бы на одном плохом input?

report_print_brief

- Какой главный input у `report_print_brief` и считается ли он trusted?
- Какой объект `report_print_brief` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `report_print_brief`?
- Какой тест или assert докажет, что `report_print_brief` работает хотя бы на одном плохом input?

report_write_csv

- Какой главный input у `report_write_csv` и считается ли он trusted?
- Какой объект `report_write_csv` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `report_write_csv`?
- Какой тест или assert докажет, что `report_write_csv` работает хотя бы на одном плохом input?

valid_utf8_sequence_length

- Какой главный input у `valid_utf8_sequence_length` и считается ли он trusted?
- Какой объект `valid_utf8_sequence_length` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `valid_utf8_sequence_length`?
- Какой тест или assert докажет, что `valid_utf8_sequence_length` работает хотя бы на одном плохом input?

json_escape

- Какой главный input у `json_escape` и считается ли он trusted?
- Какой объект `json_escape` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `json_escape`?
- Какой тест или assert докажет, что `json_escape` работает хотя бы на одном плохом input?

json_key

- Какой главный input у `json_key` и считается ли он trusted?
- Какой объект `json_key` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `json_key`?
- Какой тест или assert докажет, что `json_key` работает хотя бы на одном плохом input?

json_string

- Какой главный input у `json_string` и считается ли он trusted?

- Какой объект `json_string` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `json_string`?
- Какой тест или assert докажет, что `json_string` работает хотя бы на одном плохом input?

json_string_field

- Какой главный input у `json_string_field` и считается ли он trusted?
- Какой объект `json_string_field` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `json_string_field`?
- Какой тест или assert докажет, что `json_string_field` работает хотя бы на одном плохом input?

json_bool_field

- Какой главный input у `json_bool_field` и считается ли он trusted?
- Какой объект `json_bool_field` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `json_bool_field`?
- Какой тест или assert докажет, что `json_bool_field` работает хотя бы на одном плохом input?

json_heuristic_families

- Какой главный input у `json_heuristic_families` и считается ли он trusted?
- Какой объект `json_heuristic_families` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `json_heuristic_families`?
- Какой тест или assert докажет, что `json_heuristic_families` работает хотя бы на одном плохом input?

dynamic_signal_count

- Какой главный input у `dynamic_signal_count` и считается ли он trusted?
- Какой объект `dynamic_signal_count` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `dynamic_signal_count`?
- Какой тест или assert докажет, что `dynamic_signal_count` работает хотя бы на одном плохом input?

dynamic_session_complete

- Какой главный input у `dynamic_session_complete` и считается ли он trusted?
- Какой объект `dynamic_session_complete` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `dynamic_session_complete`?

- Какой тест или assert докажет, что `dynamic_session_complete` работает хотя бы на одном плохом input?

has_static_dynamic_correlation

- Какой главный input у `has_static_dynamic_correlation` и считается ли он trusted?
- Какой объект `has_static_dynamic_correlation` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `has_static_dynamic_correlation`?
- Какой тест или assert докажет, что `has_static_dynamic_correlation` работает хотя бы на одном плохом input?

assessment_summary

- Какой главный input у `assessment_summary` и считается ли он trusted?
- Какой объект `assessment_summary` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `assessment_summary`?
- Какой тест или assert докажет, что `assessment_summary` работает хотя бы на одном плохом input?

src/dynamic_win.c

mods_init

- Какой главный input у `mods_init` и считается ли он trusted?
- Какой объект `mods_init` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `mods_init`?
- Какой тест или assert докажет, что `mods_init` работает хотя бы на одном плохом input?

strlower_inplace

- Какой главный input у `strlower_inplace` и считается ли он trusted?
- Какой объект `strlower_inplace` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `strlower_inplace`?
- Какой тест или assert докажет, что `strlower_inplace` работает хотя бы на одном плохом input?

mods_add

- Какой главный input у `mods_add` и считается ли он trusted?
- Какой объект `mods_add` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `mods_add`?
- Какой тест или assert докажет, что `mods_add` работает хотя бы на одном плохом input?

mods_refresh_toolhelp

- Какой главный input у `mods_refresh_toolhelp` и считается ли он trusted?
- Какой объект `mods_refresh_toolhelp` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `mods_refresh_toolhelp`?
- Какой тест или assert докажет, что `mods_refresh_toolhelp` работает хотя бы на одном плохом input?

mods_refresh_accounted

- Какой главный input у `mods_refresh_accounted` и считается ли он trusted?
- Какой объект `mods_refresh_accounted` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `mods_refresh_accounted`?
- Какой тест или assert докажет, что `mods_refresh_accounted` работает хотя бы на одном плохом input?

bps_init

- Какой главный input у `bps_init` и считается ли он trusted?
- Какой объект `bps_init` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `bps_init`?
- Какой тест или assert докажет, что `bps_init` работает хотя бы на одном плохом input?

write_one_byte

- Какой главный input у `write_one_byte` и считается ли он trusted?
- Какой объект `write_one_byte` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `write_one_byte`?
- Какой тест или assert докажет, что `write_one_byte` работает хотя бы на одном плохом input?

bp_set

- Какой главный input у `bp_set` и считается ли он trusted?
- Какой объект `bp_set` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `bp_set`?
- Какой тест или assert докажет, что `bp_set` работает хотя бы на одном плохом input?

bp_clear

- Какой главный input у `bp_clear` и считается ли он trusted?
- Какой объект `bp_clear` изменяет или возвращает?

- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `bp_clear`?
- Какой тест или assert докажет, что `bp_clear` работает хотя бы на одном плохом input?

bp_add_and_arm

- Какой главный input у `bp_add_and_arm` и считается ли он trusted?
- Какой объект `bp_add_and_arm` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `bp_add_and_arm`?
- Какой тест или assert докажет, что `bp_add_and_arm` работает хотя бы на одном плохом input?

bp_peek_byte

- Какой главный input у `bp_peek_byte` и считается ли он trusted?
- Какой объект `bp_peek_byte` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `bp_peek_byte`?
- Какой тест или assert докажет, что `bp_peek_byte` работает хотя бы на одном плохом input?

format_bp_location

- Какой главный input у `format_bp_location` и считается ли он trusted?
- Какой объект `format_bp_location` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `format_bp_location`?
- Какой тест или assert докажет, что `format_bp_location` работает хотя бы на одном плохом input?

bp_add_and_arm_selfcheck

- Какой главный input у `bp_add_and_arm_selfcheck` и считается ли он trusted?
- Какой объект `bp_add_and_arm_selfcheck` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `bp_add_and_arm_selfcheck`?
- Какой тест или assert докажет, что `bp_add_and_arm_selfcheck` работает хотя бы на одном плохом input?

pending_rearm_init

- Какой главный input у `pending_rearm_init` и считается ли он trusted?
- Какой объект `pending_rearm_init` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `pending_rearm_init`?
- Какой тест или assert докажет, что `pending_rearm_init` работает хотя бы на одном плохом input?

pending_rearm_set

- Какой главный input у `pending_rearm_set` и считается ли он trusted?
- Какой объект `pending_rearm_set` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `pending_rearm_set`?
- Какой тест или assert докажет, что `pending_rearm_set` работает хотя бы на одном плохом input?

open_event_thread

- Какой главный input у `open_event_thread` и считается ли он trusted?
- Какой объект `open_event_thread` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `open_event_thread`?
- Какой тест или assert докажет, что `open_event_thread` работает хотя бы на одном плохом input?

close_event_thread

- Какой главный input у `close_event_thread` и считается ли он trusted?
- Какой объект `close_event_thread` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `close_event_thread`?
- Какой тест или assert докажет, что `close_event_thread` работает хотя бы на одном плохом input?

dr_append

- Какой главный input у `dr_append` и считается ли он trusted?
- Какой объект `dr_append` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `dr_append`?
- Какой тест или assert докажет, что `dr_append` работает хотя бы на одном плохом input?

src/app_info.c

app_utc_timestamp

- Какой главный input у `app_utc_timestamp` и считается ли он trusted?
- Какой объект `app_utc_timestamp` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `app_utc_timestamp`?
- Какой тест или assert докажет, что `app_utc_timestamp` работает хотя бы на одном плохом input?

src/main.c

print_version

- Какой главный input у `print_version` и считается ли он trusted?
- Какой объект `print_version` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `print_version`?
- Какой тест или assert докажет, что `print_version` работает хотя бы на одном плохом input?

usage

- Какой главный input у `usage` и считается ли он trusted?
- Какой объект `usage` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `usage`?
- Какой тест или assert докажет, что `usage` работает хотя бы на одном плохом input?

append_invocation_arg

- Какой главный input у `append_invocation_arg` и считается ли он trusted?
- Какой объект `append_invocation_arg` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `append_invocation_arg`?
- Какой тест или assert докажет, что `append_invocation_arg` работает хотя бы на одном плохом input?

build_invocation

- Какой главный input у `build_invocation` и считается ли он trusted?
- Какой объект `build_invocation` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `build_invocation`?
- Какой тест или assert докажет, что `build_invocation` работает хотя бы на одном плохом input?

parse_long_range

- Какой главный input у `parse_long_range` и считается ли он trusted?
- Какой объект `parse_long_range` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `parse_long_range`?
- Какой тест или assert докажет, что `parse_long_range` работает хотя бы на одном плохом input?

parse_u32_range

- Какой главный input у `parse_u32_range` и считается ли он trusted?
- Какой объект `parse_u32_range` изменяет или возвращает?

- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `parse_u32_range`?
- Какой тест или assert докажет, что `parse_u32_range` работает хотя бы на одном плохом input?

parse_args

- Какой главный input у `parse_args` и считается ли он trusted?
- Какой объект `parse_args` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `parse_args`?
- Какой тест или assert докажет, что `parse_args` работает хотя бы на одном плохом input?

print_one

- Какой главный input у `print_one` и считается ли он trusted?
- Какой объект `print_one` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `print_one`?
- Какой тест или assert докажет, что `print_one` работает хотя бы на одном плохом input?

main

- Какой главный input у `main` и считается ли он trusted?
- Какой объект `main` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `main`?
- Какой тест или assert докажет, что `main` работает хотя бы на одном плохом input?

tests/unit/unit_unwind.c

make_fake_img_1sec

- Какой главный input у `make_fake_img_1sec` и считается ли он trusted?
- Какой объект `make_fake_img_1sec` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `make_fake_img_1sec`?
- Какой тест или assert докажет, что `make_fake_img_1sec` работает хотя бы на одном плохом input?

test_alloc_small

- Какой главный input у `test_alloc_small` и считается ли он trusted?
- Какой объект `test_alloc_small` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_alloc_small`?
- Какой тест или assert докажет, что `test_alloc_small` работает хотя бы на одном плохом input?

test_push_nonvol

- Какой главный input у `test_push_nonvol` и считается ли он trusted?
- Какой объект `test_push_nonvol` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_push_nonvol`?
- Какой тест или assert докажет, что `test_push_nonvol` работает хотя бы на одном плохом input?

test_codeoffset_order_and_operand_sanity

- Какой главный input у `test_codeoffset_order_and_operand_sanity` и считается ли он trusted?
- Какой объект `test_codeoffset_order_and_operand_sanity` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_codeoffset_order_and_operand_sanity`?
- Какой тест или assert докажет, что `test_codeoffset_order_and_operand_sanity` работает хотя бы на одном плохом input?

test_real_world_compatible_operands

- Какой главный input у `test_real_world_compatible_operands` и считается ли он trusted?
- Какой объект `test_real_world_compatible_operands` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_real_world_compatible_operands`?
- Какой тест или assert докажет, что `test_real_world_compatible_operands` работает хотя бы на одном плохом input?

test_strict_abi_push_order_and_chain_restriction

- Какой главный input у `test_strict_abi_push_order_and_chain_restriction` и считается ли он trusted?
- Какой объект `test_strict_abi_push_order_and_chain_restriction` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_strict_abi_push_order_and_chain_restriction`?
- Какой тест или assert докажет, что `test_strict_abi_push_order_and_chain_restriction` работает хотя бы на одном плохом input?

test_v2_epilog_descriptors

- Какой главный input у `test_v2_epilog_descriptors` и считается ли он trusted?
- Какой объект `test_v2_epilog_descriptors` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_v2_epilog_descriptors`?
- Какой тест или assert докажет, что `test_v2_epilog_descriptors` работает хотя бы на одном плохом input?

test_chain_inherits_frame_and_handler

- Какой главный input у `test_chain_inherits_frame_and_handler` и считается ли он trusted?
- Какой объект `test_chain_inherits_frame_and_handler` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_chain_inherits_frame_and_handler`?
- Какой тест или assert докажет, что `test_chain_inherits_frame_and_handler` работает хотя бы на одном плохом input?

test_flag_and_frame_sanity

- Какой главный input у `test_flag_and_frame_sanity` и считается ли он trusted?
- Какой объект `test_flag_and_frame_sanity` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_flag_and_frame_sanity`?
- Какой тест или assert докажет, что `test_flag_and_frame_sanity` работает хотя бы на одном плохом input?

test_parse_error_is_preserved

- Какой главный input у `test_parse_error_is_preserved` и считается ли он trusted?
- Какой объект `test_parse_error_is_preserved` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_parse_error_is_preserved`?
- Какой тест или assert докажет, что `test_parse_error_is_preserved` работает хотя бы на одном плохом input?

test_unsupported_version_is_fatal

- Какой главный input у `test_unsupported_version_is_fatal` и считается ли он trusted?
- Какой объект `test_unsupported_version_is_fatal` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_unsupported_version_is_fatal`?
- Какой тест или assert докажет, что `test_unsupported_version_is_fatal` работает хотя бы на одном плохом input?

test_truncated_handler_tail_is_fatal

- Какой главный input у `test_truncated_handler_tail_is_fatal` и считается ли он trusted?
- Какой объект `test_truncated_handler_tail_is_fatal` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_truncated_handler_tail_is_fatal`?
- Какой тест или assert докажет, что `test_truncated_handler_tail_is_fatal` работает хотя бы на одном плохом input?

test_chain_validation_and_alignment

- Какой главный input у `test_chain_validation_and_alignment` и считается ли он trusted?
- Какой объект `test_chain_validation_and_alignment` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_chain_validation_and_alignment`?
- Какой тест или assert докажет, что `test_chain_validation_and_alignment` работает хотя бы на одном плохом input?

main

- Какой главный input у `main` и считается ли он trusted?
- Какой объект `main` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `main`?
- Какой тест или assert докажет, что `main` работает хотя бы на одном плохом input?

tests/unit/unit_heuristics.c

setup_sections

- Какой главный input у `setup_sections` и считается ли он trusted?
- Какой объект `setup_sections` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `setup_sections`?
- Какой тест или assert докажет, что `setup_sections` работает хотя бы на одном плохом input?

main

- Какой главный input у `main` и считается ли он trusted?
- Какой объект `main` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `main`?
- Какой тест или assert докажет, что `main` работает хотя бы на одном плохом input?

tests/unit/unit_core_edges.c

finding_index

- Какой главный input у `finding_index` и считается ли он trusted?
- Какой объект `finding_index` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `finding_index`?
- Какой тест или assert докажет, что `finding_index` работает хотя бы на одном плохом input?

test_section_range_wrap

- Какой главный input у `test_section_range_wrap` и считается ли он trusted?
- Какой объект `test_section_range_wrap` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_section_range_wrap`?
- Какой тест или assert докажет, что `test_section_range_wrap` работает хотя бы на одном плохом input?

test_header_mapping_uses_declared_size

- Какой главный input у `test_header_mapping_uses_declared_size` и считается ли он trusted?
- Какой объект `test_header_mapping_uses_declared_size` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_header_mapping_uses_declared_size`?
- Какой тест или assert докажет, что `test_header_mapping_uses_declared_size` работает хотя бы на одном плохом input?

test_empty_exception_directory

- Какой главный input у `test_empty_exception_directory` и считается ли он trusted?
- Какой объект `test_empty_exception_directory` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_empty_exception_directory`?
- Какой тест или assert докажет, что `test_empty_exception_directory` работает хотя бы на одном плохом input?

test_malformed_exception_directory_status

- Какой главный input у `test_malformed_exception_directory_status` и считается ли он trusted?
- Какой объект `test_malformed_exception_directory_status` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_malformed_exception_directory_status`?
- Какой тест или assert докажет, что `test_malformed_exception_directory_status` работает хотя бы на одном плохом input?

test_empty_heuristic_input

- Какой главный input у `test_empty_heuristic_input` и считается ли он trusted?
- Какой объект `test_empty_heuristic_input` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_empty_heuristic_input`?
- Какой тест или assert докажет, что `test_empty_heuristic_input` работает хотя бы на одном плохом input?

test_empty_nonexec_image_is_not_flagged

- Какой главный input у `test_empty_nonexec_image_is_not_flagged` и считается ли он trusted?
- Какой объект `test_empty_nonexec_image_is_not_flagged` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_empty_nonexec_image_is_not_flagged`?
- Какой тест или assert докажет, что `test_empty_nonexec_image_is_not_flagged` работает хотя бы на одном плохом input?

test_nested_overlap_detection

- Какой главный input у `test_nested_overlap_detection` и считается ли он trusted?
- Какой объект `test_nested_overlap_detection` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_nested_overlap_detection`?
- Какой тест или assert докажет, что `test_nested_overlap_detection` работает хотя бы на одном плохом input?

test_sha256

- Какой главный input у `test_sha256` и считается ли он trusted?
- Какой объект `test_sha256` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_sha256`?
- Какой тест или assert докажет, что `test_sha256` работает хотя бы на одном плохом input?

test_empty_file_and_string_overflow

- Какой главный input у `test_empty_file_and_string_overflow` и считается ли он trusted?
- Какой объект `test_empty_file_and_string_overflow` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_empty_file_and_string_overflow`?
- Какой тест или assert докажет, что `test_empty_file_and_string_overflow` работает хотя бы на одном плохом input?

test_pe_structure_findings

- Какой главный input у `test_pe_structure_findings` и считается ли он trusted?
- Какой объект `test_pe_structure_findings` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `test_pe_structure_findings`?
- Какой тест или assert докажет, что `test_pe_structure_findings` работает хотя бы на одном плохом input?

main

- Какой главный input у `main` и считается ли он trusted?

- Какой объект `main` изменяет или возвращает?
- Какая первая проверка должна защитить функцию от плохих аргументов?
- Какой ресурс может потребовать cleanup внутри `main`?
- Какой тест или assert докажет, что `main` работает хотя бы на одном плохом input?

15. Тестирование как способ понять проект

Тесты здесь особенно ценны для новичка: они уменьшают формат PE/UNWIND_INFO до маленьких искусственных массивов bytes. Если настоящий EXE кажется огромным, тест показывает минимальную ситуацию: вот bytes, вот section mapping, вот ожидаемый stack size.

Типы тестов

Тип	Что проверяет	Как использовать при обучении
unit_unwind	slots, opcodes, chain, V2 epilog, parse errors	Разбирать один тест перед написанием соответствующего opcode.
unit_heuristics	score, placement, verdict	Проверять, что finding появляется только при нужном контексте.
unit_core_edges	empty tables, overflow, mapping, SHA-256	Учиться думать о границах и пустых входах.
unit_report_json/html	serialization, escaping, schema-ish invariants	Проверять output как контракт.
cli_contract	exit codes, options, redaction, partial reports	Смотреть на программу глазами пользователя.
mutation_fuzz	устойчивость к поломанным bytes	Понять, что parser не должен падать на мусоре.

Как писать свой первый тест

1. Создай маленький `uint8_t file[0x600]` и занули его.
2. Опиши одну fake section: `.xdata`, virtual address `0x1000`, raw ptr `0x200`, raw size `0x200`.
3. Положи 4 bytes `UNWIND_INFO` header по raw offset `0x200`.
4. Положи один unwind slot.
5. Создай `Buf` и `PeImage` вручную.
6. Вызови `unwind_parse`.
7. Проверь `assert(ok)`, `assert(us.fixed_stack_bytes == ...)`, `assert(us.anomalies == 0)`.

```
Мини-идея теста:
  RVA 0x1000 -> file offset 0x200
  UNWIND_INFO at 0x200:
    version=1, flags=0
    count=1
    one slot: UWOP_ALLOC_SMALL, opinfo=0
  Ожидаем fixed_stack_bytes = 8
```

Как отлаживать

- Если parser вернул false, сначала распечатай `err.s` и `us.parse_error`.

- Если mapping не работает, вручную посчитай `delta = rva - section.virtual_address`.

- Если JSON сломан, проверь escaping до записи файла.

- Если test падает на Windows-only dynamic, отдели static tests от dynamic smoke.

- Если warning стал error, сначала пойми предупреждение, не глуши его флагами.

16. Как читать реальные строки проекта

Шаблон guard

Почти каждая публичная функция защищает свои входы. Для новичка это первый признак серьезного C-кода.

```
if (!out || !b || !err) return false;

Перевод:
  если вызывающий код не дал место для результата,
  или не дал буфер,
  или не дал буфер ошибок,
  функция не имеет права продолжать.
```

Шаблон zero-init

```
memset(out, 0, sizeof(*out));

Перевод:
  перед заполнением результата делаем все поля предсказуемыми.
  bool станут false, указатели NULL, числа 0.
```

Шаблон checked read

```
uint32_t lfanew = 0;
if (!buf_read_u32(b, 0x3c, &lfanew)) return false;

Перевод:
  хотим прочитать 4 bytes из offset 0x3c;
  если bytes нет, не продолжаем;
  если есть, little-endian число попадет в lfanew.
```

Шаблон enum flags

```
enum UnwindAnom {
    UA_TRUNCATED = 1u << 1,
    UA_BAD_OPCODE = 1u << 2
};

out->anomalies |= UA_TRUNCATED;

Перевод:
  anomalies - это набор битов.
  Можно хранить несколько проблем в одном uint32_t.
```

Шаблон cleanup

```
free(us_ok);
free(us);
free(rf);
pe_free(&img);
sb_free(&err);
buf_free_owned(owned);

Перевод:
  main завершает владение всеми объектами,
  которые создавались на этапах pipeline.
```

Главная мыслительная привычка

В C строка кода почти всегда делает одно из пяти: проверяет, читает, преобразует, сохраняет, освобождает. Если ты не можешь отнести строку к одной из этих ролей, остановись и подпиши ее на бумаге.

17. Приложение: карта функций

Эта карта автоматически извлечена из текущих исходников. Используй ее как навигатор: сначала пойми публичные функции из headers, затем переходи к static helpers внутри `.c`.

CMakeLists.txt

Размер: 288 строк, 9844 bytes.

Функции не извлечены или файл является конфигурацией/данными.

CMakePresets.json

Размер: 113 строк, 2179 bytes.

Функции не извлечены или файл является конфигурацией/данными.

include/util.h

Размер: 73 строк, 2152 bytes.

- le16, le32, le64

include/pe.h

Размер: 68 строк, 1636 bytes.

Функции не извлечены или файл является конфигурацией/данными.

include/unwind.h

Размер: 115 строк, 3582 bytes.

Функции не извлечены или файл является конфигурацией/данными.

include/heuristics.h

Размер: 110 строк, 2944 bytes.

Функции не извлечены или файл является конфигурацией/данными.

include/report.h

Размер: 73 строк, 2172 bytes.

Функции не извлечены или файл является конфигурацией/данными.

include/dynamic.h

Размер: 123 строк, 3473 bytes.

Функции не извлечены или файл является конфигурацией/данными.

include/app_info.h

Размер: 25 строк, 497 bytes.

Функции не извлечены или файл является конфигурацией/данными.

src/util.c

Размер: 349 строк, 10759 bytes.

- buf_init_from_file, buf_free_owned, buf_read, buf_read_u8, buf_read_u16, buf_read_u32
- buf_read_u64, sb_reserve, sb_init, sb_free, sb_appendn, sb_append
- sb_appendf, sb_ok, file_write_atomic, sha256_rotr, sha256_transform, sha256_init
- sha256_update, sha256_final, sha256_hex

src/pe.c

Размер: 302 строк, 10716 bytes.

- u32_max, is_power_of_two_u32, read_dos_lfanew, safe_add_u32, ranges_overlap_u64, pe_parse
- pe_free, pe_rva_to_fileoff

src/unwind.c

Размер: 682 строк, 29308 bytes.

- unwind_flags_to_string, slots_for_code, parse_unwindinfo_header, read_u16_slot, read_u32_from_slots, is_unwindable_gpr
- is_unwindable_xmm, is_abi_nonvolatile_gpr, is_abi_nonvolatile_xmm, add_fixed_stack, alloc_large_mode0_suspicious, alloc_large_mode1_suspicious
- save_nonvol_far_suspicious, save_xmm_far_suspicious, parse_chain_rf, unwind_parse_fail, unwind_parse, unwind_read_runtime_functions_ex
- unwind_read_runtime_functions

src/heuristics.c

Размер: 1045 строк, 44158 bytes.

- sec_name_eq, sec_name_contains_ci, sec_name_allowed_for_unwind, sec_name_plausibly_unwind_named, sec_name_code_like, image_has_executable_section
- h32, hmap_init, hmap_free, hmap_inc, hmap_grow, hmap_max_val
- hmap_max_entry, hmap_unique, remember_location, first_pe_section_issue, cmp_rfidx, clamp_int
- add_score, safe_ratio, heuristics_evidence_family_count, family_bit_from_name, safe_copy, annotate_finding

src/report.c

Размер: 1364 строк, 68700 bytes.

- append_regmask, anomalies_to_str, runtime_function_size, csv_cell, build_unwind_comment, report_print_brief
- report_write_csv, valid_utf8_sequence_length, json_escape, json_key, json_string, json_string_field
- json_bool_field, json_heuristic_families, dynamic_signal_count, dynamic_session_complete, has_static_dynamic_correlation, assessment_summary
- heuristics_has_finding_code, heuristics_has_family, collect_manual_validation_steps, json_manual_validation_steps, report_print_heuristics, report_metadata_valid

src/dynamic_win.c

Размер: 1254 строк, 50700 bytes.

- mods_init, strlower_inplace, mods_add, mods_refresh_toolhelp, mods_refresh_accounted, bps_init
- write_one_byte, bp_set, bp_clear, bp_add_and_arm, bp_peek_byte, format_bp_location
- bp_add_and_arm_selfcheck, pending_rearm_init, pending_rearm_set, open_event_thread, close_event_thread, dr_append
- dynamic_event_add, dynamic_report_init, dynamic_report_free, append_backslashes, append_cmd_arg, build_cmdline

src/app_info.c

Размер: 69 строк, 1553 bytes.

- app_utc_timestamp

src/main.c

Размер: 528 строк, 19091 bytes.

- print_version, usage, append_invocation_arg, build_invocation, parse_long_range, parse_u32_range
- parse_args, print_one, main

tests/unit/unit_unwind.c

Размер: 515 строк, 15912 bytes.

- make_fake_img_1sec, test_alloc_small, test_push_nonvol, test_codeoffset_order_and_operand_sanity, test_real_world_compatible_operands, test_strict_abi_push_order_and_chain_restriction
- test_v2_epilog_descriptors, test_chain_inherits_frame_and_handler, test_flag_and_frame_sanity, test_parse_error_is_preserved, test_unsupported_version_is_fatal, test_truncated_handler_tail_is_fatal
- test_chain_validation_and_alignment, main

tests/unit/unit_heuristics.c

Размер: 82 строк, 2577 bytes.

- setup_sections, main

tests/unit/unit_core_edges.c

Размер: 284 строк, 8386 bytes.

- finding_index, test_section_range_wrap, test_header_mapping_uses_declared_size, test_empty_exception_directory, test_malformed_exception_directory_status, test_empty_heuristic_input
- test_empty_nonexec_image_is_not_flagged, test_nested_overlap_detection, test_sha256, test_empty_file_and_string_overflow, test_pe_structure_findings, main

tests/test_cli_contract.py

Размер: 220 строк, 8986 bytes.

Функции не извлечены или файл является конфигурацией/данными.

18. Глоссарий

Глоссарий лучше читать дважды: первый раз до реализации, чтобы не пугаться терминов, второй раз после `unwind.c`, когда слова начнут цепляться за конкретные строки кода.

Термин	Определение
ABI	Application Binary Interface: правила вызовов, регистров, stack alignment и unwind metadata для бинарного кода.
Address	Число, указывающее место в памяти или файле. В PE важно различать virtual address, RVA и file offset.
Alignment	Требование, чтобы адрес или размер был кратен некоторому числу, например 4, 8, 0x200 или 0x1000.
Anomaly	Структурное отклонение, которое проект фиксирует как бит в `UnwindSummary.anomalies`.
Atomic write	Запись через временный файл и последующую замену, чтобы не оставить наполовину записанный отчет.
Breakpoint	Остановка выполнения. В dynamic collector часто ставится INT3 byte `0xCC`.
Buf	Пара `data + size`, безопасная обертка над bytes входного файла.
CHAININFO	Флаг `UNWIND_INFO`, означающий, что хвост записи содержит chained `RUNTIME_FUNCTION`.
CLI	Command Line Interface: способ управлять программой через аргументы командной строки.
COFF header	Часть PE header, где лежат machine type, число секций и размер optional header.
Compiler	Программа, превращающая C-код в object files.
Confidence	Уверенность в интерпретации evidence. В проекте это rule-based число, не статистическая вероятность.
CountOfCodes	Поле `UNWIND_INFO`, показывающее число 2-байтовых slots unwind code array.
CSV	Табличный текстовый формат. В проекте есть обычный CSV и Ghidra CSV.
Data directory	Массив записей optional header PE. Exception directory имеет индекс 3.
Debug API	Windows API для запуска процесса под debugger и получения debug events.
Dynamic mode	Режим, который запускает target и собирает runtime evidence. Требует изоляции для недоверенных файлов.
EHANDLER	Флаг `UNWIND_INFO`, означающий наличие exception handler RVA в tail.
Evidence family	Группа родственных findings: table integrity, unwind integrity, placement, reuse, handler dominance.
File offset	Смещение в файле от начала bytes. Не равно RVA в общем случае.
Finding	Структурированное наблюдение: code, family, severity, message, affected count, representative RVA.
Frame register	Регистр, используемый как frame pointer в unwind metadata.
Ghidra CSV	CSV, который importer использует для создания функций, комментариев и bookmarks в Ghidra.
Handler	Адрес обработчика исключения, связанный с unwind metadata или runtime exception API.
Hash map	Таблица key -> value. В heuristics нужна для подсчета reuse одинаковых RVA.
Header	Файл `.h`, где описываются структуры, константы и прототипы функций.
Heap	Область памяти для `malloc/calloc/realloc`, которую надо освобождать через `free`.

Термин	Определение
ImageBase	Базовый виртуальный адрес, по которому PE предпочитает быть загруженным.
INT3	Однобайтовая x86/x64 инструкция breakpoint, значение `0xCC`.
JSON Schema	Формальный контракт JSON report, по которому валидируются поля и типы.
Leaf function	Функция, которой может не требоваться unwind entry, если она не меняет stack опасным образом.
Linker	Собирает объектные файлы в executable и связывает вызовы функций с их реализациями.
Little-endian	Порядок байтов, где младший байт числа лежит раньше старшего.
Machine	Поле COFF header. Для проекта допустимо только `IMAGE_FILE_MACHINE_AMD64`.
Metadata placement	Где именно лежит unwind или handler metadata: `.xdata`, `.rdata`, `.pdata`, `.text`, writable section и т.д.
Mitigation	В коде эвристик - weakening rule, которая снижает confidence при слабом или изолированном evidence.
MZ	DOS signature в начале PE-файла.
Optional header	PE header с ImageBase, alignment, SizeOfImage, data directories. Для PE32+ вовсе не optional для загрузчика.
Out of bounds	Ссылка или диапазон выходит за допустимые размеры файла или image.
PE32+	64-битный вариант PE optional header, magic `0x20b`.
RUNTIME_FUNCTION	12-байтовая запись `.pdata`: BeginAddress, EndAddress, UnwindInfoAddress.
RVA	Relative Virtual Address: адрес относительно ImageBase.
Section	Область PE с virtual range, raw file range и permissions.
SizeOfImage	Размер виртуального образа PE в памяти.
StrBuf	Растущая строка проекта: pointer, length, capacity, failed flag.
Strict ABI	Дополнительный режим диагностики формальных правил Microsoft x64 unwind ABI.
Suspicion score	Оценка подозрительности 0..100. Это triage, а не доказательство.
UHANDLER	Флаг `UNWIND_INFO`, связанный с unwind/termination handler.
UNWIND_INFO	Структура Windows x64, описывающая prolog, stack effects, saved registers, handlers и chain.
UWOP_ALLOC_SMALL	Unwind opcode, кодирующий небольшое выделение stack frame.
UWOP_PUSH_NONVOL	Unwind opcode, описывающий push nonvolatile register.
Validation	Проверка инвариантов и границ перед тем, как доверять данным.
VEH	Vectored Exception Handler, runtime exception mechanism Windows.
VCH	Vectored Continue Handler.
UEF	Unhandled Exception Filter.

19. Финальные чеклисты

Перед тем как считать свой повтор проекта завершенным

- Ты можешь собрать debug и release.
- Ты можешь объяснить, почему `Buf` хранит size рядом с pointer.
- Ты можешь вручную перевести хотя бы один RVA в file offset.
- Ты можешь прочитать одну `RUNTIME_FUNCTION` entry из hex bytes.
- Ты можешь объяснить, почему `CountOfCodes` округляется до even slots для tail.
- Ты можешь назвать хотя бы пять anomaly flags и условия их установки.
- Ты можешь объяснить разницу score и confidence.
- Ты можешь показать, где JSON escaping защищает report.
- Ты можешь объяснить, почему dynamic mode нельзя запускать на недоверенном файле без изоляции.
- Ты можешь пройти один unit test от bytes до assertions.

Вопросы для самозащиты проекта

1. Почему проект ограничен PE32+ AMD64?
2. Почему static mode cross-platform, a dynamic mode Windows-only?
3. Почему `UNWIND_INFO` V3 не парсится как V1/V2?
4. Почему output JSON имеет schema version?
5. Почему score не является вероятностью?
6. Какие false positives возможны у placement heuristics?
7. Что делает `--strict-abi` и почему он не повышает score сам по себе?
8. Как проект защищается от out-of-bounds чтения?
9. Как dynamic collector ставит и перевзводит breakpoints?
10. Что будет, если exception directory unreadable, но пользователь запросил JSON?

Последнее напутствие

Самый полезный способ учить C на этом проекте - каждый день оставлять после себя работающий маленький инвариант. Сегодня безопасно читаются bytes. Завтра мапится RVA. Потом декодируется один unwind slot. Потом появляется один finding. Так большой системный проект перестает быть монолитом и становится цепочкой понятных побед.